

Resumen para el 2do parcial 2.

Oficial Tanenbaum Haters Club

Recomendación: Estudiar una unidad por día.

Gem con este resumen: <cuando lo haga>



Contenido

Resumen para el 2do parcial 2.	1
Unidad 5: I/O (Entrada / Salida) Clase 13	6
Dispositivos de I/O	6
Controladores de dispositivos	6
Técnicas para realizar la E/S	6
Evolución del sistema de E/S	7
E/S por asignación de memoria	7
DMA: Direct Memory Access	8
Interrupciones	9
Objetivos del diseño de E/S en un SO	9
Capas del software de E/S	9
Drivers de dispositivos	10
Software de E/S independiente del dispositivo	10
Uso de búfer	11
Reporte de errores	11
Asignación y liberación de dispositivos dedicados	11
Software de E/S en espacio de usuario	12
Unidad 5 – Clase 14 – Discos (Como dispositivos de E/S)	12
Hardware Disco	12
RAID	13
CD-ROMs	14
DVD	14
Formato de Disco	15
Algoritmos de programación del brazo del disco.	15
<i>Algoritmo de planificación de disco de la búsqueda de trabajo más corta primero.</i>	16
<i>Algoritmo del ascensor (elevador)</i>	16
Manejo de errores	16
<i>Cuando el SO se hace cargo</i>	17
<i>Errores inducidos por el brazo</i>	17

Almacenamiento estable	17
<i>Fallas por parte del CPU</i>	18
Unidad 5 - Clase 15 Entrada / Salida	19
Relojes – Temporizadores	19
<i>Software de reloj</i>	19
<i>Temporizadores</i>	20
Interfaces de usuario – Teclados, mouse, etc.	20
<i>Software de teclado</i>	20
<i>Software de entrada</i>	20
<i>Software del mouse</i>	21
<i>Ventanas de texto</i>	21
<i>Sistema X Window</i>	22
<i>Interfaces gráficas</i>	23
<i>Mapas de bits</i>	23
<i>Tipos de Letras</i>	23
Clientes delegados	24
Administración de Energía	24
<i>Hardware</i>	24
<i>Pantalla</i>	25
<i>Disco duro</i>	25
<i>CPU</i>	25
<i>Memoria</i>	25
<i>Comunicación inalámbrica</i>	25
<i>Administración térmica</i>	25
<i>Administración de baterías</i>	25
<i>Interfaz de drivers</i>	25
Unidad 6: Clase 16.1 Interbloqueos	25
Recursos	26
Grafos de asignación de recursos.	28
Condiciones para el interbloqueo	29
Prevención del interbloqueo	30
Unidad 6: Clase 16.2	30
Predicción del Interbloqueo	30
<i>Conceptos Clave para la Predicción</i>	30
<i>Denegación de la Iniciación del Proceso</i>	31
<i>Denegación de Asignación de Recursos (Algoritmo del Banquero)</i>	31

<i>Restricciones de la Predicción:</i>	33
Detección del Interbloqueo	33
<i>Algoritmo de Detección</i>	33
<i>Recuperación</i>	34
Estrategia Integrada de Tratamiento del Interbloqueo	34
<i>Clases de Recursos y Estrategias Sugeridas:</i>	35
El Problema de los Filósofos Comensales	35
<i>Solución Utilizando Semáforos (Primera Solución)</i>	35
<i>Solución sin Interbloqueo (Segunda Solución)</i>	35
<i>Solución Utilizando un Monitor</i>	36
Unidad 6: Clase 16.3 Interbloqueos	36
Mecanismos de Concurrencia de UNIX	36
Mecanismos de Concurrencia del Núcleo de Linux	38
<i>Operaciones Atómicas</i>	38
<i>Cerrosos Cíclicos (Spinlocks)</i>	39
<i>Cerrosos Cíclico de Lectura-Escritura:</i>	40
<i>Semáforos en el Núcleo</i>	40
<i>Barreras</i>	41
Funciones de Sincronización de Hilos de Solaris	41
Mecanismos de Concurrencia de Windows	42
<i>Funciones de Espera</i>	42
<i>Objetos de Sincronización</i>	43
<i>Objetos de Sección Crítica</i>	44
Unidad 7: Clase 17 Seguridad: Amenazas y Protección.	44
Requisitos y Amenazas de Seguridad	44
<i>Requisitos de Seguridad</i>	44
<i>Tipos de Peligros (Ataques)</i>	44
Peligros en los Componentes de un Sistema	45
<i>Peligros por Componente y Requisito</i>	46
<i>Ataques a Redes (Líneas de Comunicación)</i>	46
<i>Ataques pasivos</i>	46
<i>Ataque Activo:</i>	46
Protección y Control de Acceso	46
<i>Niveles de Protección</i>	47
<i>Protección de la Memoria</i>	47
<i>Control de Acceso Orientado a Usuario</i>	47

<i>Control de Acceso Orientado a los Datos</i>	47
Mecanismos de Protección	48
<i>Dominios de Protección</i>	48
<i>Listas de Control de Acceso (ACLs)</i>	49
<i>Capacidades (Capabilities)</i>	49
Unidad 7: Clase 18 Intrusos y Software Malicioso	50
Intrusos	50
<i>Clases de Intrusos</i>	50
<i>Técnicas de Intrusión</i>	50
<i>Protección de Contraseñas</i>	50
<i>Técnicas para Conocer Contraseñas</i>	51
<i>Sistema de contraseñas</i>	51
<i>Esquema de Contraseñas de UNIX</i>	51
<i>Estrategias para la Selección de Contraseñas</i>	52
<i>Detección de Intrusos</i>	52
<i>Técnicas de Detección</i>	53
<i>Registro de Auditoría</i>	53
Software Malicioso (Malware)	54
<i>Tipos Específicos de Malware</i>	54
<i>Naturaleza de los virus</i>	55
<i>Categorías de virus</i>	55
<i>Virus Basados en Macros</i>	56
<i>Estrategias de Antivirus</i>	56
<i>Virus por Correo</i>	57
Unidad 7: Clase 19	57
Sistemas Confiables	57
<i>Monitor de Referencias</i>	58
<i>Defensa Contra Troyanos (Caso de Estudio)</i>	58
Seguridad en Windows	59
<i>Testigo de Acceso</i>	59
<i>Descriptor de Seguridad</i>	60
<i>Listas de Control de Acceso (ACL y ACE)</i>	60
Criptografía Simétrica (Convencional)	61
Unidad 8: Clase 20 Sistemas Distribuidos	65
Computación Cliente/Servidor	65
<i>Terminología Clave</i>	65

<i>Características Distintivas</i>	66
Aplicaciones Cliente/Servidor	66
<i>Aplicaciones de Base de Datos</i>	67
<i>Clases de Aplicaciones Cliente/Servidor</i>	68
<i>Arquitectura Cliente/Servidor de Tres Capas</i>	69
<i>Consistencia de la Cache de Ficheros</i>	70
<i>Arquitectura Middleware</i>	71
Paso de Mensajes Distribuido	72
Unidad 8: Clase 21 <Development>	74
Llamadas a Procedimiento Remoto (RPC)	74
<i>Enlace Cliente/Servidor</i>	75
<i>RPC Síncronos vs. Asíncronos</i>	75
<i>Mecanismos Orientados a Objetos</i>	75
Clusters	75
<i>Beneficios y Objetivos de Diseño</i>	76
Aspectos de Diseño del SO en Clusters	78
<i>Arquitectura de un Cluster</i>	79
<i>Servicios y Funciones Deseables para una Imagen Única del Sistema</i>	79
<i>Cluster frente a SMP</i>	80
Servidor Clusters de Windows (Wolfpack)	80
Sun Clusters	81
Clusters de Beowulf y Linux	83
Características de Beowulf	83

Unidad 5: I/O (Entrada / Salida) Clase 13

Dispositivos de I/O

El sistema operativo debe ofrecer una interfaz simple y fácil de usar entre los dispositivos y el resto del sistema.

Independencia de dispositivos: Que la interfaz sea lo más igual posible para todos los dispositivos.

Los dispositivos **externos de I/O** se agrupan en 3 categorías:

1. **Legibles para el usuario:** Adecuados para la comunicación con el usuario, como impresoras, teclado, pantalla, mouse.
2. **Legibles para la máquina:** Para comunicación con la máquina, como las unidades de almacenamiento, sensores, controladores, activadores.
3. **Comunicación:** Para dispositivos remotos, como la línea digital y los módems.

Principales diferencias entre cada categoría (6):

- Velocidad de transferencia de datos.
- Aplicación.
- Complejidad de control.
- Unidad de transferencia.
- Representación de datos.
- Condiciones de error.

Acá parece como que cambiamos de autor por lo que se ve en el pdf, por lo que es lo mismo que leíste arriba, pero desde otra visión.

Categorías de dispositivos de I/O (2):

- Dispositivos de bloque: Almacena la info en bloques de tamaño fijo. Lo que lo hace esencial, es que puede leer o escribir cada bloque de manera independiente a los demás. Ejemplos: Unidades de almacenamiento.
- Dispositivos de carácter: Envía o acepta un flujo de caracteres, sin importar la estructura del bloque. No es direccionable y no tiene ninguna operación de búsqueda. Ejemplo: Impresoras, interfaces de red, mouses, etc.

No es una buena clasificación, ya que algunos dispositivos no se adaptan.

Controladores de dispositivos

Los dispositivos de E/S consisten en un componente mecánico y uno electrónico. El componente electrónico se conoce como controlador de dispositivo o adaptador. El mecánico es el dispositivo en sí (tdah)

Técnicas para realizar la E/S

- E/S Programada: El procesador envía un mandato de E/S, a petición de un proceso, a un módulo de E/S.
- E/S dirigida por interrupciones: El procesador emite el mandato a petición de un proceso como el anterior, pero es interrumpido por el módulo de E/S cuando este completó su trabajo.
- Acceso directo de memoria (DMA): Un módulo DMA controla el intercambio de datos entre la memoria principal y el módulo de E/S. El procesador manda una petición de

transferencia de datos de un bloque al módulo DMA y resulta interrumpido sólo cuando se transfirió el bloque completo.

De las 3 se usa más el DMA.

	Sin interrupciones	Con interrupciones
Transferencia de E/S a memoria a través del procesador	E/S programada	E/S dirigida por interrupciones
Transferencia directa de E/S a memoria		Acceso directo a memoria (DMA)

Evolución del sistema de E/S

1. El procesador controla directamente un dispositivo periférico.
2. Se añade un controlador o módulo E/S: El procesador usa E/S programada sin interrupciones, por lo que se independiza de detalles específicos de las interfaces de los dispositivos externos.
3. Se usa la misma configuración anterior pero con interrupciones: Es otra manera de hacer lo mismo, pero el CPU no gasta tiempo esperando a que se realice una operación de E/S, por lo que es más eficiente.
4. Al módulo de E/S se le da control directo de la memoria mediante DMA.
5. Se mejora el módulo de E/S para convertirse en un CPU independiente.
6. El módulo de E/S tiene su propia memoria local, por lo que termina siendo una computadora propia.

Básicamente a medida que avanzamos, el CPU de la compu va laburando menos, y se deja la cosa más en mano de las computadoras que traen dentro los propios dispositivos.

E/S por asignación de memoria

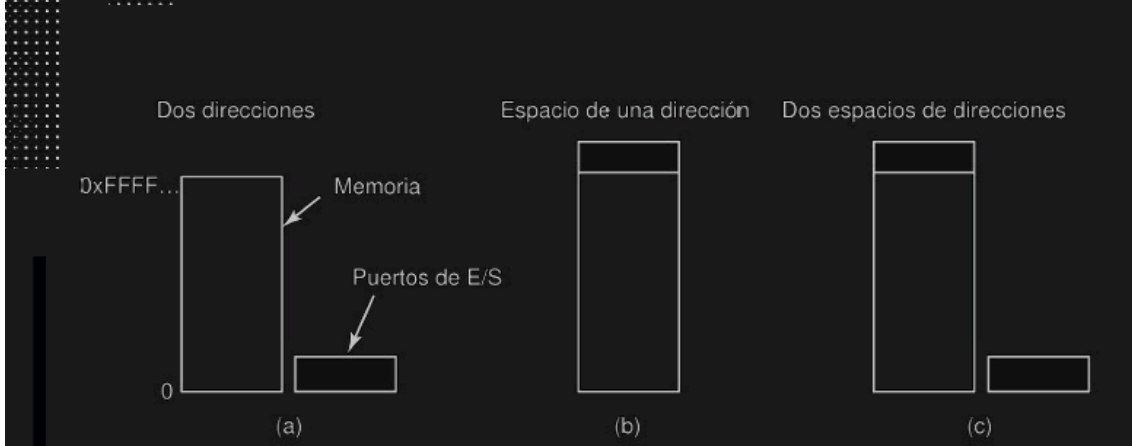
Cada controlador tiene unos cuantos registros que usa para comunicarse con el CPU. El SO usa estos registros para conocer el estado del dispositivo, además traen un buffer de datos que el SO puede leer y escribir.

Debido a esto, se plantea como se puede comunicar la CPU con estos 2 nuevos artefactos.

Métodos (3):

1. Cada registro de control se le asigna un numero de puerto E/S. Los puertos están protegidos y los usa solo el SO.
2. Mapped Memory: Asignar todos los registros de control al espacio de memoria (Figura B del gráfico).
3. Híbrido, con búferes para datos de E/S por asignación de memoria y puertos E/S separados para registros de control.

(a) Espacio separado de E/S y memoria. (b) E/S por asignación de memoria. (c) Híbrido.

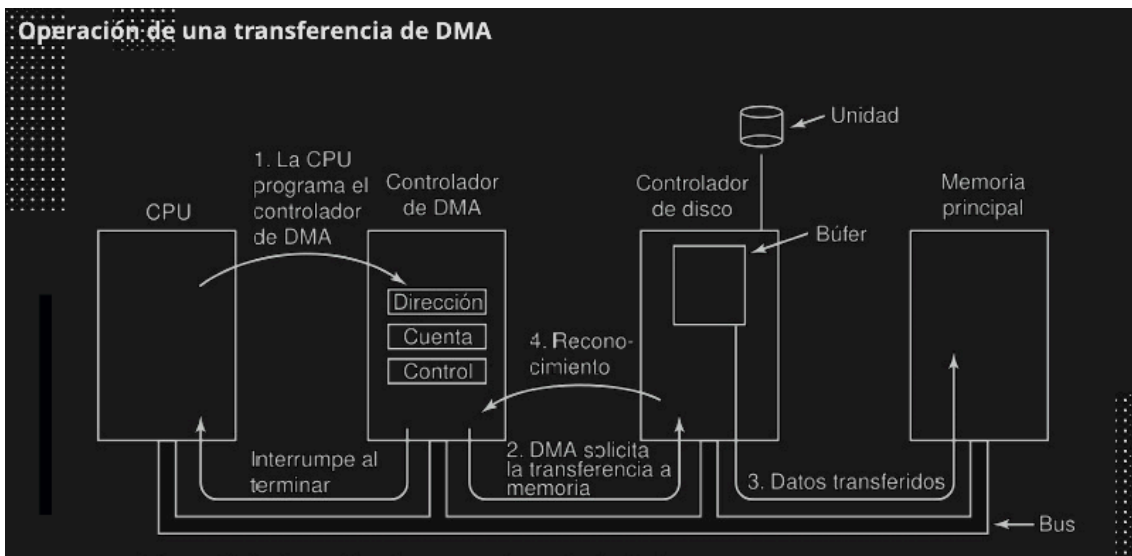


El Pentium usa la arquitectura híbrida.

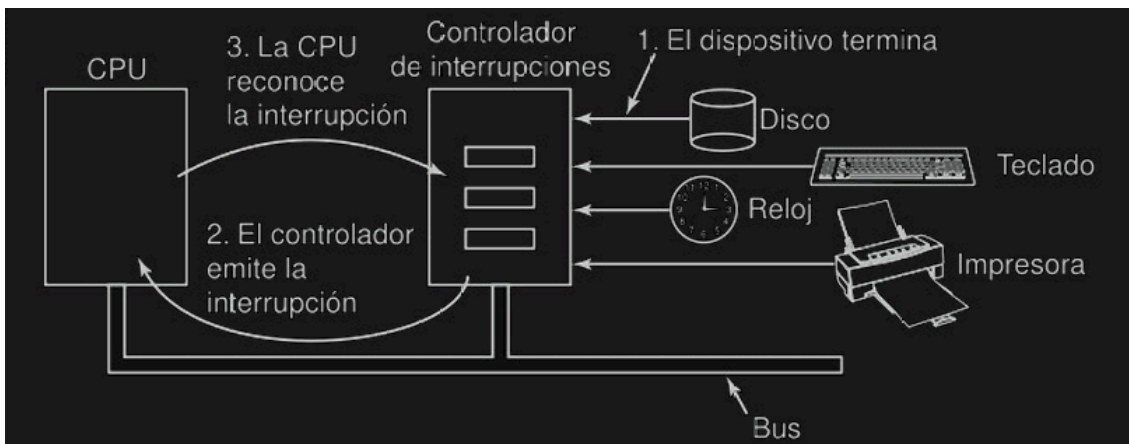
DMA: Direct Memory Access

Más allá de si el CPU tiene E/S por asignación de memoria, la CPU necesita direccionar los controladores para intercambiar datos con ellos, pero para esto solicita datos de un bit a la vez por lo que pierde eficiencia.

Por eso se usa el DMA, es un controlador que tiene acceso al bus del sistema de manera independiente de la CPU.



Interrupciones



Cuando un dispositivo E/S termina una tarea que se le asignó, produce una interrupción (si el SO lo permite) imponiendo una señal en una línea de bus que se le asignó. Esa señal la detecta el chip controlador de interrupciones en la tarjeta principal.

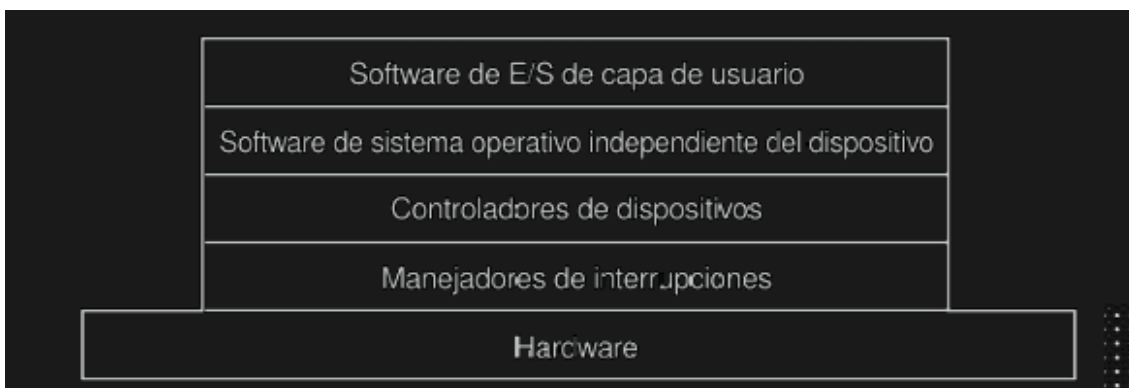
Si no hay otras interrupciones pendientes, el controlador la procesa de inmediato, sino espera y asigna que procesar según prioridad. Dado este último caso, el dispositivo continúa imponiendo la señal hasta ser atendido por la CPU.

Objetivos del diseño de E/S en un SO

1. Eficiencia: Los dispositivos de E/S son extremadamente lentos comparados con la memoria principal y el proce. Una forma de afrontar esto es mediante la multiprogramación. El problema es que la E/S a veces no le sigue el ritmo al procesador.
2. Generalidad: Es necesario manejar todos los dispositivos de manera uniforme.

Capas del software de E/S

Son 4 capas, cada capa tiene su interfaz definida para los niveles adyacentes.



Manejadores de interrupciones: Las interrupciones son un hecho incomodo y no se pueden evitar, se deben ocultar en la profundidad de las entrañas del sistema operativo.

Las oculta bloqueando el controlador que inicie la operación de E/S hasta que se haya completado.

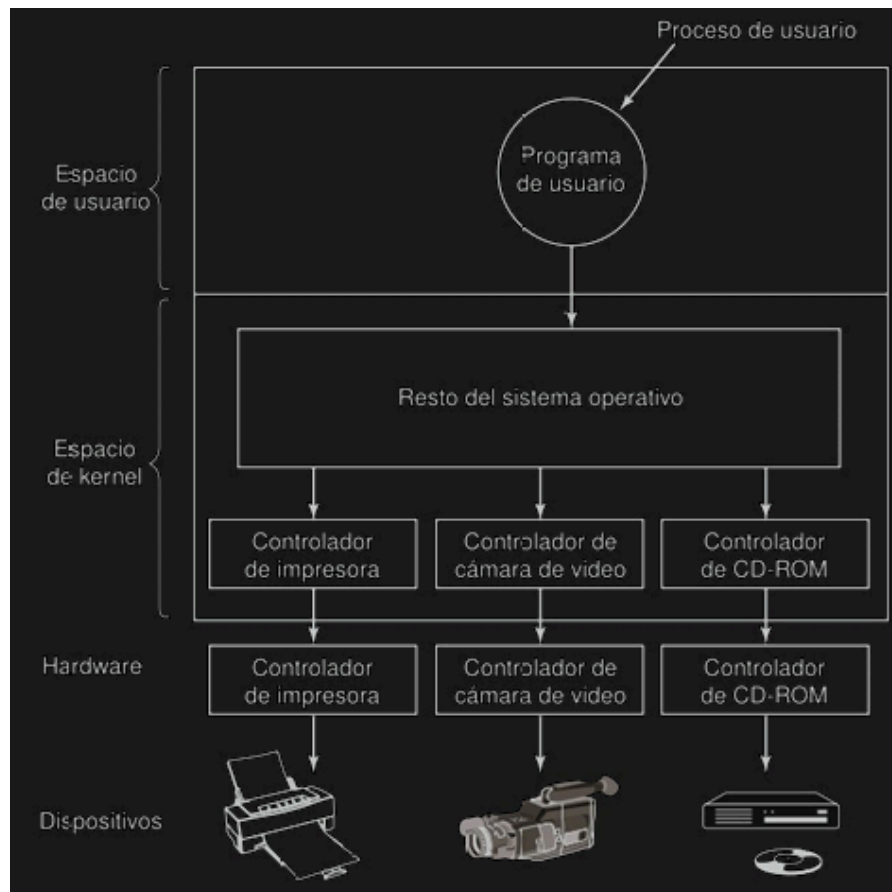
- La llamada **down** a un semáforo hace que el controlador se bloquee a sí mismo.
- **Wait** en una variable de condición también

Drivers de dispositivos

Código que sirve para manejar el dispositivo de E/S, es escrito por el fabricante del dispositivo y se incluye en el mismo.

Es necesario tener un modelo bien definido de lo que hace un driver y la forma en la que interactúa con el resto del sistema operativo.

Normalmente los SO definen una interfaz estándar que para todos los controladores de bloque. Estas interfaces tienen procedimientos que el SO usa para mandar trabajo al controlador.

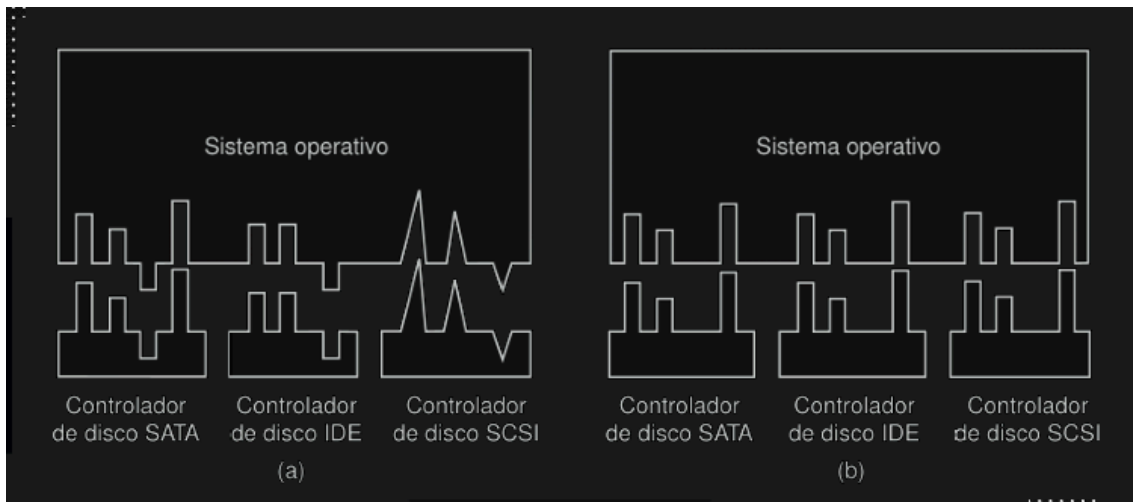


Al conectar un dispositivo nuevo, el controlador se carga de forma dinámica en el sistema durante la ejecución. Cada sistema maneja la carga de controladores de formas distintas.

Software de E/S independiente del dispositivo

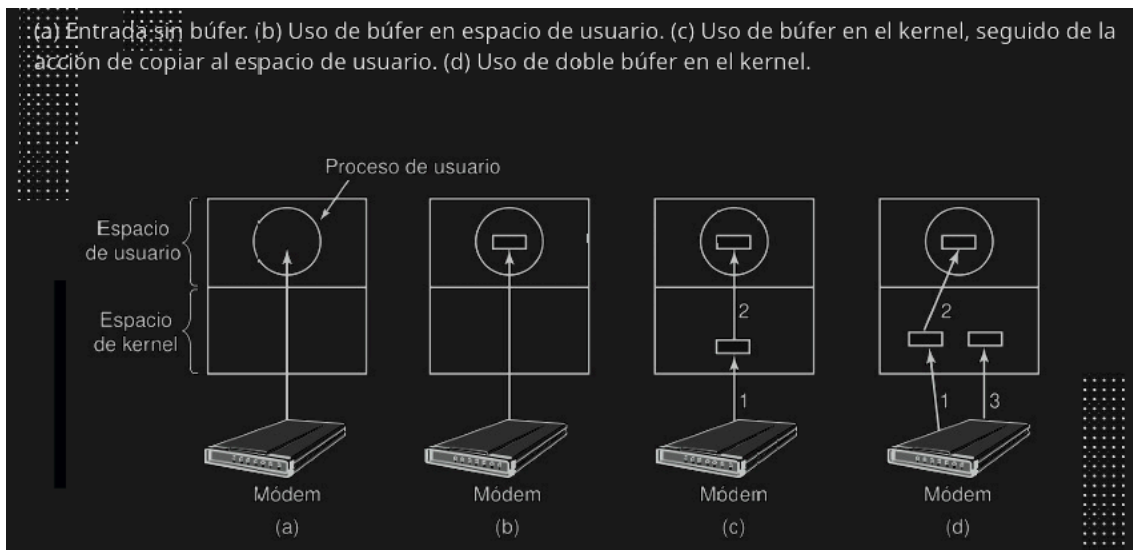
La función básica del software independiente del dispositivo es realizar las funciones de E/S que son comunes para todos los dispositivos y proveer una interfaz uniforme para el software a nivel de usuario.

Es importante tener una interfaz uniforme para los controladores de software de los dispositivos. Una importante cuestión en un SO es como hacer que todos los dispositivos de E/S y sus controladores se vean más o menos iguales.



En la figura B todos los controladores tienen la misma interfaz, por lo que es más fácil conectar un nuevo controlador.

Uso de búfer



Los búferes son una técnica muy utilizada, pero con una desventaja, y es que si los datos se colocan en el búfer demasiadas veces, el rendimiento se reduce.

Reporte de errores

El SO debe saber cómo manejar los errores de forma eficiente.

Los errores de programación son una clase de errores de E/S, cuando por ejemplo un proceso pide algo imposible, existen otros como errores de dirección de buffer invalida, o especificación de un dispositivo inválido.

Errores de E/S reales: Tratar de recibir un bloque de disco dañado o tratar de leer de una cámara apagada. En estas circunstancias depende del controlador determinar que hacer.

Asignación y liberación de dispositivos dedicados

Una manera simple de manejar peticiones para escribir en dispositivos que solo pueden ser usados por un proceso, es las llamadas a open de archivos especiales para los dispositivos. Si el

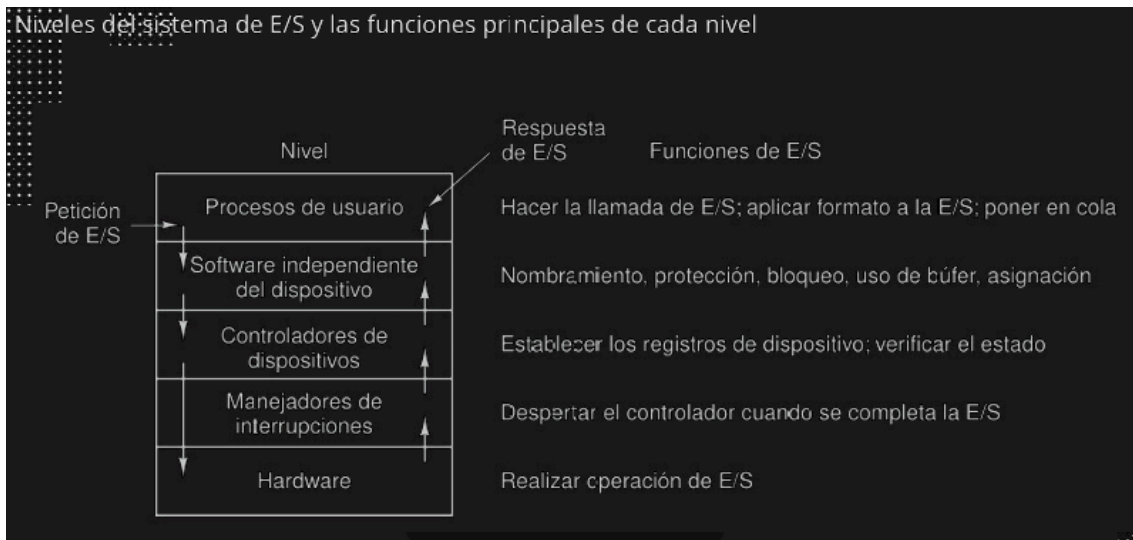
dispositivo no está disponible, la open falla. Al cerrar un dispositivo dedicado de este tipo, se liberan.

Software de E/S en espacio de usuario

Normalmente las llamadas al sistema, incluyendo al sistema de E/S, se realizan mediante procedimientos de biblioteca. No todo software de E/S de bajo nivel consiste en procedimientos de biblioteca.

Otra forma de lidiar con dispositivos de E/S es el uso de colas (Spooling) .

Para la cola de impresión, se crea un demonio y un directorio de cola de impresión, para imprimir el archivo, el demonio genera primero todo el archivo que va a imprimir y lo coloca en el directorio de la cola de impresión.



Unidad 5 – Clase 14 – Discos (Como dispositivos de E/S)

btw toda esta unidad va a estar obsoleta pero toca estudiarla jejejeje

Hardware Disco

Son ideales como memoria secundaria porque la lectura y escritura es igual de rápida. Se usan arreglos de estos discos para ofrecer un almacenamiento altamente confiable (el famoso RAID).

El controlador coloca pistas en caché, reasigna bloques defectuosos y más.

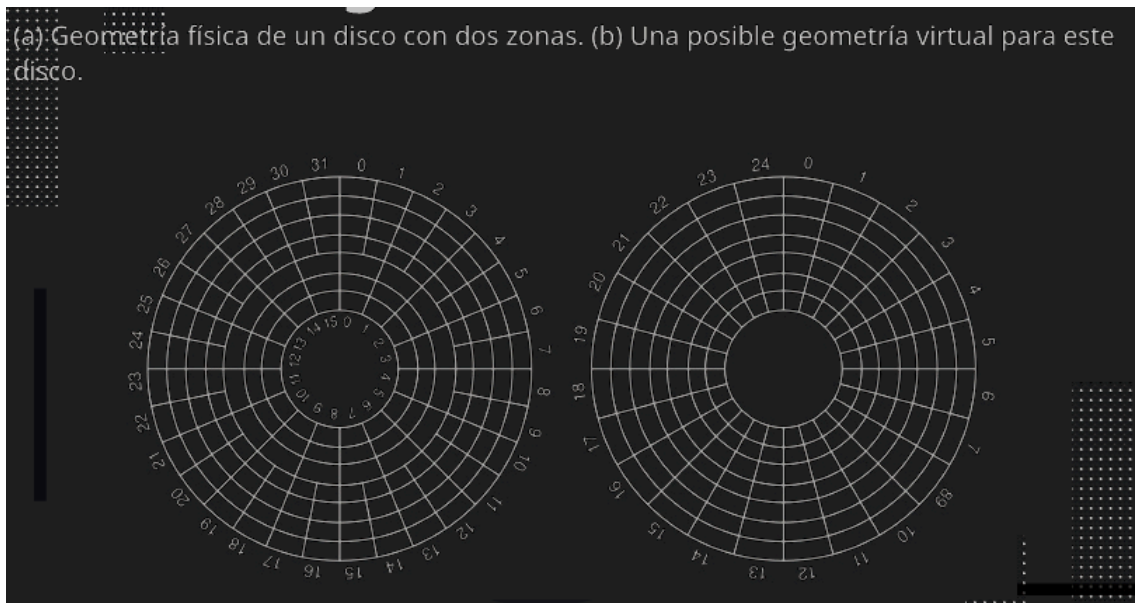
Búsquedas traslapadas: Un controlador puede realizar búsquedas en dos o más unidades al mismo tiempo.

Discos magnéticos

Parámetros de disco para el disco flexible IBM PC 360 KB original y un disco duro Western Digital WD 18300

Parametro	Disco flexible IBM de 360-KB	Disco duro WD 18300
Numero de cilindros	40	10601
Pistas por cilindro	2	12
Sectores por pista	9	281 (promedio)
Sectores por disco	720	35742000
Bytes por sector	512	512
Capacidad del disco	360 KB	18.3 GB
Tiempo de búsqueda (cilindros adyacentes)	6 msec	0.8 msec
Tiempo de búsqueda (caso promedio)	77 msec	6.9 msec
Tiempo de rotación	200 msec	8.33 msec
Tiempo de arranque para el motor	250 msec	20 seg
Tiempo para transferir 1 sector	22 msec	17 µseg

Para ocultar detalles como cuantos sectores tiene cada pista, los discos modernos tienen una geometría virtual que se presenta al sistema operativo.



El arreglo publicado, que sería el 2, difiere del real.

Los discos modernos aceptan un sistema llamado **Direccionamiento de Bloques Lógicos**, en que los sectores del disco sólo se enumeran en forma consecutiva empezando en 0, sin importar la geometría del disco. Esto se creó debido a unos parámetros que hacían que como mucho un disco pudiera ser de 31.5 GB.

EN 1988, Patterson y colaboradores sugirieron seis organizaciones de discos específicas para mejorar el rendimiento del disco, su confiabilidad o ambas.

RAID

La idea es instalar una caja llena de discos a un lado de la computadora (por lo general un servidor), reemplazar la tarjeta controladora de discos por un controlador RAID, y copiar los datos al raid y luego operar normal.

Un RAID debe verse como un SLED (Single Large Expensive Disk) para el SO, de tal forma queda mejor rendimiento y compatibilidad.



CD-ROMs

Se usan también discos ópticos, tienen densidades de grabación mucho más altas que los discos magnéticos convencionales. El CD (Disco Compacto) reemplazo al vinilo.

Utilizan un sistema de archivos llamado High Sierra, que evoluciono luego a ser un Estándar Internacional (IS 9660). Tiene tres niveles.

El nivel 1: Utiliza nombres de archivo de hasta 8 caracteres, con una extensión opcional de hasta 3 caracteres.

El nivel 2 permite nombres de hasta 32 caracteres, y el nivel 3 permite archivos no contiguos.

CD-Rs (CD-Grabables) son los CD-ROMs fabricados por empresas para iniciar operaciones.

Los CD-ROMs tenían una sola VTOC (Volume Table of Contents), lo cual no funciona con escrituras incrementales, es decir, en varias pistas, que era un estándar que estaban incorporando los nuevos CDS, por eso, para solucionarlo implementaron que cada pista del cd tenga su propia VTOC.

Los CD-RW (CD Regrabable) en blanco son más costosos que los CD-R.

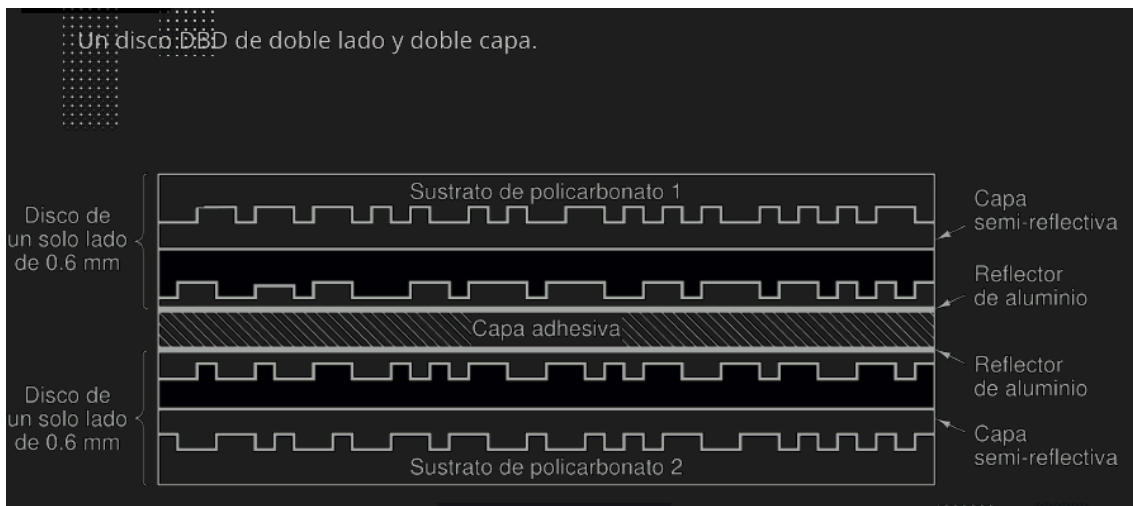
DVD

Utilizan el mismo diseño que los CDs, pero con mejoras que elevan su capacidad siete veces.

Existen 4 formatos:

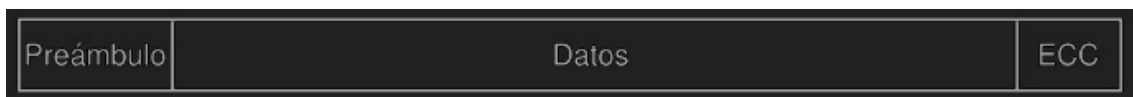
1. Solo lado, una sola capa. 4,7 GB
2. Un solo lado, doble capa. 8.5 GB
3. Doble Lado, una capa. 9.4 GB.
4. Doble lado, doble capa, 17 GB.

Según el libro basado hay muchos formatos gracias a la política.



Formato de Disco

Antes de usar cada disco, hay que hacer un formateo de cada nivel mediante software a cada plato.



El preámbulo tiene un cierto patrón de bits que le permite al hardware reconocer el inicio del sector. ECC Tiene información redundante para recuperarse en caso de errores de lectura.

La posición del sector 0 en cada pista se desfasa de la pista anterior cuando se hace el formateo de bajo nivel. Esto se llama **desajuste de cilindros** y sirve para mejorar el rendimiento.

El formato de bajo nivel reduce la capacidad del disco, reduciéndola hasta un 20%, y además afecta al rendimiento.

Luego de formatear, el disco se particiona, y cada partición se la toma como un disco en sí misma. El sector 0 contiene el MBR (blabla lo q estudiamos pa el primer parcial).

Luego, el paso final para preparar el disco es hacer un formateo de alto nivel a cada partición, donde se establece un bloque de inicio, administración de espacio de almacenamiento libre, directorio raíz y sistema de archivos vacío.

Algoritmos de programación del brazo del disco.

Primero se debe determinar el tiempo requerido para leer o escribir en un bloque de disco. Se determina en base a 3 factores:

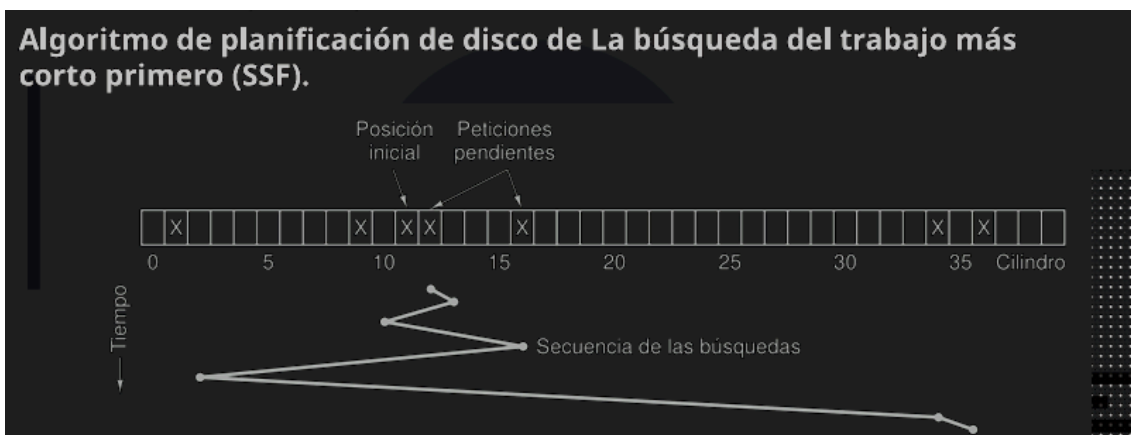
1. Tiempo de búsqueda (tiempo para desplazar el brazo al cilindro determinado).
2. Retraso rotacional (tiempo para que el sector apropiado se coloque debajo de la cabeza)
3. Tiempo de transferencia de datos actual.

Generalmente el **tiempo de búsqueda** domina los otros dos tiempos.

Si el controlador del disco funciona como FCFS (First come first serve), no se puede hacer mucho para mejorar la optimización del tiempo de búsqueda.

Algoritmo de planificación de disco de la búsqueda de trabajo más corta primero.

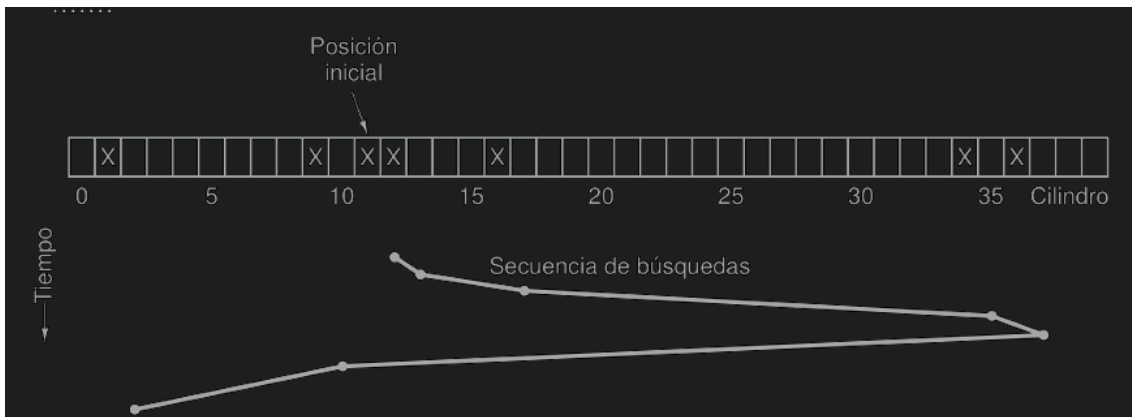
Los controladores de disco mantienen una tabla indexada por número de cilindro con las peticiones pendientes para cada cilindro encadenadas en una lista enlazada, encabezada por las entradas de la tabla



Con un disco que tenga mucha carga, el brazo tiende a permanecer en la parte media del disco la mayor parte del tiempo, por lo que las peticiones en cualquier extremo tendrán que esperar.

Algoritmo del ascensor (elevator)

El software debe mantener un bit, el bit de dirección a cual, arriba o abajo. Cuando termina una petición, se comprueba el bit, si es arriba, el brazo se desplaza a la siguiente petición con mayor prioridad, sino el bit de dirección se invierte. Cuando el bit se establece en ABAJO, se avanza a la siguiente petición de menor prioridad si hay.



Manejo de errores

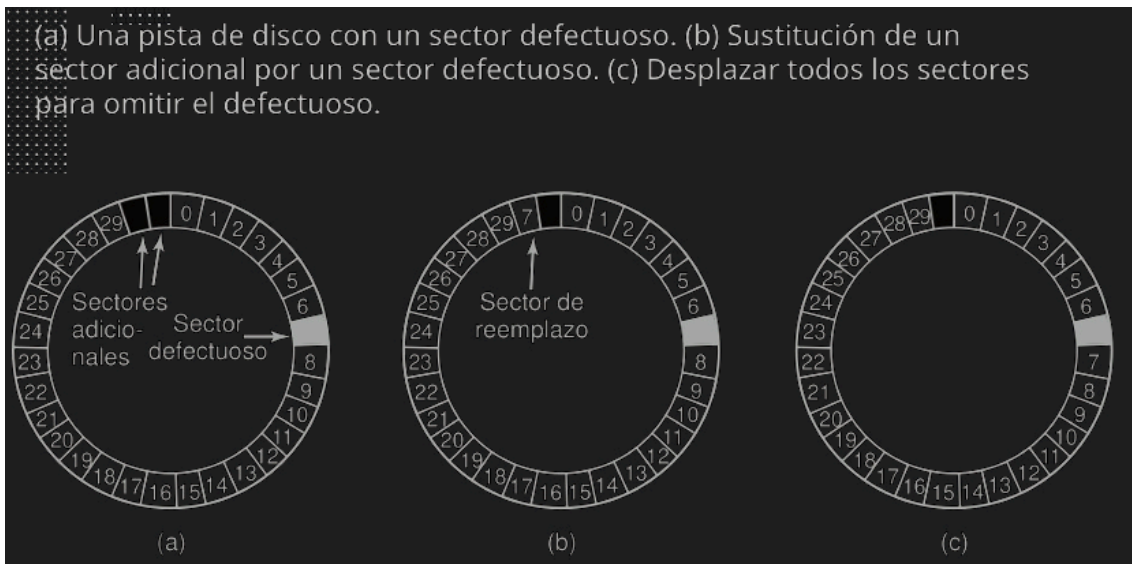
Se exceden los límites de la tecnología constantemente al incrementar la densidad de bits lineales. Fabricar discos sin defectos es imposible, ya que siempre que se aumenta la densidad de los discos se introducen nuevos errores.

Los defectos de fabricación incluyen sectores defectuosos; ósea sectores que no se leen correctamente el valor que se acaba de escribir en ellos.

Hay 2 métodos para trabajar con estos sectores defectuosos, pelear con ellos en el controlador o en el sistema operativo.

Primera forma: El controlador reasigna uno de los sectores adicionales como el sector dañado. (figura b)

Segunda forma: Desplaza los todos los sectores una posición hacia arriba (figura c).



Para lograr esto el controlador debe saber cuál sector es acá sector, puede saberlo mediante tablas internas o volviendo a escribir los preámbulos para proporcionar los números de sectores reasignados. Para este último caso el segundo método es más complicado, pero ofrece mejor rendimiento.

Cuando el SO se hace cargo

Se pueden presentar errores durante una operación normal, una vez se haya instalado la unidad. Al obtener un error que el ECC no puede manejar, lo primero que se suele hacer es leer devuelta, ya que algunos errores de lectura son transitorios.

Para solucionar los problemas con los sectores desde la perspectiva del SO, el SO adquiere una lista de sectores defectuosos, ya sea leyéndolos del disco o probando todo el disco. Una vez lo sabe, construye las tablas de reasignación.

Si el SO se hace cargo de la reasignación, debe asegurarse de que no haya sectores malos en ningún archivo, y que tampoco haya en la lista libre o mapa de bits.

Errores inducidos por el brazo

A veces los errores ocurren por problemas mecánicos en el brazo. Si el brazo está en una posición incorrecta, si el brazo está en la posición incorrecta, ha ocurrido un error de búsqueda.

La mayoría de los controladores de disco duro corrigen los errores de búsqueda de manera automática

Almacenamiento estable

Los RAIDs protegen contra el hecho de que unos cuantos sectores se vuelvan defectuosos, o incluso que falle una unidad. Sin embargo, no protegen contra los errores de escritura que establecen datos defectuosos en primer lugar. Tampoco protegen contra las fallas durante las escrituras que corrompen los datos originales sin reemplazarlos con datos nuevos

Lo que se puede lograr es un subsistema de disco que tenga la siguiente propiedad: cuando se emita una escritura, el disco debe escribir correctamente los datos o no hacer nada

El modelo asume que cuando un disco escribe un bloque (uno o más sectores), la escritura es correcta o incorrecta y este error se puede detectar en una lectura subsiguiente, al examinar los valores de los campos ECC.

El modelo también supone que un sector escrito correctamente puede volverse defectuoso de manera repentina, quedando ilegible.

El modelo supone además que la CPU puede fallar, en cuyo caso sólo se detiene.

El almacenamiento estable utiliza un par de discos idénticos, en donde los bloques correspondientes funcionan en conjunto para formar un bloque libre de errores.

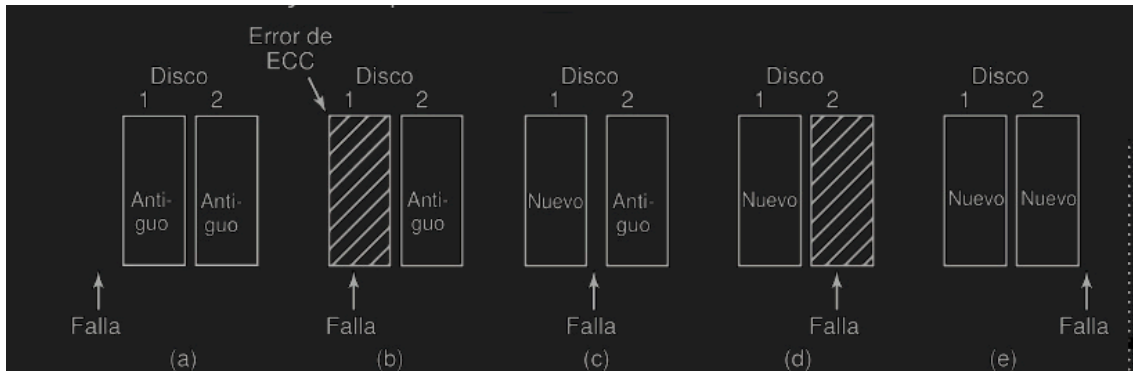
Para lograr este objetivo, se definen las siguientes tres operaciones:

1. Escrituras estables.
Una escritura estable consiste en primero escribir el bloque en la unidad 1, y después volver a leerlo para verificar que se haya escrito correctamente.
2. Lecturas estables.
En una lectura estable primero se lee el bloque de la unidad 1. Si esto produce un ECC incorrecto, la operación de lectura se vuelve a intentar hasta n veces. Si todas estas operaciones producen ECCs defectuosos, se lee el bloque correspondiente de la unidad 2.
3. Recuperación de fallas.

Después de una falla, un programa de recuperación explora ambos discos y compara los bloques correspondientes. Si un par de bloques están bien y son iguales, no se hace nada.

Fallas por parte del CPU

Existen 5 posibilidades de falla dadas por la CPU en escrituras estables:



Para hacerla corta, está indicando en qué momento falla. La primera falla antes de arrancar, la otra falla al escribir la unidad 1 destruyendo el contenido (luego lo recupera el programa de recuperación), y así va con las otras.

Ninguna falla es grave.

Unidad 5 - Clase 15 Entrada / Salida

Relojes – Temporizadores

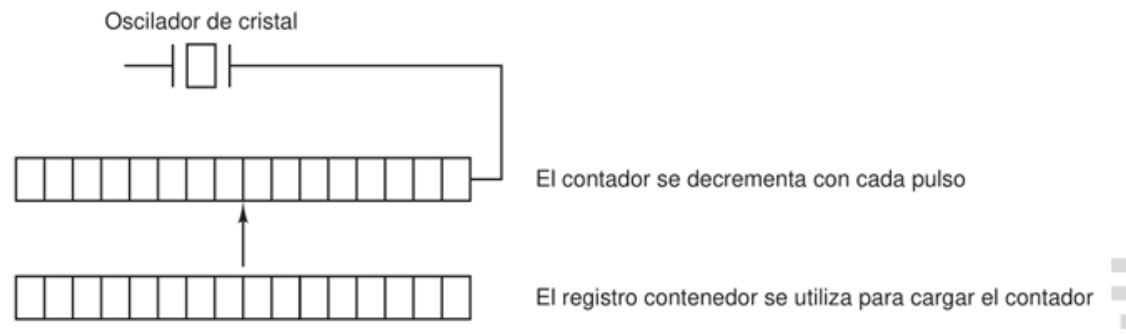
Son esenciales para sistemas multiprogramación, mantienen la hora del día y evitan que un proceso monopolice la CPU.

Tipos de relojes:

- Simples: Están enlazados a la línea de energía, producen una interrupción cada ciclo de voltaje. Ya no se usan.
- Programable: Con oscilador de cristal, un contador y un registro contenedor.

Para este ultimo tipo, cuando una pieza de cristal de cuarzo se corta en forma apropiada y se monta bajo tensión, puede generar una señal periódica con una precisión muy grande.

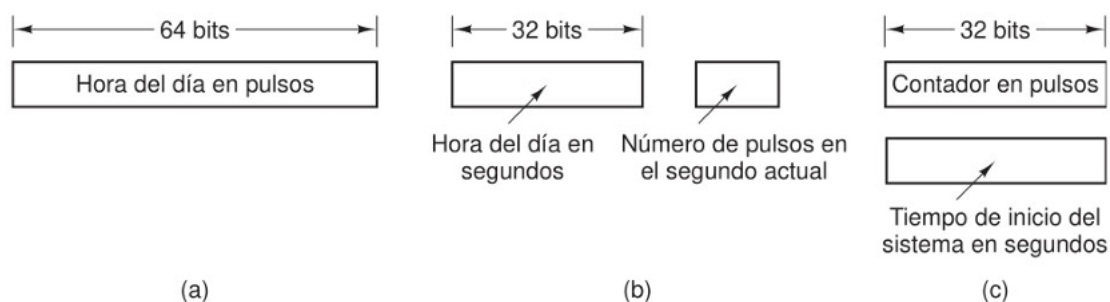
Su frecuencia de interrupción se puede controlar por software. El reloj solo genera interrupciones en intervalos conocidos, el resto relacionado al tiempo lo realiza el software controlador del reloj.



Software de reloj

Primera función: Mantener hora del día. Al mantener la hora del día, surge un problema que el número de bits en el contador de la hora del día se puede desbordar. Esto se resuelven con 3 métodos:

1. Un contador de 64 bits.
2. Mantener la hora del día en segundos.
3. Contar los pulsos.



Segunda función: Evitar que los procesos se ejecuten en demasiado tiempo.

Tercera: Contabilizar uso de la CPU.

Cuarta: Un proceso puede solicitar que el sistema operativo proporcione una advertencia después de cierto intervalo, esta advertencia es una señal, interrupción o mensaje o algo similar.

Quinta: Watchdogs, son temporizadores guardianes que sirven a partes del SO.

Sexta: Perfilamiento. Se usa para estadística, por ejemplo para saber cuánto se usó el sistema o un software.

Temporizadores

Los temporizadores de software evitan interrupciones, por ejemplo, si el kernel se ejecuta por alguna razón, antes de regresar al modo usuario comprueba el reloj de tiempo real para ver si algún temporizador de software expiro. Si alguno expiro, realiza el evento que tenga programado, para no tener que cambiar devuelta al modo kernel.

Los temporizadores de software marcan o descienden con la velocidad a la que se realizan entradas en el kernel por otras razones. Estas razones son: 1. Llamadas al sistema; 2. Fallos del TLB; 3. Fallos de página; 4. Interrupciones de E/S ; y 5. La CPU queda inactiva.

Interfaces de usuario – Teclados, mouse, etc.

La entrada del usuario proviene principalmente del teclado y del ratón. Cuando se apreta una tecla del teclado, se genera una interrupción, y se genera una segunda al soltarla. En cada interrupción del teclado, el software controlador del mismo extrae información acerca de que ocurre.

Software de teclado

Los teclados necesitan 7 bits para representar el número de tecla. El 8vo bit se pone en 0 cuando se apreta la tecla y en 1 cuando se suelta.

Software de entrada

Se pueden adoptar dos filosofías posibles para el controlador.

En la primera, el trabajo del controlador es sólo aceptar la entrada y pasarla hacia arriba sin modificarla.

En la segunda filosofía: el controlador maneja toda la edición entre líneas, y envía sólo las líneas corregidas a los programas de usuario.

El estándar POSIX utiliza el término **modo canónico** para describir el modo orientado a líneas. El **modo no canónico** es equivalente al modo crudo, aunque es posible cambiar muchos detalles del comportamiento.

Si el teclado está en modo **canónico** (cocido), los caracteres se deben almacenar hasta que se haya acumulado una línea completa, debido a que tal vez el usuario decida posteriormente borrar parte de ella.

Aunque el teclado y el monitor son dispositivos lógicamente separados, muchos usuarios han crecido acostumbrados a ver los caracteres que acaban de escribir aparecer en la pantalla. A este proceso se le conoce como producir eco (echo).

Cuando se opera en modo canónico, algunos caracteres de entrada tienen significado especial.

Carácter	Nombre POSIX	Comentario
CTRL-H	ERASE	Retrocede un carácter
CTRL-U	KILL	Borra toda la línea que se está escribiendo
CTRL-V	LNEXT	Interpreta el siguiente carácter literalmente
CTRL-S	STOP	Detiene la salida
CTRL-Q	START	Inicia la salida
DEL	INTR	Proceso de interrupción (SIGINT)
CTRL-\	QUIT	Obliga un vaciado de núcleo (SIGQUIT)
CTRL-D	EOF	Fin de archivo
CTRL-M	CR	Retorno de carro (no se puede modificar)
CTRL-J	NL	Avance de línea (no se puede modificar)

Software del mouse

Cada vez que un ratón se ha desplazado cierta distancia mínima en cualquier dirección, o cuando se oprime o suelta un botón, se envía un mensaje a la computadora.

El mensaje para la computadora contiene tres elementos: dx, dy, botones.

El formato del mensaje depende del sistema y del número de botones que tenga el ratón.

La mayoría de los ratones se reportan con la computadora un máximo de 40 veces/seg (Esto equivale a 40hz de polling rate, lo cual está obsoleto, hoy en día se utiliza mínimo 250hz para mouses comunes genéricos, y en casos de mouses ganar competitivos, 1000hz y 8000hz. En resumen, tenembaum cogido).

Observe que el ratón indica sólo los cambios en la posición, no la posición absoluta en sí (Por los diferenciales).

Ventanas de texto

La salida es más simple que la entrada cuando se envía secuencialmente en un solo tipo de letra, tamaño y color.

El problema era que no estaban estandarizadas dichos datos, por lo que Berkeley UNIX fue una base de datos de terminales conocida como termcap. Este paquete de software definía una variedad de acciones básicas

En cierto momento, la industria vio la necesidad de estandarizar la secuencia de escape, por lo que se desarrolló un estándar ANSI

Las secuencias de escape ANSI

Secuencia de escape	Significado
ESC [<i>n</i> A	Se desplaza <i>n</i> líneas hacia arriba
ESC [<i>n</i> B	Se desplaza <i>n</i> líneas hacia abajo
ESC [<i>n</i> C	Se desplaza <i>n</i> espacios a la derecha
ESC [<i>n</i> D	Se desplaza <i>n</i> espacios a la izquierda
ESC [<i>m</i> ; <i>n</i> H	Desplaza el cursor a (<i>m</i> , <i>n</i>)
ESC [<i>s</i> J	Borra la pantalla del cursor (0 al final, 1 del inicio, 2 todo)
ESC [<i>s</i> K	Borra la línea del cursor (0 al final, 1 al inicio, 2 todo)
ESC [<i>n</i> L	Inserta <i>n</i> líneas en el cursor
ESC [<i>n</i> M	Elimina <i>n</i> líneas en el cursor
ESC [<i>n</i> P	Elimina <i>n</i> caracteres en el cursor
ESC [<i>n</i> @	Inserta <i>n</i> caracteres en el cursor
ESC [<i>n</i> m	Habilita la reproducción <i>n</i> (0=normal, 4=negritas, 5=parpadeo, 7=inverso)
ESC M	Desplaza la pantalla hacia atrás si el cursor está en la línea superior

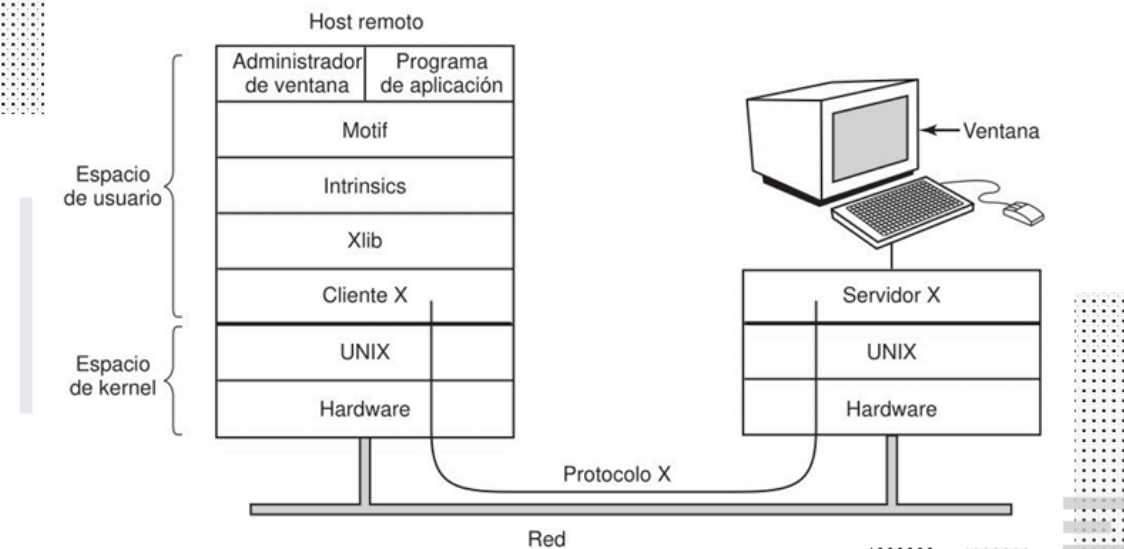
Sistema X Window

Casi todos los sistemas UNIX basan su interfaz de usuario en el Sistema X Window (que a menudo sólo se le llama X), desarrollado en el M.I.T. como parte del proyecto Athena en la década de 1980. Sirve para conectar y manejar terminales remotas con un server central.

Básicamente absorbe la entrada del mouse y teclado, y se la envía a los programas que la necesiten, generalmente la ventana que se este visualizando.

La razón por la que es posible ejecutar el Sistema X Window encima de UNIX (o de cualquier otro sistema operativo) en una sola máquina o a través de una red, es que lo que X define en realidad es el protocolo X entre el cliente X y el servidor X.

Cientes y servidores en el Sistema X Window del M.I.T.



X es sólo un sistema de ventanas; no es una GUI completa. Para obtener una GUI completa, se ejecutan otras capas de software encima.

Por lo que tiene un diseño modular que permite a X ser portátil y flexible.

Interfaces gráficas

Una GUI tiene cuatro elementos esenciales, denotados por los caracteres WIMP. Las letras representan ventanas (Windows), iconos (Icons), menús (Menus) y dispositivo señalador (Pointing device), respectivamente.

El software de GUI se puede implementar en código a nivel de usuario, como en los sistemas UNIX, o en el mismo sistema operativo, como en el caso de Windows

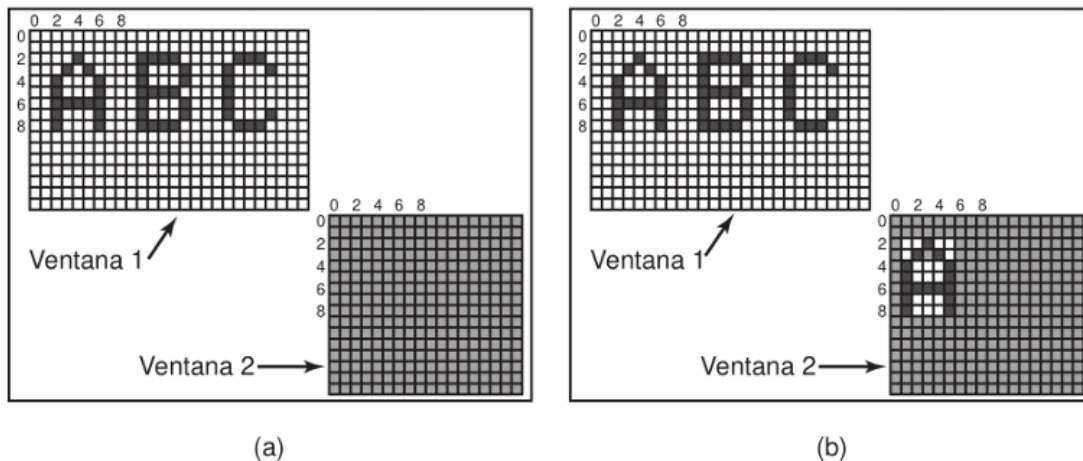
Los programas de Windows están orientados a los mensajes. Cada programa tiene una cola de mensajes a donde se envían los mensajes relacionados con todas sus ventanas.

Mapas de bits

En Windows podés copiar algo como parte de un dibujo y pegarlo en otro programa, esto se con métodos como representar el dibujo como un metarchivo de Windows y que al copiarlo se coloque en el portapapeles como formato .wmf.

No toda imagen de la PC se puede generar mediante gráficos vectoriales, las fotos y videos son ejemplo de ello. Estos elementos se exploran al sobreponer una rejilla en la imagen. Los valores rojo, verde y azul promedio de cada cuadro de la rejilla se muestrean y se guardan como el valor de un pixel. A dicho archivo se le conoce como mapa de bits. Hay muchas herramientas en Windows para manipular mapas de bits.

Copiado de mapas de bits



El problema de los mapas de bits es que no escalan, por lo que Windows usa DIB que permite escalar la información.

Tipos de Letras

TrueType, que no son mapas de bits sino contornos de los caracteres. Cada carácter TrueType se define mediante una secuencia de puntos alrededor de su perímetro.

20 pt: *abcdefgh*

53 pt: *abcdefgh*

81 pt: *abcdefgh*

Cientes delegados

Existen modelos de PC descentralizado y centralizado. Básicamente, dice que los PC descentralizados en algún momento van a morir porque los centralizados son mejores.

Administración de Energía

Es importante administrar la energía en por ejemplo equipos portátiles.

Hay dos métodos generales para reducir el consumo de energía. El primero es que el sistema operativo apague partes de la computadora.

El segundo método es que el programa de aplicación utilice menos energía.

Hardware

Se diseña el hardware para tener múltiples estados, como encendido, inactivo, hibernado y apagado. Cuando el dispositivo no se va a utilizar durante un tiempo corto se puede poner en inactividad, lo cual reduce el consumo de energía. Cuando no se va a utilizar durante un

intervalo mayor, se puede poner en hibernación, lo cual reduce el consumo de energía aún más.

Consumo de energía en una notebook, no me pregunten si es importante, pero si tiene ganas de jodernos la vida lo puede tomar..

Dispositivo	Li y colaboradores (1994)	Lorch y Smith (1998)
Pantalla	68%	39%
CPU	12%	18%
Disco duro	20%	12%
Módem		6%
Sonido		2%
Memoria	0.5%	1%
Otros		22%

Se buscan algoritmos y heurística que permitan al sistema operativo tomar buenas decisiones sobre lo que se va a apagar y cuándo se debe hacer.

Pantalla

El principal elemento consumidor de energía es la pantalla. Para obtener una imagen nítida y brillante, la pantalla debe tener luz posterior, y eso requiere una energía considerable.

Disco duro

Requiere una energía considerable para mantenerse girando a una alta velocidad, incluso aunque no haya accesos. Generalmente se hace que el disco deje de girar después de cierto número de segundos o minutos de inactividad. Otra forma de ahorrar energía del disco es tener una caché de disco de un tamaño considerable en la RAM. Si se necesita un bloque que se encuentra en la caché, no hay que reiniciar un disco inactivo para satisfacer la lectura.

CPU

A menudo el voltaje de la CPU se puede reducir mediante software, lo cual ahorra energía pero también reduce el ciclo de reloj.

Memoria

Existen dos opciones posibles para ahorrar energía con la memoria. En primer lugar, la caché se puede vaciar y después apagarse.

Una opción más drástica es escribir el contenido de la memoria principal en el disco y apagar la memoria principal. Este método es la hibernación. Cuando se apaga la memoria, la CPU tiene que apagarse, o tiene que ejecutarse de la ROM.

Comunicación inalámbrica

Para ahorrar energía, la radio que recibe los mensajes se apaga al pasar un cierto tiempo de inactividad. Para no perder los mensajes, estos se colocan en un buffer, que si esta por llenarse, prende la radio y hace que reciba todos los mensajes.

Administración térmica

El sistema operativo tiene que monitorear la temperatura en forma continua. Cuando se acerca a la temperatura máxima permisible, el sistema operativo tiene que tomar una decisión. Como encender el ventilador o reducir voltajes.

Administración de baterías

Hoy en día las baterías son inteligentes y se pueden comunicar con el SO.

Interfaz de drivers

Windows usa ACPI (Advanced Configuration and Power Interface, Interfaz avanzada de configuración y energía) para administrar las interfaces.

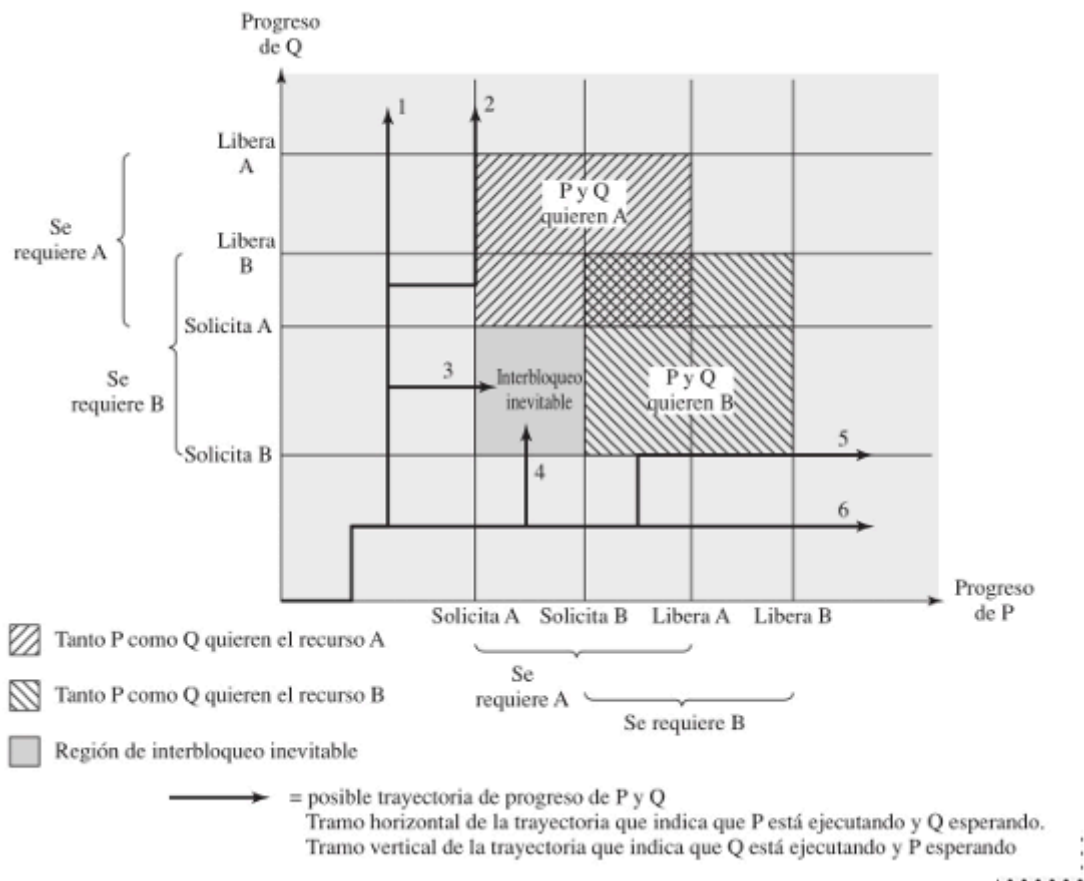
Unidad 6: Clase 16.1 Interbloqueos

Se puede definir el interbloqueo como el bloqueo permanente de un conjunto de procesos que o bien compiten por recursos del sistema o se comunican entre sí.

Todos los interbloqueos involucran necesidades conflictivas que afectan a los recursos de dos o más procesos.

En la siguiente figura, denominada diagrama de progreso conjunto, muestra el progreso de dos procesos compitiendo por dos recursos.

Proceso P	Proceso Q
...	...
Solicita A	Solicita B
...	...
Solicita B	Solicita A
...	...
Libera A	Libera B
...	...
Libera B	Libera A
...	...



Recursos

Existen 2 categorías: reutilizables y consumibles.

Reutilizable

Un recurso reutilizable es aquél que sólo lo puede utilizar de forma segura un proceso en cada momento y que no se destruye después de su uso. Por ejemplo Procesadores, canales de E/S, memoria principal, secundaria, dispositivos, ficheros, bases de datos, semáforos.

Como ejemplo, muestra dos procesos que compiten por el acceso exclusivo al disco D y la unidad C.

$p_0 p_1 q_0 q_1 p_2 q_2$

Proceso P		Proceso Q	
Paso	Acción	Paso	Acción
p_0	Solicita (D)	q_0	Solicita (C)
p_1	Bloquea (D)	q_1	Bloquea (C)
p_2	Solicita (C)	q_2	Solicita (D)
p_3	Bloquea (C)	q_3	Bloquea (D)
p_4	Realiza función	q_4	Realiza función
p_5	Desbloquea (D)	q_5	Desbloquea (C)
p_6	Desbloquea (C)	q_6	Desbloquea (D)

Estos interbloqueos se pueden producir por la lógica del programa, por lo que es difícil detectarlos. Una estrategia para tratar con este interbloqueo es imponer restricciones en el diseño del sistema con respecto al orden en que se pueden solicitar los recursos.

Este es otro ejemplo de interbloqueo, es con la memoria principal: donde por ejemplo tenemos 200 Kbytes para reservar. Si se producen estas peticiones:

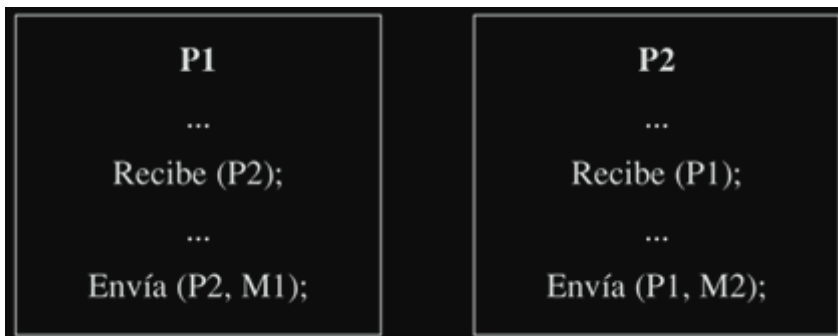
P1	P2
...	...
Solicita 80 Kbytes;	Solicita 70 Kbytes;
...	...
Solicita 60 Kbytes;	Solicita 80 Kbytes;

A los ambos procesos ejecutar la 2da instrucción, tenemos un interbloqueo porque nos quedamos sin memoria. Esto se soluciona con la memoria virtual.

Consumible

Un recurso consumible es aquél que puede crearse (producirse) y destruirse (consumirse). Normalmente, no hay límite en el número de recursos consumibles de un determinado tipo.

Algunos ejemplos de recursos consumibles son las interrupciones, las señales, los mensajes y la información en buffers de E/S.



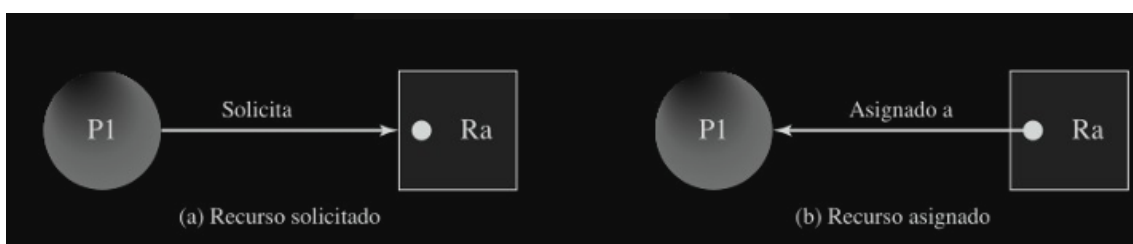
Si la función que recibe es bloqueante, tenemos un interbloqueo.

La causa del interbloqueo es un error de diseño. Estos errores pueden ser bastante sutiles y difíciles de detectar.

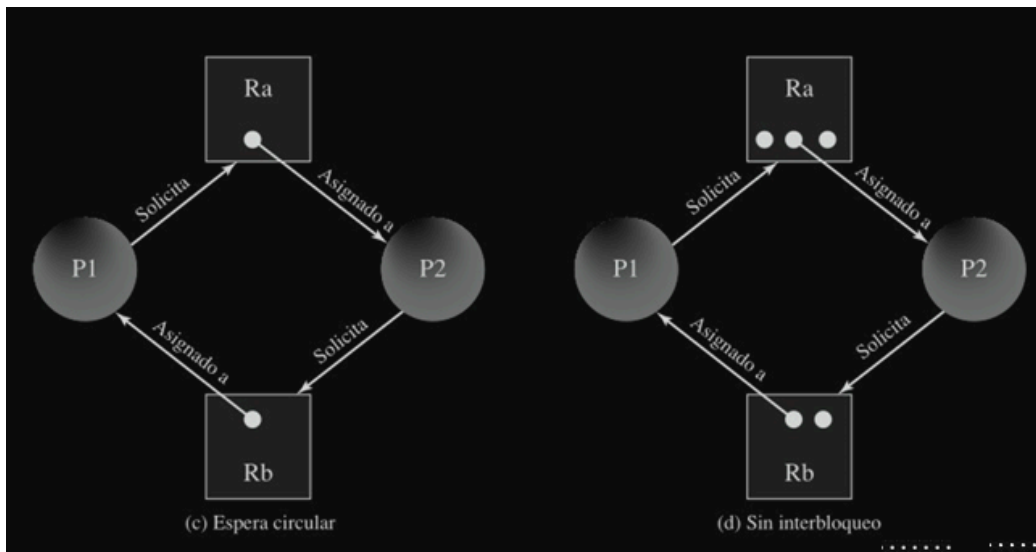
Estrategia	Política de reserva de recursos	Esquemas alternativos	Principales ventajas	Principales desventajas
Prevención	Conservadora; infrutiliza recursos	Solicitud simultánea de todos los recursos	- Adecuada para procesos que realizan una sola ráfaga de actividad - No es necesaria la expropiación	- Ineficiente - Retrasa la iniciación del proceso - Los procesos deben conocer sus futuros requisitos de recursos
		Expropiación	Conveniente cuando se aplica a recursos cuyo estado se puede guardar y restaurar fácilmente	Expropia con más frecuencia de lo necesario
		Ordenamiento de recursos	- Es posible asegurarlo mediante comprobaciones en tiempo de compilación - No necesita cálculos en tiempo de ejecución ya que el problema se resuelve en el diseño del sistema	Impide solicitudes graduales de recursos
Predicción	A medio camino entre la detección y la prevención	Asegura que existe al menos un camino seguro	No es necesaria la expropiación	- El SO debe conocer los futuros requisitos de recursos de los procesos - Los procesos se pueden bloquear durante largos periodos
Detección	Muy liberal; los recursos solicitados se conceden en caso de que sea posible	Se invoca periódicamente para comprobar si hay interbloqueo	- Nunca retrasa la iniciación del proceso - Facilita la gestión en línea	Pérdidas inherentes por expropiación

Grafos de asignación de recursos.

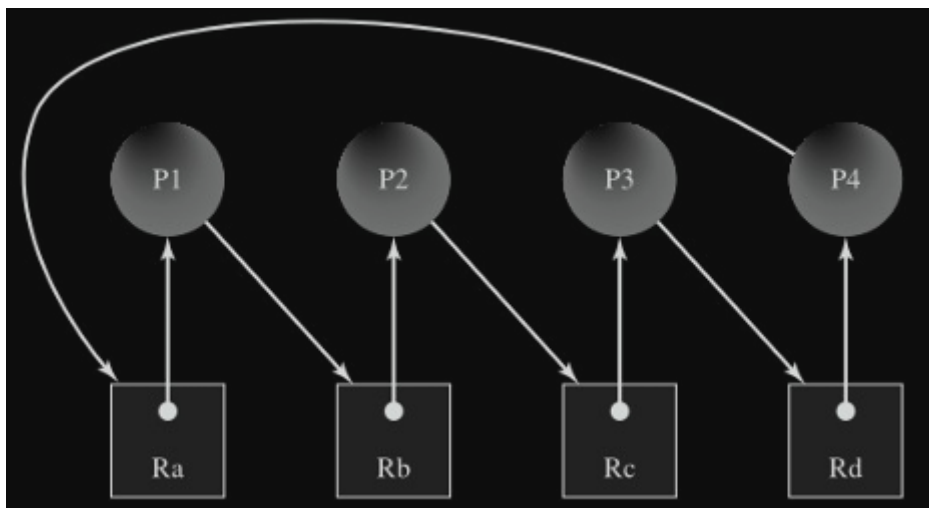
El grafo de asignación de recursos es un grafo dirigido que representa el estado del sistema en lo que se refiere a los recursos y los procesos, de tal forma que cada proceso y cada recurso se representa por un nodo.



La figura muestra un ejemplo de interbloqueo. Hay sólo una unidad de cada recurso Ra y Rb. El proceso P1 mantiene Rb y solicita Ra, mientras que P2 mantiene Ra pero pide Rb.



Este otro grafo representa un interbloqueo



Condiciones para el interbloqueo

1. Exclusión mutua: Solo un proceso utiliza un recurso en cada momento. Ningún proceso puede acceder a una unidad de un recurso que se haya asignado a otro.
2. Retención y espera. Un proceso puede mantener los recursos asignados mientras espera la asignación de otros recursos.
3. Sin expropiación. No se puede forzar la expropiación de un recurso a un proceso que lo posee.
4. Espera circular. Existe una lista cerrada de procesos, de tal manera que cada proceso posee al menos un recurso necesitado por el siguiente proceso de la lista.

Si se cumplen las 3 primeras, es probable que ocurra la 4ta.

Región fatal: Región donde los procesos se verán involucrados en un interbloqueo, solo existe si se cumplen conjuntamente las 3 primeras condiciones.

Posibilidad de interbloqueo	Existencia de interbloqueo
1. Exclusión mutua	1. Exclusión mutua
2. Sin expropiación	2. Sin expropiación
3. Retención y espera	3. Retención y espera
	4. Espera circular

Prevención del interbloqueo

La idea es diseñar un sistema de manera que se excluya la posibilidad del interbloqueo.

- Exclusión mutua: Es inevitable.
- Retención y espera: Se puede eliminar estableciendo que un proceso debe solicitar al mismo tiempo todos los recursos requeridos, y se lo bloquea hasta que los obtenga. No es muy práctica.
- Sin expropiación: En casos de recursos que se pueden salvar y restaurar más tarde, se le pide al proceso que si tiene muchos recursos los libere, para que luego los pida devuelta.
- Espera Circular: Se puede impedir definiendo el orden lineal entre los distintos tipos de recursos. La condición de espera circular se puede impedir definiendo un orden lineal entre los distintos tipos de recursos. Si a un proceso le han asignado recursos de tipo R, posteriormente puede pedir sólo aquellos recursos cuyo tipo tenga un orden posterior al de R. Puede ser ineficiente.

Unidad 6: Clase 16.2

Predicción del Interbloqueo

La **predicción del interbloqueo** es una estrategia que permite las tres condiciones necesarias para el interbloqueo (exclusión mutua, retención y espera, sin expropiación), pero toma decisiones dinámicas para **asegurarse de que nunca se alcanza el punto del interbloqueo**. Permite más **conurrencia** que la prevención, que restringe las solicitudes de recurso para evitar una de las cuatro condiciones.

La predicción requiere el **conocimiento de las futuras solicitudes de recursos** del proceso. Se basa en dos técnicas:

1. **No iniciar un proceso** si sus demandas podrían llevar al interbloqueo.
2. **No conceder una petición adicional** de un recurso si esta asignación podría provocar un interbloqueo.

Conceptos Clave para la Predicción

Para la predicción, se definen los siguientes vectores y matrices:

Recursos = $\mathbf{R} = (R_1, R_2, \dots, R_m)$		cantidad total de cada recurso en el sistema
Disponibles = $\mathbf{D} = (D_1, D_2, \dots, D_m)$		cantidad total de cada recurso no asignada a ningún proceso
Necesidad = $\mathbf{N} =$	$\begin{vmatrix} N_{11} & N_{12} & \dots & N_{1m} \\ N_{21} & N_{22} & \dots & N_{2m} \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ N_{n1} & N_{n2} & \dots & N_{nm} \end{vmatrix}$	N_{ij} = necesidades del proceso i con respecto al recurso j
Asignación = $\mathbf{A} =$	$\begin{vmatrix} A_{11} & A_{12} & \dots & A_{1m} \\ A_{21} & A_{22} & \dots & A_{2m} \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ A_{n1} & A_{n2} & \dots & A_{nm} \end{vmatrix}$	A_{ij} = asignación actual al proceso i con respecto al recurso j

Denegación de la Iniciación del Proceso

Se inicia un nuevo proceso P_{n+1} solo si sus necesidades máximas, sumadas a las necesidades máximas de todos los procesos actuales, pueden ser satisfechas por los recursos totales del sistema. Esta estrategia es ineficiente porque asume el peor caso: que todos los procesos solicitarán sus necesidades máximas simultáneamente.

Denegación de Asignación de Recursos (Algoritmo del Banquero)

Esta estrategia, propuesta por Dijkstra, se centra en asegurar que el sistema se mantiene en un **estado seguro**.

- **Estado:** Refleja la asignación actual de recursos a procesos (vectores $\mathbf{R}$ y $\mathbf{D}$, y matrices $\mathbf{N}$ y $\mathbf{A}$).
- **Estado Seguro:** Es aquel en el que hay **al menos una secuencia de asignación de recursos** que permite que **todos los procesos se ejecuten al completo** sin interbloqueo.
- **Estado Inseguro:** Es un estado que no es seguro.

Estrategia: Cuando un proceso solicita recursos, se simula la concesión y se determina si el resultado es un estado seguro. Si es seguro, se concede la petición; en caso contrario, se bloquea el proceso.

Ejemplo de Determinación de un Estado Seguro : (No creo q lo tome pero igual)

	R1	R2	R3		R1	R2	R3		R1	R2	R3
P1	3	2	2	P1	1	0	0	P1	2	2	2
P2	6	1	3	P2	6	1	2	P2	0	0	1
P3	3	1	4	P3	2	1	1	P3	1	0	3
P4	4	2	2	P4	0	0	2	P4	4	2	0
Matriz de necesidad N				Matriz de asignación A				C - A			
	R1	R2	R3		R1	R2	R3				
	9	3	6		0	1	1				
Vector de recursos R				Vector de disponibles D							

(a) Estado inicial

	R1	R2	R3		R1	R2	R3		R1	R2	R3
P1	3	2	2	P1	1	0	0	P1	2	2	2
P2	0	0	0	P2	0	0	0	P2	0	0	0
P3	3	1	4	P3	2	1	1	P3	1	0	3
P4	4	2	2	P4	0	0	2	P4	4	2	0
Matriz de necesidad N				Matriz de asignación A				C - A			
	R1	R2	R3		R1	R2	R3				
	9	3	6		6	2	3				
Vector de recursos R				Vector de disponibles D							

(b) P2 ejecuta hasta completarse

	R1	R2	R3		R1	R2	R3		R1	R2	R3
P1	0	0	0	P1	0	0	0	P1	0	0	0
P2	0	0	0	P2	0	0	0	P2	0	0	0
P3	3	1	4	P3	2	1	1	P3	1	0	3
P4	4	2	2	P4	0	0	2	P4	4	2	0
Matriz de necesidad N				Matriz de asignación A				C - A			
	R1	R2	R3		R1	R2	R3				
	9	3	6		7	2	3				
Vector de recursos R				Vector de disponibles D							

(c) P1 ejecuta hasta completarse

	R1	R2	R3		R1	R2	R3		R1	R2	R3
P1	0	0	0	P1	0	0	0	P1	0	0	0
P2	0	0	0	P2	0	0	0	P2	0	0	0
P3	0	0	0	P3	0	0	0	P3	0	0	0
P4	4	2	2	P4	0	0	2	P4	4	2	0
Matriz de necesidad N				Matriz de asignación A				C - A			
	R1	R2	R3		R1	R2	R3		R1	R2	R3
	9	3	6		9	3	4				
Vector de recursos R				Vector de disponibles D							

(d) P3 ejecuta hasta completarse

Dado que se pudo encontrar una secuencia de ejecución que permite que todos los procesos terminen (P2, P1, P3, P4), el estado inicial es **seguro**.

Restricciones de la Predicción:

- Los **requisitos máximos de recursos** de cada proceso deben establecerse por anticipado.
- Los procesos deben ser **independientes** (el orden de ejecución no debe estar restringido por sincronización).
- Debe haber un **número fijo de recursos** a asignar.
- Ningún proceso puede terminar mientras mantenga recursos.

Detección del Interbloqueo

La **detección del interbloqueo** es la estrategia menos restrictiva: **no limita el acceso a los recursos** ni restringe las acciones de los procesos. Los recursos se conceden siempre que sea posible. Periódicamente, el sistema operativo ejecuta un algoritmo para detectar la condición de **espera circular**.

Algoritmo de Detección

La comprobación se puede realizar por cada petición de recurso (detección temprana) o con menos frecuencia (menor consumo de procesador).

El algoritmo de detección (ej., el de Coffman) utiliza la matriz de **Asignación A** y el vector de **Disponibles D**, además de una matriz de **Solicitud S**, donde S_{ij} es la cantidad de recurso j solicitada por el proceso i .

Pasos del Algoritmo:

1. Marcar cada proceso sin recursos asignados (fila de A a cero).
2. Inicializar un vector temporal T con el vector **Disponibles (D)**.
3. Buscar un proceso i **sin marcar** cuyas solicitudes (S_i) sean menores o iguales que T. Si no se encuentra, el algoritmo termina.

- Si se encuentra, se **marca** el proceso i, y se **suman los recursos asignados** (A_i) a T ($T = T + A_i$). Se vuelve al paso 3.

	R1	R2	R3	R4	R5
P1	0	1	0	0	1
P2	0	0	1	0	1
P3	0	0	0	0	1
P4	1	0	1	0	1

Matriz de solicitud S

	R1	R2	R3	R4	R5
P1	1	0	1	1	0
P2	1	1	0	0	0
P3	0	0	0	1	0
P4	0	0	0	0	0

Matriz de asignación A

R1	R2	R3	R4	R5
2	1	1	2	1

Vector de recursos

R1	R2	R3	R4	R5
0	0	0	0	1

Vector de disponibles

Resultado: Existe un **interbloqueo si y solo si hay procesos sin marcar** al final del algoritmo⁴⁰. Los procesos sin marcar están en un interbloqueo⁴¹. El algoritmo asume que un proceso cuyas peticiones se satisfacen se ejecuta hasta terminar y libera todos sus recursos⁴².

Recuperación

Una vez detectado el interbloqueo, se necesita una estrategia de recuperación. Las opciones, en orden de sofisticación creciente, son:

- Abortar todos los procesos** involucrados en el interbloqueo (la solución más común).
- Retroceder cada proceso** en interbloqueo a un punto de control (checkpoint) y reorganizar todos los procesos (riesgo de repetición del interbloqueo).
- Abortar sucesivamente los procesos** en el interbloqueo, basándose en criterios de coste mínimo, hasta que este cese.
- Expropiar sucesivamente los recursos**, basándose en criterios de coste mínimo, hasta que el interbloqueo cese. El proceso expropiado debe retroceder a un punto anterior a la adquisición del recurso.

Criterios de Selección (para abortar/expropiar): el proceso con menor tiempo de procesador consumido, menor salida producida, mayor tiempo restante estimado, menor número total de recursos asignados, o menor prioridad.

Estrategia Integrada de Tratamiento del Interbloqueo

Debido a que cada estrategia (prevención, predicción, detección) tiene ventajas y desventajas, puede ser más eficiente usar un **enfoque integrado**. Howard sugiere:

- Agrupar los recursos** en diversas clases.
- Utilizar la estrategia del **orden lineal** para **prevenir la espera circular** entre clases de recursos.
- Dentro de cada clase, usar el algoritmo más apropiado.

Clases de Recursos y Estrategias Sugeridas:

- **Espacio de intercambio:** Prevención (solicitud simultánea de todos los recursos) o Predicción.
 - **Recursos del proceso** (ej. dispositivos de cinta, ficheros): Predicción (esperando que declaren recursos anticipadamente) o Prevención (ordenamiento de recursos).
 - **Memoria principal:** Prevención por **expropiación** (expulsando el proceso a memoria secundaria).
 - **Recursos internos** (ej. canales de E/S): Prevención mediante ordenamiento de recursos.
-

El Problema de los Filósofos Comensales

Este problema, presentado por Dijkstra, es un caso de prueba estándar para evaluar las estrategias de sincronización y **representa los problemas de coordinación de recursos compartidos** en la programación concurrente.

Contexto: Cinco filósofos se sientan alrededor de una mesa redonda con un tenedor entre cada par. Cada filósofo necesita **dos tenedores** (recursos) para comer.

Objetivo: Diseñar un algoritmo que satisfaga la **exclusión mutua** (no dos pueden usar el mismo tenedor) evitando el **interbloqueo** y la **inanición**.



Solución Utilizando Semáforos (Primera Solución)

El algoritmo donde cada filósofo toma primero el tenedor de la izquierda y luego el de la derecha conduce al **interbloqueo**: si todos los filósofos están hambrientos al mismo tiempo, todos asirán el tenedor de la izquierda y esperarán por el de la derecha, que estará ocupado.

Solución sin Interbloqueo (Segunda Solución)

Para superar el riesgo de interbloqueo, se puede introducir un **semáforo** adicional llamado **comedor** con un valor inicial de 4 (máximo cuatro filósofos al mismo tiempo en el comedor). Esto asegura que **al menos un filósofo tendrá acceso a los dos tenedores**, haciendo la solución libre de interbloqueos e inanición.

Solución Utilizando un Monitor

La solución con un **monitor** define un monitor controlador_de_comensales con un vector de variables de condición (TenedorListo) y un vector booleano de disponibilidad (tenedor).

- El procedimiento obtiene_tenedores intenta conceder los tenedores izquierdo y derecho.
- Si un tenedor no está disponible, el filósofo utiliza **cwait** para encolarse en la variable de condición correspondiente, permitiendo que otro filósofo entre en el monitor.
- El procedimiento libera_tenedores utiliza **csignal** para despertar a un proceso que espera por un tenedor liberado.
- A diferencia de la primera solución de semáforo, esta solución del monitor **no sufre interbloqueos** porque solo puede haber **un proceso en el monitor en cada momento**. Esto garantiza que el primer filósofo que entre puede usar ambos tenedores antes de que el siguiente tenga la oportunidad de asir el tenedor compartido.

Unidad 6: Clase 16.3 Interbloqueos

Mecanismos de Concurrencia de UNIX

UNIX proporciona varios mecanismos de comunicación y sincronización entre procesos.

Mecanismo	Uso Principal	Características Clave
Tuberías (Pipes)	Comunicación de datos (modelo productor-consumidor).	Un buffer circular de tipo FIFO. Se crea una tubería de tamaño fijo en bytes. El proceso se bloquea si se intenta escribir en una tubería llena o leer de una vacía. El sistema operativo asegura la exclusión mutua . Hay tuberías con nombre (compartidas por procesos relacionados o no) y sin nombre (solo procesos relacionados).
Mensajes	Transferencia de mensajes entre procesos.	Un mensaje es un conjunto de bytes de tipo asociado. UNIX proporciona llamadas al sistema msgsnd y msgrcv para que los procesos puedan transferir mensajes. Cada proceso tiene una cola de mensajes que funciona como un buzón. Los mensajes se recuperan por orden de llegada o por su tipo . Un proceso se bloquea si intenta enviar a una cola llena o leer de una vacía, pero no se bloquea si intenta leer un tipo de mensaje que no está presente.
Memoria Compartida	La forma más rápida de	Un bloque de memoria virtual compartido por múltiples procesos. Los procesos leen y escriben directamente en este espacio. Las restricciones

	comunicación entre procesos.	de exclusión mutua no son parte de este mecanismo; deben ser proporcionadas por los procesos.
Semáforos	Sincronización y coordinación de procesos.	Generalización de semWait y semSignal que permite varias operaciones simultáneas atómicamente. Los semáforos se crean realmente como conjuntos. El semáforo tiene los siguientes elementos: • El valor actual del semáforo. • El identificador del último proceso que operó con el semáforo. • El número de procesos en espera de que el valor del semáforo sea mayor que su valor actual. • El número de procesos en espera de que el valor del semáforo sea cero. • Asociado con el semáforo están las colas de los procesos bloqueados en ese semáforo. Análisis de sem_op: > 0: Incrementa valor del semáforo, despierta todos los procesos en espera de que el valor del semáforo incremente. = 0: El núcleo comprueba el valor del semáforo, si es 0, continua con otras operaciones, sino, incrementa el número de procesos en espera de que ese semáforo tenga valor 0 y suspende al proceso para que espere por el evento de que el valor del semáforo se haga igual a 0. < 0: Si su valor abs es mayor que el del semáforo, el núcleo suspende al proceso en espera del evento de que el valor del semáforo se incremente.
Señales	Informar a un proceso de la existencia de eventos asíncronos .	Similar a las interrupciones hardware, pero no emplean prioridades . Se usan para disparar acciones en otros procesos. El proceso responde realizando una acción por defecto (ej., terminación), ejecutando una función de manejo de la señal o ignorándola . No pueden

		encolarse, ya que solo se mantiene un único bit por cada señal.
--	--	---

Dejó las señales acá aunque no creo que las tome:

Valor	Nombre	Descripción
01	SIGHUP	Desconexión; enviada al proceso cuando el núcleo asume que el usuario de ese proceso no está haciendo trabajo útil
02	SIGINT	Interrupción
03	SIGQUIT	Abandonar; enviada por el usuario para provocar la parada del proceso y la generación de un volcado de su memoria (<i>core dump</i>)
04	SIGILL	Instrucción ilegal
05	SIGTRAP	<i>Trap</i> de traza; activa la ejecución de código para realizar un seguimiento de la ejecución del proceso
06	SIGIOT	Instrucción IOT
07	SIGEMT	Instrucción EMT
08	SIGFPE	Excepción de coma flotante
09	SIGKILL	Matar; terminar el proceso

Valor	Nombre	Descripción
10	SIGBUS	Error de bus
11	SIGSEGV	Violación de segmento; el proceso intenta acceder a una posición fuera de su espacio de direcciones
12	SIGSYS	Argumento erróneo en una llamada al sistema
13	SIGPIPE	Escritura sobre una tubería que no tiene lectores asociados
14	SIGALRM	Alarma; emitida cuando un proceso desea recibir una señal cuando transcurra un determinado periodo de tiempo
15	SIGTERM	Terminación por software
16	SIGUSR1	Señal 1 definida por el usuario
17	SIGUSR2	Señal 2 definida por el usuario
18	SIGCHLD	Muerte de un proceso hijo
19	SIGPWR	Interrupción en el suministro de energía

Mecanismos de Concurrencia del Núcleo de Linux

Linux (versión 2.6 y posteriores) incluye los mecanismos de UNIX, pero también un conjunto de mecanismos para la concurrencia cuando se ejecuta código en **modo núcleo**.

Operaciones Atómicas

Son funciones que garantizan que las operaciones sobre una variable son **atómicamente** (sin interrupción ni interferencia).

- En un **uniprocador**, el hilo no puede ser interrumpido.
- En un **multiprocador**, se establece un **cerrojo** en la variable.
- Hay operaciones con **enteros** (usando el tipo especial `atomic_t`) y con **mapas de bits**. El tipo `atomic_t` se usa habitualmente para implementar **contadores**.

Por si las dudas dejo la tabla con las funciones aunque dudo que lo tome:

Operaciones atómicas con enteros	
ATOMIC_INT (int i)	En una declaración: inicia un atomic_t con el valor i
int atomic_read(atomic_t *v)	Lee el valor entero de v
void atomic_set(atomic_t *v, int i)	Asigna el valor entero i a v
void atomic_add(int i, atomic_t *v)	Suma i a v
void atomic_sub(int i, atomic_t *v)	Resta i de v
void atomic_inc(atomic_t *v)	Suma 1 a v
void atomic_dec(atomic_t *v)	Resta 1 de v
int atomic_sub_and_test(int i, atomic_t *v)	Resta i de v; devuelve 1 si el resultado es cero; y 0 en caso contrario
int atomic_dec_and_test(atomic_t *v)	Resta 1 de v; devuelve 1 si el resultado es cero; y 0 en caso contrario

Operaciones atómicas con mapas de bits	
void set_bit(int n, void *dir)	Pone a 1 el bit n del mapa de bits apuntado por dir
void clear_bit(int n, void *dir)	Pone a 0 el bit n del mapa de bits apuntado por dir
void change_bit(int n, void *dir)	Invierte el valor del bit n del mapa de bits apuntado por dir
int test_and_set_bit(int n, void *dir)	Pone a 1 el bit n del mapa de bits apuntado por dir; devuelve su valor previo
int test_and_clear_bit(int n, void *dir)	Pone a 0 el bit n del mapa de bits apuntado por dir; devuelve su valor previo
int test_and_change_bit(int n, void *dir)	Invierte el valor del bit n del mapa de bits apuntado por dir; devuelve su valor previo
int test_bit(int n, void *dir)	Devuelve el valor del bit n del mapa de bits apuntado por dir

Cerrosjos Cíclicos (*Spinlocks*)

La técnica más frecuente para proteger una **sección crítica** en Linux.

- Solo un hilo puede adquirir un cerrojo cíclico a la vez. Los hilos que esperan a adquirirlo siguen **intentándolo cíclicamente** (espera activa).
- Son efectivos cuando se prevé que el tiempo de espera será **muy breve**.
- **Modalidades de Cerrosjos Cíclicos Básicos:**
 - **Sencillo:** No afecta al estado de las interrupciones.
 - **_irq:** Prohíbe las interrupciones en el procesador local.
 - **_irqsave:** Guarda el estado actual de las interrupciones antes de adquirir el cerrojo y lo restaura al liberarlo.
 - **_bh:** Se usa para inhabilitar, y después habilitar, la ejecución de rutinas de **mitad inferior** (*bottom half*) para evitar conflictos.

void spin_lock(spinlock_t *cerrojo)	Adquiere el cerrojo especificado, comprobando cíclicamente el valor hasta que esté disponible
void spin_lock_irq(spinlock_t *cerrojo)	Igual que spin_lock, pero prohibiendo las interrupciones en el procesador local
void spin_lock_irqsave(spinlock_t *cerrojo, unsigned long indicadores)	Igual que spin_lock_irq, pero guardando el estado actual de las interrupciones en indicadores
void spin_lock_bh(spinlock_t *cerrojo)	Igual que spin_lock, pero prohibiendo la ejecución de mitades inferiores
void spin_unlock(spinlock_t *cerrojo)	Libera el cerrojo especificado
void spin_unlock_irq(spinlock_t *cerrojo)	Libera el cerrojo especificado y habilita las interrupciones locales

.....

void spin_unlock_irqrestore(spinlock_t *cerrojo, unsigned long indicadores)	Libera el cerrojo especificado y restaura el estado de las interrupciones locales
void spin_unlock_bh(spinlock_t *cerrojo)	Libera el cerrojo especificado y habilita la ejecución de mitades inferiores
void spin_lock_init(spinlock_t *cerrojo)	Inicia el cerrojo especificado
int spin_trylock(spinlock_t *cerrojo)	Intenta adquirir el cerrojo especificado, devolviendo un valor distinto de cero si lo posee otro proceso y cero en caso contrario
int spin_is_locked(spinlock_t *cerrojo)	Devuelve un valor distinto de cero si el cerrojo lo posee otro proceso y cero en caso contrario

Cerrojo Cíclico de Lectura-Escritura:

Permite que **múltiples hilos accedan simultáneamente para lectura**, pero solo un **único hilo para escritura** (acceso exclusivo). Este cerrojo **favorece a los lectores**.

Contador	Indicador	Interpretación
0	1	El cerrojo cíclico está disponible para su uso
0	0	El cerrojo cíclico se ha adquirido para la escritura de 1 hilo
n ($n > 0$)	0	El cerrojo cíclico se ha adquirido para la lectura de n hilos
n ($n > 0$)	1	Inválido

Semáforos en el Núcleo

Linux proporciona una implementación de semáforos para su **uso interno**, que son más eficientes que los semáforos visibles para el usuario. Proporciona tres tipos:

- **Semáforos binarios (MUTEX):** Inicializados a 1 (abierto) o 0 (cerrado).
- **Semáforos con contador.**
- **Semáforos de lectura-escritura:** Permite múltiples lectores o un único escritor.

Operaciones down (semWait):

1. down(): Operación tradicional, el hilo se bloquea si no está disponible.
2. down_interruptible(): Permite al hilo **recibir y responder a una señal** mientras está bloqueado, pudiendo ser abortada la función.
3. down_trylock(): Intenta adquirir el semáforo **sin bloquear** el hilo.

Barreras

Mecanismo para **fijar el orden de ejecución** de lecturas y escrituras, que pueden ser modificadas por el compilador o el hardware para optimizar el rendimiento.

- **rmb()**: Barrera de **lectura**.
- **wmb()**: Barrera de **escritura**.
- **mb()**: Barrera de **lectura y escritura**.
- **barrier()**: Solo controla el comportamiento del **compilador**.

Las operaciones de barrera tienen 2 aspectos importantes:

1. Las barreras están relacionadas con las instrucciones máquina.
2. Las operaciones rmb, wmb y mb dictan el comportamiento tanto del compilador como del procesador.

Funciones de Sincronización de Hilos de Solaris

Solaris implementa sus funciones de sincronización **dentro del núcleo** (para hilos de núcleo) y en la **biblioteca de hilos** (para hilos de usuario). Cuando se crea un objeto de sincronización, sólo se pueden realizar 2 operaciones: Entrar (adquirir un cerrojo) y liberar (desbloquearlo). No hay mecanismos automáticos en el núcleo que aseguren la exclusión mutua o impidan el interbloqueo; el programador debe usar las funciones apropiadas.

Las funciones de sincronización de hilos de Solaris son:

- **Cerrojos de exclusión mutua (Mutex)**: Asegura que solo un hilo accede al recurso protegido. Por defecto, usa un **cerrojo cíclico** como política bloqueante.

```
mutex_enter() Adquiere el cerrojo, bloqueando potencialmente si ya lo
posee otro hilo
mutex_exit() Libera el cerrojo, desbloqueando potencialmente a un hilo
en espera
mutex_tryenter() Adquiere un cerrojo si no lo posee otro hilo
```

- **Semáforos**: Proporciona los clásicos **semáforos con contador** (sema_p(), sema_v(), sema_tryv()).
- **Cerrojos de múltiples lectores y un único escritor (Lectura/Escritura)**: Permite **múltiples lectores** concurrentes, pero solo un **único escritor** (excluyendo a lectores y otros escritores).

Las funciones son las siguientes:

rw_enter() Intenta adquirir un cerrojo como lector o escritor

rw_exit() Libera un cerrojo como lector o escritor

rw_tryenter() Decrementa el semáforo si no se requiere un bloqueo

rw_downgrade() Un hilo que ha adquirido un cerrojo de escritura lo convierte en uno de lectura. Cualquier escritor en espera permanece esperando hasta que este hilo libere el cerrojo. Si no hay escritores en espera, la función despierta a los lectores pendientes, si los hay

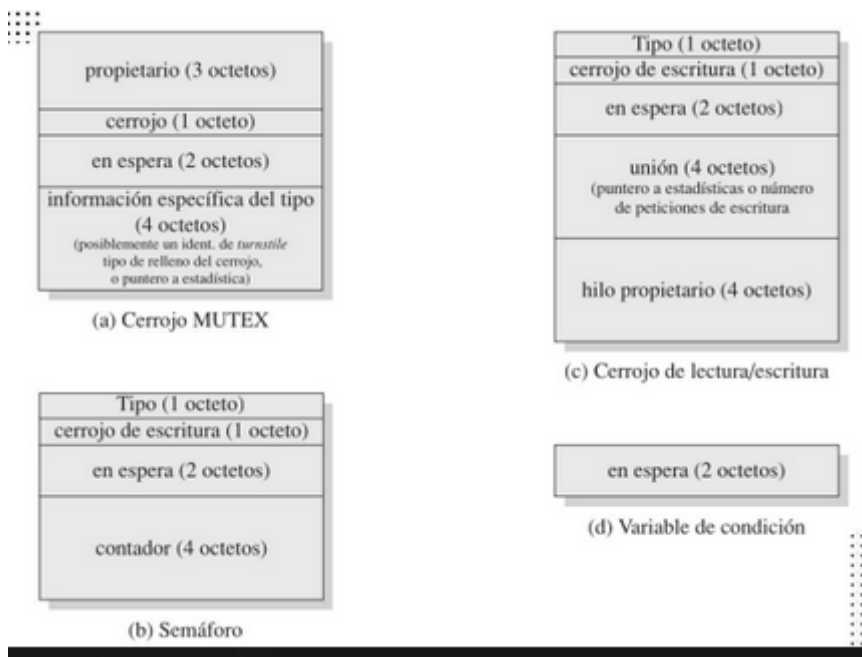
rw_tryupgrade() Intenta convertir un cerrojo de lectura en uno de escritura

- **Variables de Condición:** Se utilizan para **esperar hasta que se cumpla una determinada condición**. Deben usarse en conjunción con un **cerrojo mutex** (implementando un monitor). Sus primitivas son:

cv_wait() Bloquea hasta que se activa la condición

cv_signal() Despierta uno de los hilos bloqueados en cv_wait()

cv_broadcast() Despierta todos los hilos bloqueados en cv_wait()



Mecanismos de Concurrency de Windows

Los dos métodos de sincronización más importantes son los objetos de sincronización y los de sección crítica.

Funciones de Espera

Permiten a un hilo **bloquear su propia ejecución** hasta que se cumplen ciertos criterios. El hilo transita al **estado de espera** y no consume tiempo de procesador.

- **WaitForSingleObject:** Espera por un solo objeto hasta que el objeto está en el **estado de señalado** o transcurre el plazo máximo de espera.

Objetos de Sincronización

Son el mecanismo principal del ejecutivo de Windows. Cada objeto puede estar en el **estado de señalado** o de **no señalado**. Un hilo se bloquea si el objeto está en el estado de no señalado y se desbloquea cuando transita al estado de señalado.

Tipo de Objeto	Definición	Pasa al Estado de Señalado Cuando	Efecto sobre Hilos en Espera
Evento	Aviso de que ha ocurrido un evento.	Un hilo genera un evento.	Desbloquea a todos.
Mutex	Proporciona exclusión mutua (semáforo binario).	El hilo propietario u otro hilo libera el mutex.	Desbloquea un hilo. Se pueden utilizar para sincronizar hilos en procesos diferentes .
Semáforo	Contador que regula el número de hilos que pueden usar un recurso.	El contador del semáforo llega a cero (o es liberado).	Desbloquea un hilo. Es un semáforo con contador y se puede compartir entre procesos.
Temporizador con espera	Contador que registra el paso del tiempo.	Se cumple el tiempo especificado.	Desbloquea a todos.

Notificación de cambio en fichero	Una notificación de cambios en el sistema de ficheros	Ocurre un cambio en el sistema de ficheros que encaja con los criterios de filtro de ese objeto	Se desbloquea un hilo
-----------------------------------	---	---	-----------------------

Entrada de consola	Una ventana de texto en la pantalla (por ejemplo, usada para manejar E/S de pantalla en una aplicación MS-DOS)	Hay entrada disponible para procesar	Se desbloquea un hilo
Trabajo	Una instancia de un fichero abierto o un dispositivo de E/S	Se completa una operación de E/S	Desbloquea a todos
Notificación sobre el recurso de memoria	Una notificación de un cambio en el recurso de memoria	Se produce el tipo de cambio especificado en la memoria física	Desbloquea a todos
Proceso	Una invocación de un programa, incluyendo el espacio de direcciones y los recursos requeridos para ejecutar el programa	El último hilo termina	Desbloquea a todos
Hilo	Una entidad ejecutable dentro de un proceso	El hilo termina	Desbloquea a todos

Objetos de Sección Crítica

Proporcionan un mecanismo de sincronización similar al de los objetos mutex, pero **sólo pueden utilizarlos hilos del mismo proceso**. Son **más rápidos** y eficientes que los mutex para exclusión mutua dentro de un único proceso. Se usan las funciones EnterCriticalSection y LeaveCriticalSection para solicitar y liberar la posesión.

Unidad 7: Clase 17 Seguridad: Amenazas y Protección.

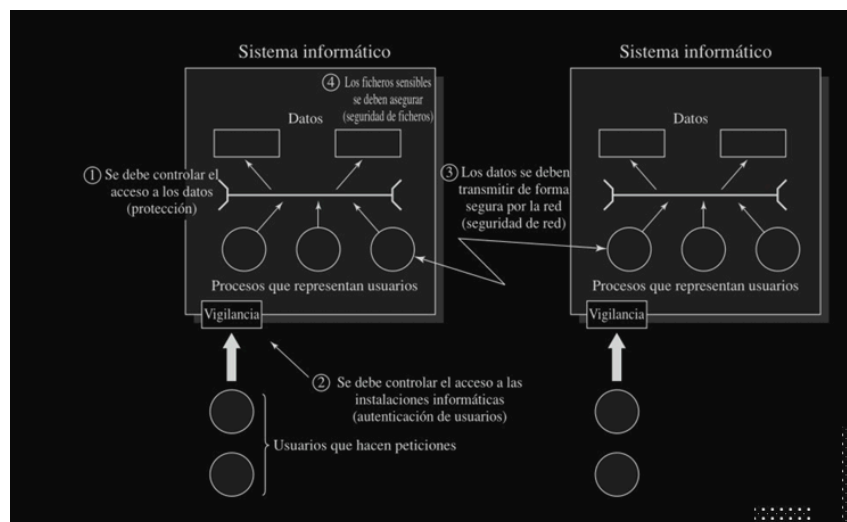
Requisitos y Amenazas de Seguridad

La seguridad de los sistemas informáticos se dirige a cuatro requisitos básicos:

Confidencialidad, Integridad, Disponibilidad y Autenticación.

Requisitos de Seguridad

- **Confidencialidad:** Requiere que la información sea accesible **solo para lectura** por partes autorizadas. Esto incluye observación, impresión o la simple revelación de la existencia de un elemento.
- **Integridad:** Requiere que el contenido de un sistema informático solo pueda **modificarse** por las partes autorizadas. Las modificaciones incluyen escritura, cambio de estado, borrado y creación.
- **Disponibilidad:** Requiere que los componentes de un sistema informático estén **disponibles** para todas aquellas partes autorizadas.
- **Autenticación:** Requiere que el sistema informático sea capaz de **verificar la identidad de los usuarios**.

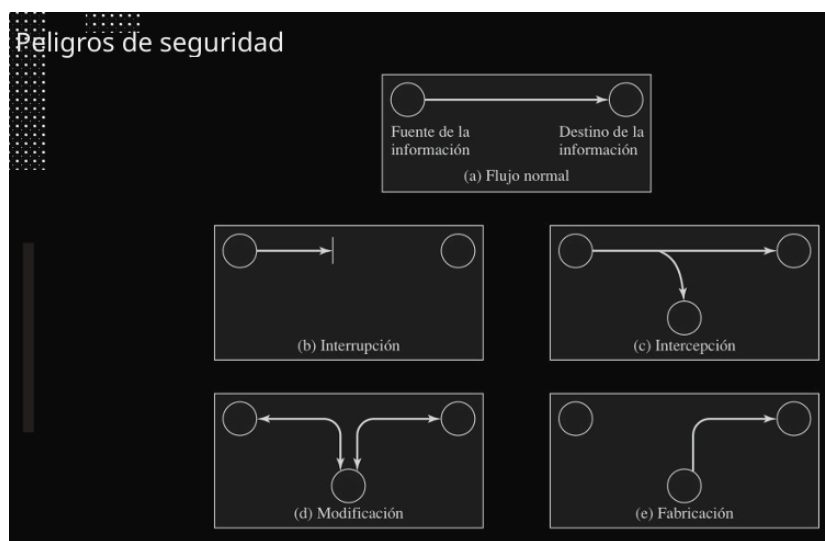


Tipos de Peligros (Ataques)

Los ataques contra la seguridad se clasifican según la función del sistema informático como proveedor de información (flujo de una **fuentes** a un **destino**).

1. **Interrupción:** Se destruye un componente del sistema o se encuentra **no disponible** o utilizable. Es un ataque centrado en la **Disponibilidad**.
 - *Ejemplo:* Destrucción de un disco duro o interrupción de un canal de comunicación.

2. **Intercepción:** Una parte no autorizada consigue **acceso** a un componente. Es un ataque dirigido hacia la **Confidencialidad**.
 - *Ejemplo:* Escucha en un canal de comunicación o copia ilícita de ficheros.
3. **Modificación:** Un elemento no autorizado no solo tiene acceso, sino que también es capaz de **modificar** un componente. Es un ataque dirigido hacia la **Integridad**.
 - *Ejemplo:* Cambiar valores en un fichero de datos o alterar un programa.
4. **Fabricación:** Un elemento no autorizado **inserta objetos extraños** en el sistema. Estos son ataques contra la **Autenticación**.
 - *Ejemplo:* Inserción de mensajes falsos en una red o inclusión de un registro falso en un fichero.



Peligros en los Componentes de un Sistema

Los componentes de un sistema informático se clasifican en: **Hardware, Software, Datos y Líneas de comunicaciones y redes.**

Peligros por Componente y Requisito

	Disponibilidad	Privacidad	Integridad/Autenticación
Hardware	Equipamiento robado o deshabilitado, por lo tanto denegación de servicio.		
Software	Borrado de programas, denegación de acceso a los usuarios.	Copia no autorizada de software.	Modificación de un programa, bien para hacer que falle durante la ejecución o para que realice una tarea diferente.
Datos	Borrar ficheros, denegación de acceso a los usuarios.	Lectura no autorizada de datos. Un análisis estadístico de los datos que revele la información subyacente.	Modificación de los ficheros existentes o creación de nuevos ficheros.
Líneas de comunicación	Borrado o destrucción de mensajes. Las líneas de comunicación o redes no se encuentran disponibles.	Lectura de mensajes. Observación de los patrones de tráfico de mensajes.	Modificación, borrado, reordenación o duplicación de mensajes. Fabricación de mensajes falsos.

Ataques a Redes (Líneas de Comunicación)

Ataques pasivos

Intenta aprender o hacer uso de la información del sistema, pero **no altera los recursos**. Son **difíciles de detectar**. El énfasis se centra en su **prevención**, habitualmente por medio de **cifrado**. Existe un **segundo** tipo, que son los ataques de análisis de tráfico.

- Tipos: **Lectura de los contenidos de los mensajes** y **Análisis de tráfico** (observación de patrones de mensajes, identidad y ubicación de ordenadores).

Ataque Activo:

Intenta **alterar los recursos** del sistema o afectar su operativa. Es **difícil de prevenir** completamente. El objetivo es la **detección** y la recuperación.

Categorías:

- **Enmascaramiento**: Un elemento intenta hacerse pasar por otro (suplantación de identidad).
- **Reenvío**: Captura pasiva de una unidad de datos y su posterior retransmisión para producir un efecto no autorizado.
- **Modificación de mensajes**: Se altera parte de un mensaje válido, o se borran/reordenan mensajes.
- **Denegación de servicio**: Previene o imposibilita el uso normal o la gestión de las instalaciones de comunicaciones.

Protección y Control de Acceso

La **protección** es la necesidad introducida por la **multiprogramación** y la posibilidad de compartir recursos (memoria, dispositivos, programas, datos) entre usuarios.

Niveles de Protección

El sistema operativo debe ofrecer niveles de protección que van desde **Sin protección alguna** hasta **Uso limitado de un objeto**.

- Sin protección alguna. Apropiado por los procedimientos que son sensibles de ejecutar en instantes diferentes.
- **Aislamiento**: Cada proceso opera separadamente, sin compartición ni comunicación alguna.
- **Compartición completa o sin compartición**: El propietario del objeto declara si va a ser público o privado. En público, cualquier proceso puede acceder a él, en privado solo los del propietario.
- **Compartición vía limitaciones acceso**: El sistema operativo actúa como **guardián** o vigilante, verificando la permisibilidad de cada acceso de cada usuario sobre cada objeto.
- **Acceso vía capacidades dinámicas**. Permite la creación dinámica de derechos de acceso a los objetos.
- **Uso limitado de un objeto**: Limita no solo el acceso sino también el uso que se puede realizar del objeto (ej., ver un documento sensible, pero no imprimirlo).

Protección de la Memoria

La **separación de los espacios de memoria** de los diferentes procesos es un requisito fundamental en entornos multiprogramados para el correcto funcionamiento. Los esquemas de memoria virtual (segmentación o paginación) proporcionan los mecanismos para esta tarea. Si se desea la **compartición**, el mismo segmento o página puede aparecer en más de una tabla.

Control de Acceso Orientado a Usuario

La técnica más común es el **registro o conexión de usuario (user log on)**, que requiere el **identificador usuario (ID)** y una **contraseña (password)**. Este sistema es notablemente **poco fiable** ya que los usuarios pueden olvidarse las contraseñas o intencionalmente revelarlas.

- El control de acceso puede ser **centralizado** (un servicio de conexión de red) o **descentralizado** (el ordenador destino realiza el mecanismo de acceso).

Control de Acceso Orientado a los Datos

Una vez conectado, el sistema debe controlar el acceso a recursos granulares (ficheros, registros, campos). La decisión de acceso se basa en el **perfil de usuario**, la **parte específica de los datos** y la información ya divulgada.

El modelo general es la **matriz de acceso**. Los elementos básicos del modelo son:

- **Sujeto**: Un elemento capaz de acceder a los objetos, asimilado al **proceso** que representa al usuario o aplicación.
- **Objeto**: Todo elemento sobre el que se accede de forma controlada (ficheros, programas, segmentos de memoria).
- **Derecho de acceso**: La forma en la cual un objeto es accedido por un sujeto (lectura, escritura, ejecución).

	Fichero 1	Fichero 2	Fichero 3	Fichero 4	Account 1	Account 2
Usuario A	Own R W		Own R W		Información crédito	
Usuario B	R	Own R W	W	R	Información débito	Información crédito
Usuario C	R W	R		Own R W		Información débito

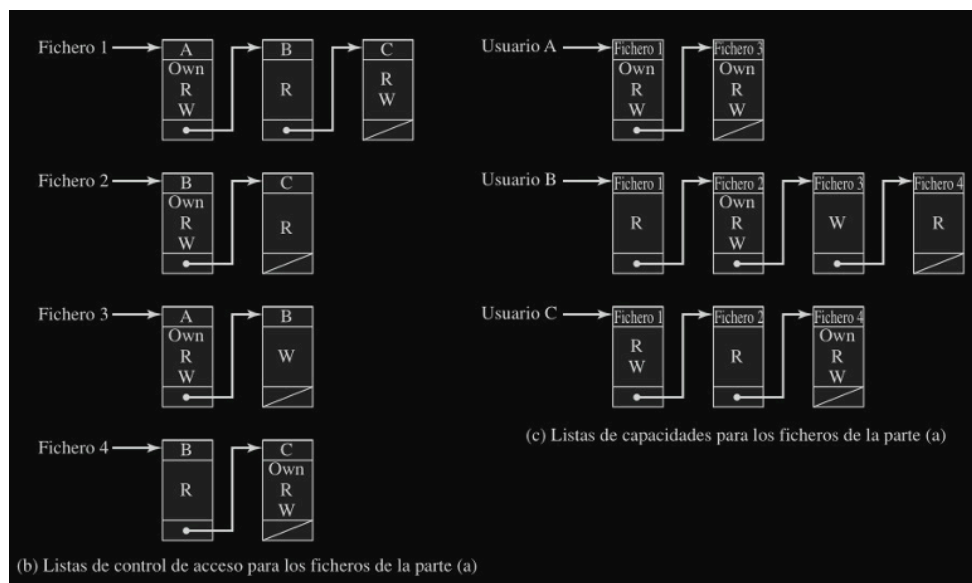
(a) Matriz de acceso

Una matriz de acceso es habitualmente bastante dispersa y se implementa mediante su descomposición, ya sea por columnas o definiendo listas de control de acceso.

Por cada objeto, hay una lista de control de acceso que muestra los usuarios y sus derechos de acceso.

La descomposición por filas lleva a la definición de los tickets de capacidades. Un ticket de capacidad específica objetos y operaciones autorizadas para un determinado usuario.

Habitualmente, el control de acceso a datos se encuentra descentralizado, esto es, controlando por sistemas de gestión de bases de datos basados en servidores.



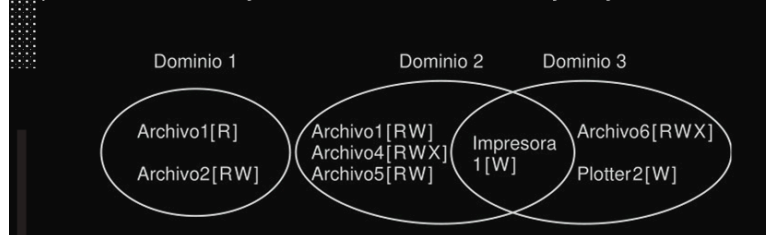
Mecanismos de Protección

Dominios de Protección

Un **dominio** es un conjunto de pares (**objeto**, **permisos**). Un permiso indica la autorización para realizar una de las operaciones posibles

- **POLA (Principle of Least Authority)**: Principio de **menor autoridad** o necesidad de saber, donde cada dominio tiene los **mínimos objetos y privilegios** necesarios para realizar su trabajo.

Tres dominios de protección, los objetos en cada dominio y los permisos disponibles en cada objeto: R (lectura), W (escritura) y X (ejecución).

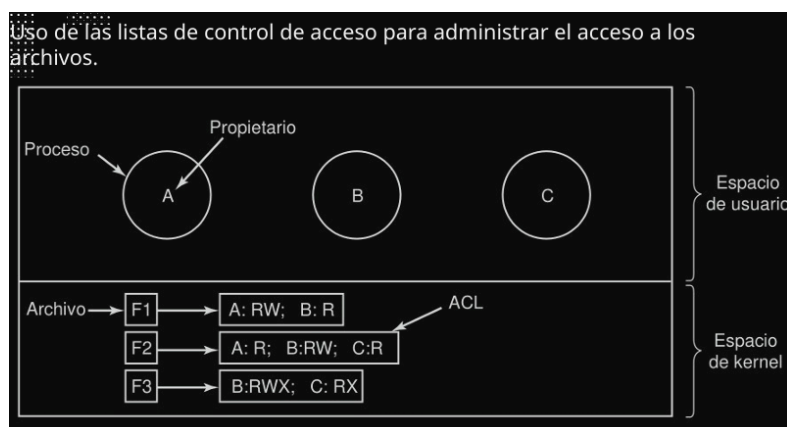


Dominio	Objeto								Dominio1	Dominio2	Dominio3
	Archivo1	Archivo2	Archivo3	Archivo4	Archivo5	Archivo6	Impresora1	Plotter2			
1	Lectura	Lectura Escritura								Enter	
2			Lectura	Lectura Escritura Ejecución	Lectura Escritura		Escritura				
3						Lectura Escritura Ejecución	Escritura	Escritura			

Listas de Control de Acceso (ACLs)

La **matriz de acceso** es dispersa y se implementa descomponiéndola por **columnas**.

- Una **Lista de control de acceso (ACL)** es una lista asociada a **cada objeto** que contiene todos los **dominios (usuarios)** que pueden acceder al objeto y la forma de hacerlo.



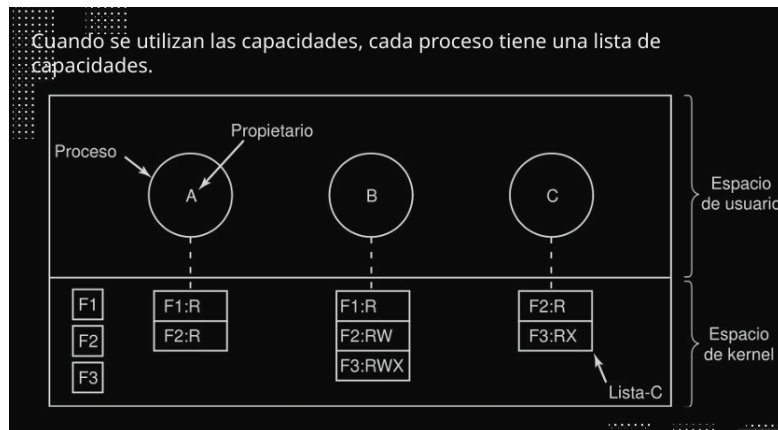
Una entrada ACL contiene entradas de la forma:

UID1, GID1: permisos1; UID2, GID2: permisos2; ...

Capacidades (Capabilities)

La implementación se realiza descomponiendo la **matriz de acceso** por **filas**.

- Una **lista de capacidades (Lista-C)** se asocia a **cada proceso** (su **dominio**) y contiene una lista de objetos que puede utilizar junto con las operaciones permitidas.
- Un **ticket de capacidad** especifica objetos y operaciones autorizadas para un determinado usuario.



Protección de Capacidades: Las listas de capacidades deben protegerse contra la alteración de los usuarios. Los métodos incluyen:

1. **Arquitectura etiquetada:** El hardware usa un bit (etiqueta) para indicar si una palabra de memoria contiene una capacidad.
2. Mantener la **Lista-C dentro del sistema operativo** (ej., como los descriptores de archivos en UNIX).
3. Mantener la Lista-C en espacio de usuario, pero administrarlas de manera **criptográfica** para evitar alteraciones.

Unidad 7: Clase 18 Intrusos y Software Malicioso

Intrusos

Los **intrusos** son uno de los dos peligros de seguridad más conocidos, generalmente denominados **hackers** o **crackers**.

Clases de Intrusos

Se pueden identificar tres clases de intrusos:

- **Enmascarado:** Un individuo no autorizado que penetra los controles de acceso para aprovechar una cuenta de usuario legítimo. Generalmente es un usuario externo.
- **Trasgresor:** Un **usuario legítimo** que accede a datos o recursos para los cuales no está autorizado, o utiliza sus privilegios de forma maliciosa. Generalmente es interno.
- **Usuario clandestino:** Un usuario que sobrepasa el control de supervisión del sistema para evadir la auditoría y el control de acceso, o para suprimir la recogida de registros de acceso. Puede ser interno o externo.

Técnicas de Intrusión

El objetivo es ganar acceso o incrementar privilegios, a menudo obteniendo la **contraseña** de otro usuario. Habitualmente, un sistema debe mantener un fichero que asocia las contraseñas con cada usuario autorizado.

Protección de Contraseñas

Para proteger el fichero de contraseñas, se usan contramedidas como:

- **Cifrado unidireccional:** El sistema almacena solo una forma cifrada y no reversible de la contraseña.

- **Control acceso:** El acceso al fichero de contraseñas se limita a muy pocas cuentas privilegiadas.

Técnicas para Conocer Contraseñas

Los intrusos usan diversos métodos para adivinar contraseñas, especialmente cuando pueden **evaluar un gran número de hipótesis de forma automática y verificar su validez** sin ser detectados:

- Intentar contraseñas por defecto.
- Ensayar contraseñas de pequeño tamaño (ej., 1 a 3 caracteres).
- Probar palabras del diccionario del sistema o listas de contraseñas habituales.
- Recolectar información personal del usuario (nombres, familia, etc.).
- Intentar con el número de teléfono del usuario.
- Probar números de matrículas.
- Utilización de **troyanos** para sobrepasar las restricciones de acceso.
- **Pinchado o escucha de la línea** entre un usuario remoto y el sistema destino (cuestión de seguridad física, evitable con cifrado).

Sistema de contraseñas

Es la primera línea de defensa contra intrusos. Necesita que un usuario proporcione tanto su nombre o ID como una contraseña. El identificar proporciona seguridad de la siguiente forma:

- El identificador determina si el usuario está autorizado a conseguir el acceso al sistema.
- El identificador determina los privilegios asociados al usuario.
- El identificador también se utiliza para un control de acceso discrecional. Por ejemplo indicando la lista de los identificadores de otros usuarios, un usuario puede otorgar permisos para que estos puedan leer ficheros que le pertenecen a él.

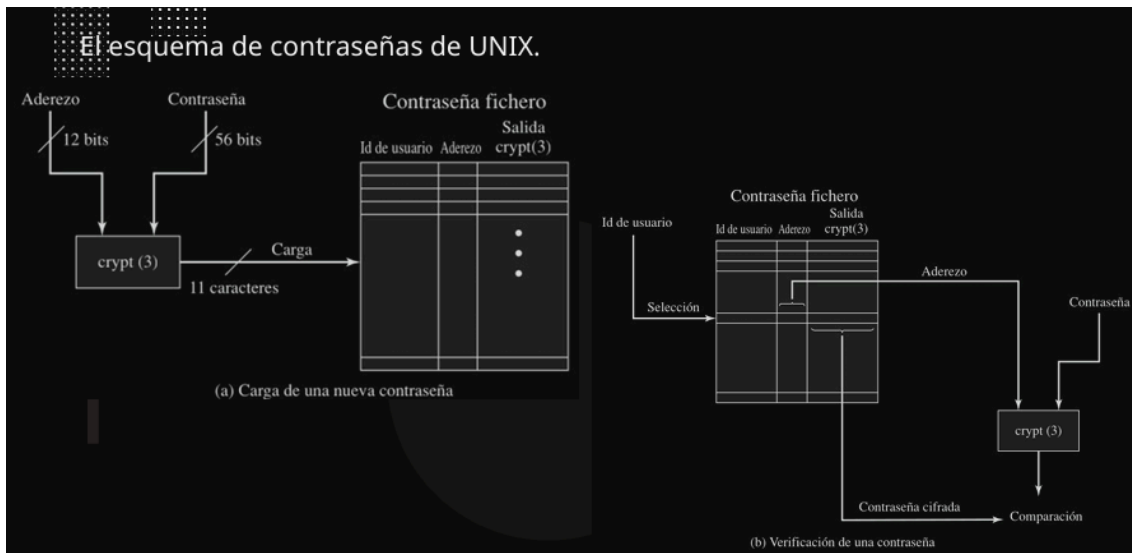
Esquema de Contraseñas de UNIX

En UNIX, las contraseñas nunca se almacenan en claro. Se utiliza la rutina de cifrado **crypt(3)**, basada en DES y modificada con un valor de **12 bits de "aderezo" (salt)**. El proceso se repite 25 veces. La contraseña cifrada final y la copia del aderezo se almacenan en el fichero de contraseñas. Esta rutina multiplica el tiempo necesario para adivinar las contraseñas por 25.

Propósito del Aderezo:

- Evita que contraseñas duplicadas sean visibles
- Incrementa la longitud efectiva de la contraseña
- Evita el uso de implementaciones hardware de DES.

Vulnerabilidad: Un intruso puede obtener una copia del fichero de contraseñas y ejecutar un programa de adivinación (password cracker) en otra máquina para probar millones de combinaciones. Los *crackers* explotan que la mayoría de la gente elige contraseñas **fácilmente adivinables**.



Estrategias para la Selección de Contraseñas

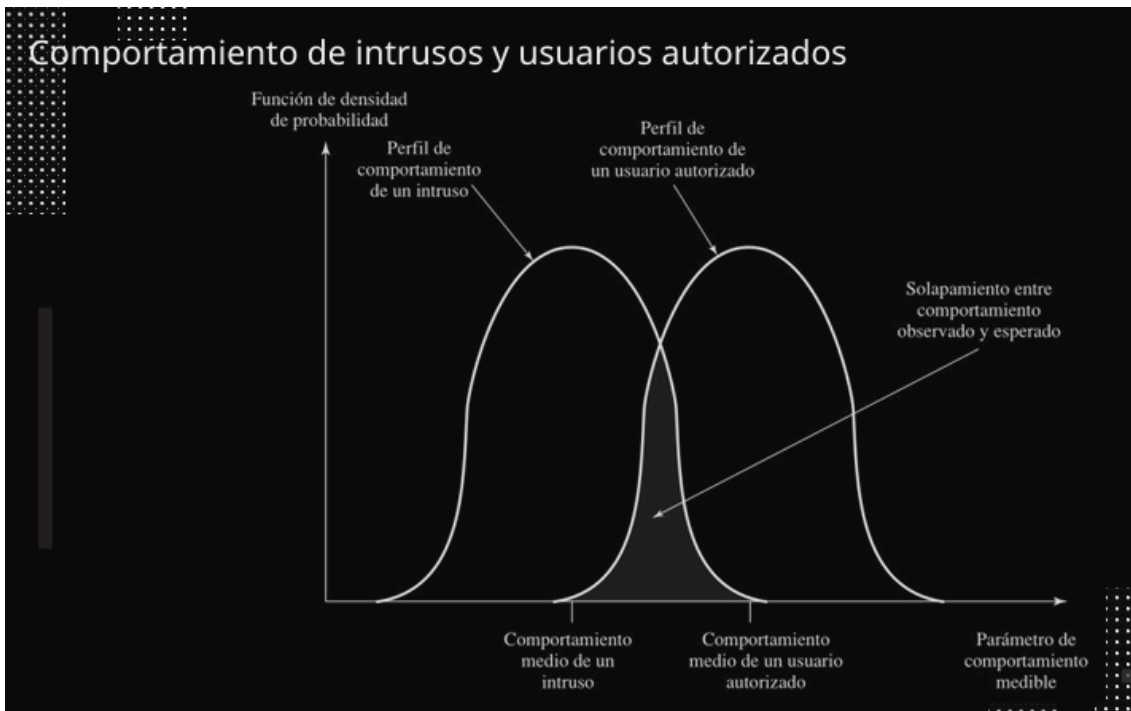
El objetivo principal es eliminar las contraseñas predecibles, facilitando al mismo tiempo que el usuario pueda recordar las suyas.

Métodos de Selección:

1. **Educación del Usuario:** Se informa a los usuarios sobre la relevancia de usar contraseñas robustas, aunque con frecuencia estas recomendaciones son ignoradas.
2. **Contraseñas Generadas por Computadora:** El sistema genera contraseñas (a menudo pronunciables), pero los usuarios pueden encontrar dificultades para memorizarlas, lo que los lleva a escribirlas.
3. **Verificación Reactiva de Contraseñas:** El sistema ejecuta periódicamente su propio programa para identificar contraseñas débiles y desactivarlas. Este método consume muchos recursos y deja las contraseñas expuestas hasta su detección.
4. **Verificación Proactiva de Contraseñas:** El sistema examina la contraseña **en el momento de su selección** y la rechaza si no cumple con los criterios establecidos. Busca un equilibrio entre la aceptación del usuario y la robustez de la contraseña. Puede implementarse a través de **reglas sencillas** (por ejemplo, longitud mínima, tipos de caracteres) o mediante la **creación de un diccionario** de contraseñas inadecuadas.

Detección de Intrusos

La **detección de intrusos** es la segunda línea de defensa cuando la prevención falla. Se basa en la suposición de que el comportamiento de un intruso difiere del de un usuario legítimo de forma cuantificable.



Técnicas de Detección

1. **Detección estadística de anomalías:** Recolección de datos sobre el comportamiento legítimo para luego aplicar tests estadísticos a un nuevo comportamiento.
 - **Detección por umbral:** Define umbrales de frecuencia para eventos.
 - **Basado en el perfil:** Desarrolla un perfil de actividad por cada usuario para detectar cambios en el comportamiento.
 - *Efectiva contra:* **Ataques enmascarados.**
2. **Detección basada en reglas:** Intenta definir un conjunto de reglas para decidir si un comportamiento es sospechoso.
 - **Detección de anomalías:** Reglas para detectar desviación de patrones de usos previos.
 - **Identificación de penetración:** Sistema experto que busca comportamiento sospechoso.
 - *Efectiva contra:* **Trasgresores.**

Registro de Auditoría

El **registro de auditoría** es una herramienta fundamental, utilizada como entrada para los sistemas de detección. Puede ser:

- **Registros de auditoría nativos:** Software incluido en el sistema operativo.
- **Registros de auditoría específicos para detección:** Funcionalidad de recolección de datos implementada para el sistema de detección (implica sobrecarga extra).

Buen ejemplo de registro de auditoría (Un log):

- **Sujeto:** Iniciador de la acción.
- **Acción:** Operación realizada
- **Objeto:** Receptor de acciones

- **Condiciones de excepción:** Que condiciones de excepción se lanzan como respuesta si se dan.
- **Utilización de recursos:** Lista de elementos cuantitativos en los cuales los elementos calculan la cantidad de recursos utilizados.
- **Sello de tiempo:** Cuando fue la acción.

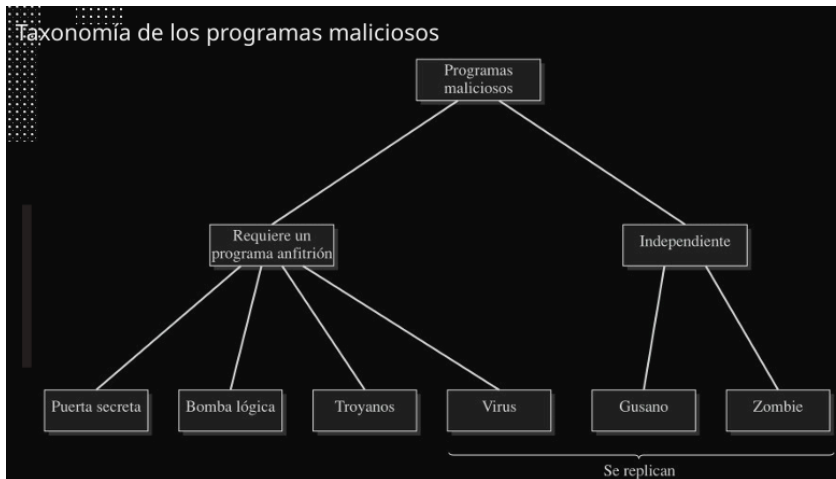
Software Malicioso (*Malware*)

Software malicioso o malware es software diseñado para **causar daño** o **utilizar recursos de un ordenador**.

Taxonomía de Programas Maliciosos

El software malicioso se divide en dos categorías principales:

1. **Requiere un programa anfitrión** (fragmentos de programa que no existen independientemente): **Puerta secreta, Bomba lógica, Troyanos y Virus**.
2. **Independiente** (programas autónomos): **Gusano y Zombie**.
 - Los **Virus, Gusanos y Zombies** se replican.



Tipos Específicos de Malware

- **Puerta secreta (backdoor):** Punto de entrada secreto en un programa que permite el acceso sin utilizar los procedimientos de seguridad estándar. Es difícil de controlar por el sistema operativo. Se dispara ante determinadas secuencias especiales de entrada o cuando se ejecuta por parte de determinado identificador de usuario o una secuencia de eventos poco habitual.
- **Bomba lógica:** Código insertado en un programa legítimo que **explotará bajo ciertas condiciones** (ej., fecha, usuario, criterio). Una vez activada, puede alterar o borrar datos.
- **Troyano (Caballo de Troya):** Programa útil (o aparentemente útil) que contiene un **código oculto** que realiza una función no deseada o dañina al invocarse. Se utiliza para realizar tareas de forma indirecta que un usuario no autorizado no podría realizar directamente.
- **Virus:** Programa que **infecta otros programas modificándolos** e insertando copias de sí mismo que pueden infectar más programas. Durante su vida, pasa por las fases

latente, de **propagación**, de **activación** y de **ejecución**.

- **Tipos de Virus:** **Virus parásito** (infecta archivos ejecutables), **Virus residente en memoria** (infecta la memoria principal), **Virus en el sector de arranque**, **Virus oculto** (diseñado para esconderse del software antivirus), **Virus polimórfico** (muta con cada infección, dificultando la detección por "firma").
- **Gusano (Worm):** Utiliza las **conexiones de red para expandirse** de un sistema a otro. Utiliza vehículos de comunicación como **herramientas de correo electrónico**, **ejecución remota**, o **conexión remota**. Tiene un ciclo de vida interesante:
 - Busca otros sistemas a infectar examinando tablas de servidores o repositorios similares en direcciones remotas.
 - Establece una conexión con el sistema remoto.
 - Se copia a si mismo en el sistema remoto y ejecución.

Naturaleza de los virus

Un virus puede realizar lo mismo que cualquier otro programa. La única diferencia es que se inserta él mismo dentro de otro programa y se ejecuta secretamente cuando el programa anfitrión se va a ejecutar.

Pasa por las siguientes etapas típicamente en su tiempo de vida:

- **Fase latente:** El virus está dormido. El virus se activará finalmente cuando se dé un evento. No todos los virus pasan por esta etapa.
- **Fase de propagación.** El virus inserta copias idénticas de sí mismo en otros programas o en ciertas áreas de disco.
- **Fase de activación.** El virus se activa para realizar la función para la cual fue concebido.
- **Fase de ejecución.** La función en cuestión se realiza. La función puede variar desde un mensaje inofensivo en la pantalla, hasta la dañina destrucción de programas y ficheros de datos.

Categorías de virus

- **Virus parásito.** Forma tradicional y asimismo la más habitual de un virus. Un virus parásito se inserta él mismo dentro de ficheros ejecutables y se replica, cuando el programa infectado se encuentra en ejecución, buscando otros ficheros ejecutables que infectar.
- **Virus residente en memoria.** Infecta la memoria principal como parte del programa del sistema residente. Desde ese punto, el virus infectar todo programa que se ejecuta.
- **Virus en el sector de arranque.** Infecta el sector de arranque maestro (master boot record) y se dispersan cuando el sistema arranca desde el disco que contiene el virus.
- **Virus oculto.** Una forma de virus diseñada expresamente para esconderse del software de detección de los antivirus. Un ejemplo de un virus oculto es aquel que utiliza compresión de forma que el programa infectado tenga exactamente la misma longitud que la versión no infectada.
- **Virus polimórfico.** Un virus que muta con cada infección, haciendo que la detección por medio de la «firma» del virus sea imposible. Crea copias durante la fase de replicación que son funcionalmente equivalentes pero que representan diferentes patrones de bits.

Virus Basados en Macros

En los últimos años, ha habido una proliferación de virus basados en macros, que son especialmente peligrosos porque:

1. Son **independientes de la plataforma** (ej., infectan documentos de Microsoft Word).
2. Infectan **documentos**, no fragmentos de código ejecutables.
3. Tienen una **distribución sencilla** (ej., por correo electrónico).

Se aprovechan de las **macros autoejecutables** (macros que se invocan automáticamente al abrir/cerrar un fichero, o al comenzar una aplicación).

Estrategias de Antivirus

La solución ideal es la **prevención**, aunque imposible de lograr por completo. Las estrategias se centran en:

- **Detección:** Determinar que ha ocurrido una infección y localizar el virus.
- **Identificación:** Identificar el virus específico.
- **Eliminación:** Eliminar los restos del virus y recuperar el programa a su estado original.

Las estrategias antivirus sofisticadas incluyen:

Descifrado Genérico (GD)

Utiliza un **emulador de CPU** (un ordenador virtual basado en software) para interpretar las instrucciones del código a analizar. El virus se ve forzado a descifrarse a sí mismo, exponiendo su código para que el **escáner de firma de virus** lo detecte.

Sistema de Inmunidad Digital (IBM)

Estrategia de **respuesta rápida** para virus que se propagan por Internet. Utiliza la emulación de programas para **capturar, analizar, eliminar** el virus y **distribuir la defensa** a otros sistemas de la organización casi tan rápido como se introduce.

Los sistemas de correo integrado y sistemas de programas móviles han sido focos que incrementan la propagación de virus.

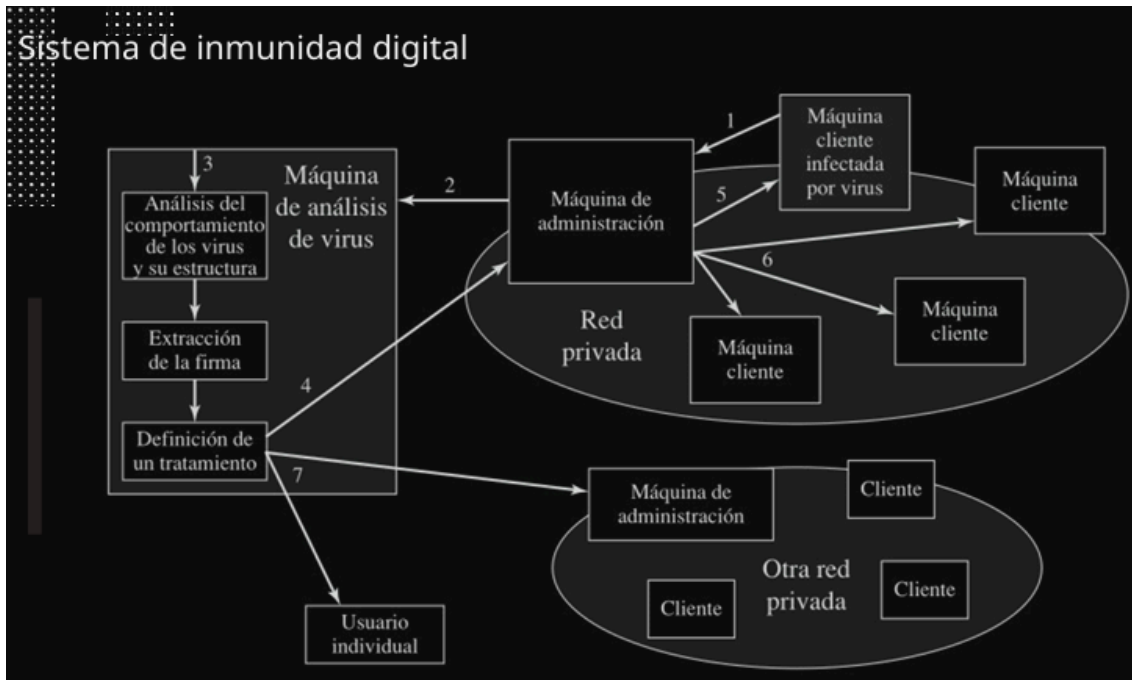
Este sistema extiende el uso de la emulación de programas y proporciona una emulación de propósito general y un sistema de detección de virus. El objetivo de este sistema es proporcionar un tiempo de respuesta rápido de forma que los virus puedan ser expulsados casi tan rápido como se introducen.

Pasos típicos de una operación de un sistema de inmunidad digital:

1. Un programa de monitorización.
2. La máquina de administración cifra la muestra y la envía a una máquina de análisis de virus central.
3. Esta máquina crea un entorno en el cual el programa infectado puede ejecutarse de forma segura para su análisis. Las técnicas utilizadas para ese fin incluyen la emulación, o la creación de un entorno protegido dentro del cual el programa sospechoso puede ejecutarse

y monitorizarse.

4. El tratamiento resultante se envía de nuevo a la máquina de administración.
5. La máquina de administración reenvía el tratamiento al cliente infectado.
6. El tratamiento también se envía a otras máquinas clientes en la organización.
7. Los suscriptores reciben actualizaciones regulares de los antivirus que les protegen de cualquier nuevo virus.



Virus por Correo

Los desarrollos más recientes son los **virus por correo**, que se activan simplemente al **abrir un correo** que contiene el virus (en lugar de necesitar que se abra el fichero adjunto). Estos virus utilizan las funcionalidades del software de correo para **replicarse rápidamente** (en cuestión de horas) a todas las direcciones conocidas por la máquina infectada.

Unidad 7: Clase 19

Sistemas Confiables

Un **sistema confiable** es aquel que ha sido verificado matemáticamente para aplicar reglas de seguridad, mediación completa y aislamiento. Estos sistemas son un requisito para la protección de datos por medio de **niveles de seguridad**. Cuando se definen múltiples niveles o categorías de datos, a los requisitos se les denomina **seguridad multinivel**.

Reglas de Seguridad Multinivel

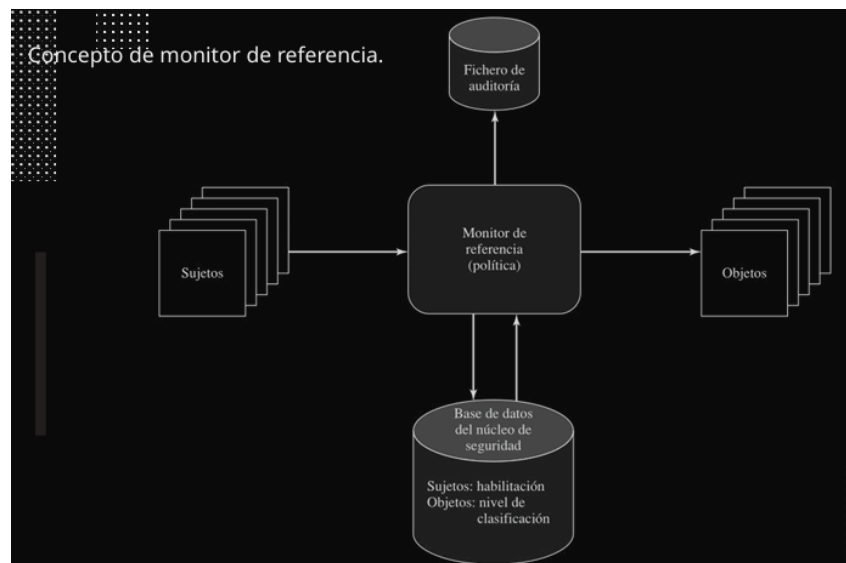
La seguridad multinivel se articula en dos partes:

- **No leer hacia arriba** (*no read up*): Un sujeto solo puede leer un objeto de un nivel de seguridad **igual o inferior**. Esto se llama la **propiedad de seguridad simple**.
- **No escribir hacia abajo** (*no write down*): Un sujeto solo puede escribir en un objeto de un nivel de seguridad **igual o superior**. Esto se llama la **propiedad-estrella** (propiedad estrella).

Monitor de Referencias

La estrategia para implementar la seguridad multinivel se basa en el concepto de **monitor de referencias**.

- **Definición:** Un elemento de control (en hardware con el sistema operativo) que **regula el acceso de sujetos a objetos** basándose en sus parámetros de seguridad.
- **Componentes:** Tiene acceso a un fichero llamado **base de datos del núcleo de seguridad**, que lista los **privilegios de acceso (habilitación)** de cada sujeto y los **atributos de protección (nivel de clasificación)** de cada objeto. También genera un **fichero de auditoría**.



- **Propiedades del Monitor de Referencia:**
 - **Mediación completa:** Las reglas de seguridad se aplican a **cada acceso**.
 - **Aislamiento:** El monitor y la base de datos están **protegidos de cualquier modificación no autorizada**.
 - **Verificabilidad:** La corrección del monitor de referencia debe estar **probada matemáticamente**.

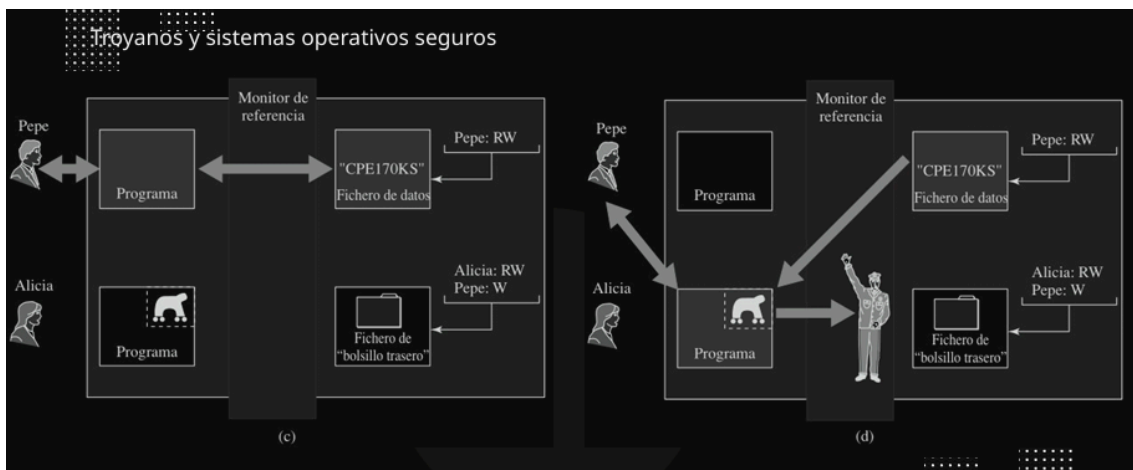
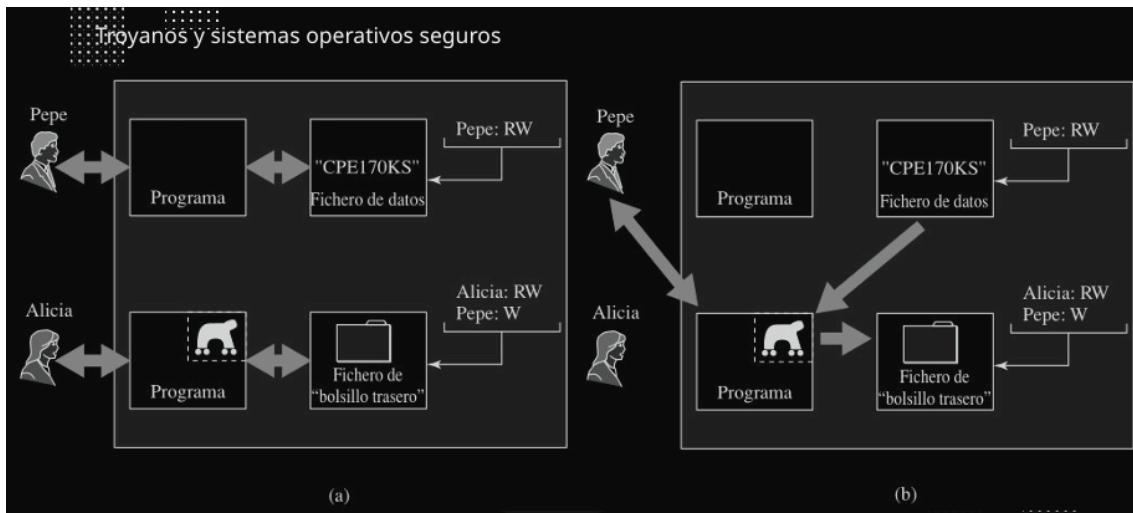
Defensa Contra Troyanos (Caso de Estudio)

Los sistemas confiables son una vía para defenderse contra los ataques de **troyanos**. El ataque tradicional de troyanos se basa en que el usuario hostil (Alicia) induce a un usuario legítimo (Pepe) a ejecutar un programa con el troyano. Este programa, al ejecutarse con los privilegios de Pepe, lee un fichero sensible de Pepe y lo copia a un fichero privado de Alicia ("bolsillo trasero") al que Pepe tiene permiso de escritura, evitando la **lista de control de acceso**.

Con un sistema seguro multinivel:

1. Los ficheros y procesos de Pepe se asignan a un nivel de seguridad **sensible** (más alto).
2. Los ficheros y procesos de Alicia se asignan a un nivel **público** (más bajo).
3. Cuando el troyano (ejecutándose con el nivel sensible de Pepe) intenta **escribir** la cadena sensible en el fichero de Alicia (nivel público), se **viola la propiedad- $\star$ (no escribir hacia abajo)**.

4. El **monitor de referencia aborta** la escritura, incluso aunque la lista de control de acceso lo permita.



Seguridad en Windows

Windows proporciona servicios de control de acceso **uniformes** aplicables a procesos, *threads*, ficheros y otros objetos. El control se regula por dos entidades:

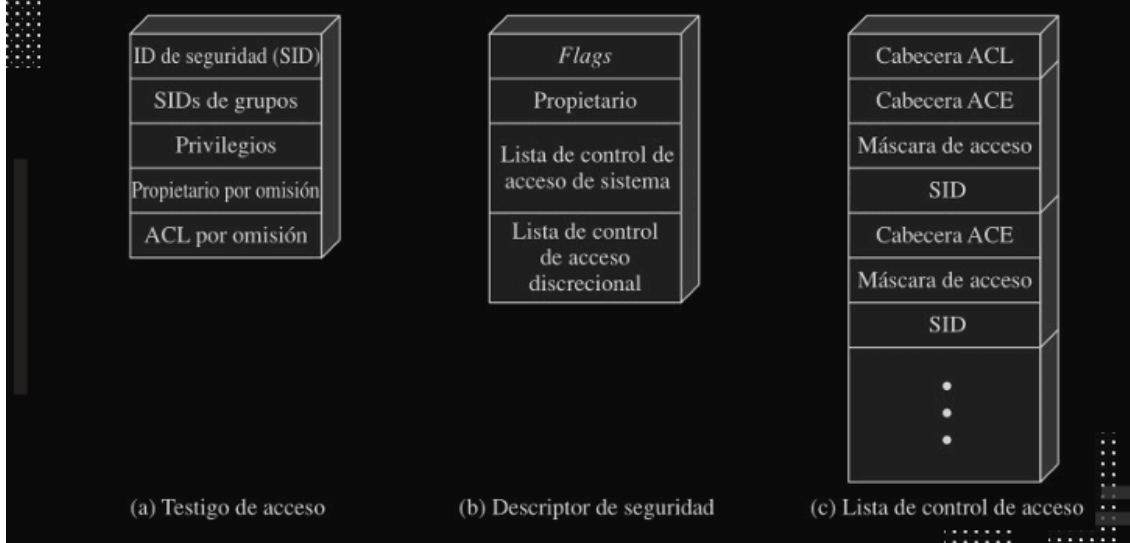
- **Testigo de acceso** (*access token*): Asociado a cada **proceso**.
- **Descriptor de seguridad** (*security descriptor*): Asociado a cada **objeto** al que acceden varios procesos.

Testigo de Acceso

Se crea cuando un usuario se conecta al sistema y se le asocia al proceso inicial. Sirve para:

1. Agrupar la información de seguridad para **acelerar la validación de acceso**.
2. Permitir a cada proceso **modificar sus características de seguridad de forma limitada**.

Estructuras de seguridad de Windows



Estructura: Incluye el **Identificador de seguridad (SID)** del usuario (identificador único) y una lista de **SIDs de grupos** a los que pertenece. También contiene una lista de **Privilegios** (servicios de sistema sujetos a seguridad), **Propietario por omisión** y **ACL por omisión**.

Descriptor de Seguridad

Contiene la **Lista de control de acceso (ACL)** que especifica los derechos de acceso. Cuando un proceso accede a un objeto, el SID del proceso se compara con la ACL del objeto.

Estructura: Contiene **Flags**, **Propietario**, **Lista de control de acceso de sistema (SACL)** y **Lista de control de acceso discrecional (DACL)**.

- **SACL (Lista de control de acceso de sistema):** Indica qué tipo de operaciones generan mensajes de auditoría.
- **DACL (Lista de control de acceso discrecional):** Indica **qué usuarios y grupos pueden acceder** al objeto y para **qué operaciones**. Consiste en una lista de **entradas de control de acceso (ACE)**.

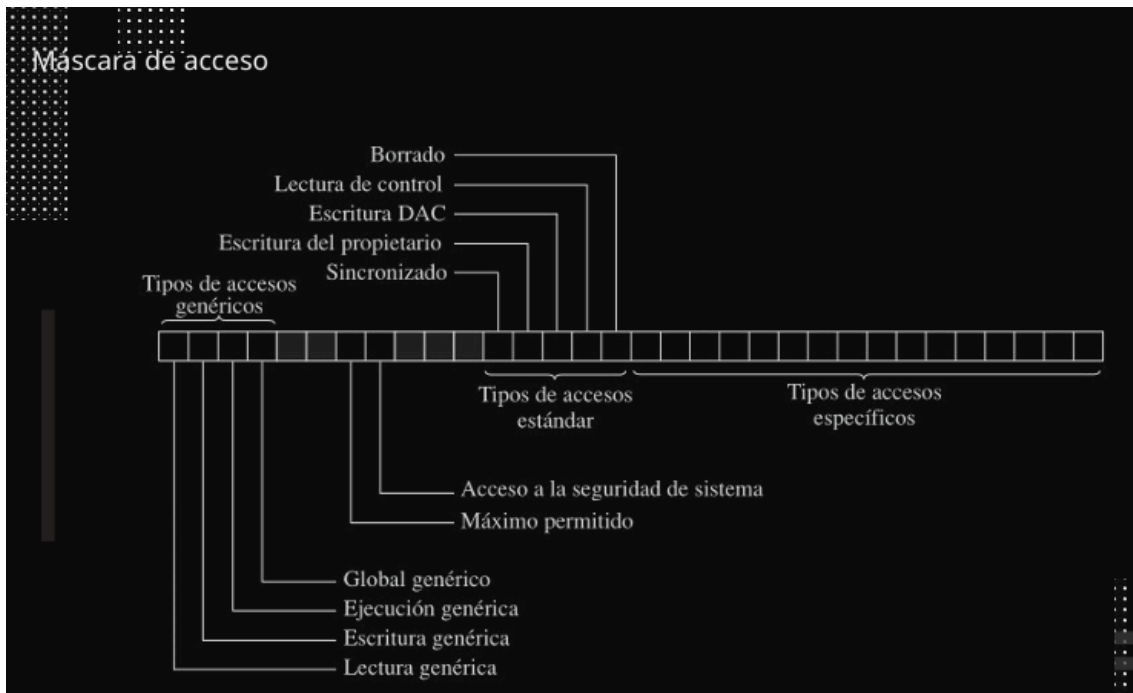
Listas de Control de Acceso (ACL y ACE)

Las listas consisten en una cabecera y entradas de control de acceso (**ACE**). Cada ACE especifica un **SID** y una **máscara de acceso**.

Máscara de Acceso: Se divide en 16 bits menos significativos para **tipos de accesos específicos** (del objeto) y 16 bits más significativos para **tipos de accesos estándar y genéricos**.

- **Tipos de accesos estándar:** Incluyen **Sincronizado**, **Escritura del propietario**, **Escritura DAC (Permite a la app modificar la DACL y protección del objeto)**, **Lectura de control** (Lectura de los campos del propietario, y DACL del descriptor de seguridad del objeto) y **Borrado**.
- **Tipos de accesos genéricos:** Proporcionan una forma conveniente de agrupar tipos de

accesos específicos y estándar (ej., **Lectura genérica**, **Escritura genérica**, **Ejecución genérica**, **Global genérico**).



Los dos bits restantes en la máscara de acceso tienen un significado especial. El bit de Acceso a la Seguridad del Sistema permite modificar los controles de auditoría y alarmas para este objeto.

Opciones para solicitar acceso:

- Intentar abrir el objeto para todos los accesos posibles.
- Sólo abrir el objeto cuando el acceso específico se va a solicitar, y abrir un nuevo manejador de objeto con cada uno de los diferentes tipos de solicitudes.
- Intentar abrir el objeto para el mayor número de posibles accesos que el objeto permita para este SID.

Una característica importante de la seguridad en Windows es que las aplicaciones pueden utilizar el marco de seguridad que proporciona el sistema operativo para los objetos definidos por el usuario. Por ejemplo, un servidor de bases de datos puede crear sus propios descriptores de seguridad y asociarlos a diferentes partes de la base de datos.

Criptografía Simétrica (Convencional)

La criptografía es esencial para la seguridad en redes y computadoras, utilizando dos enfoques principales: la criptografía simétrica y la criptografía de clave pública.

Fue el único método de cifrado hasta finales de los años 70 y sigue siendo el más utilizado. Se caracteriza por emplear una única clave.

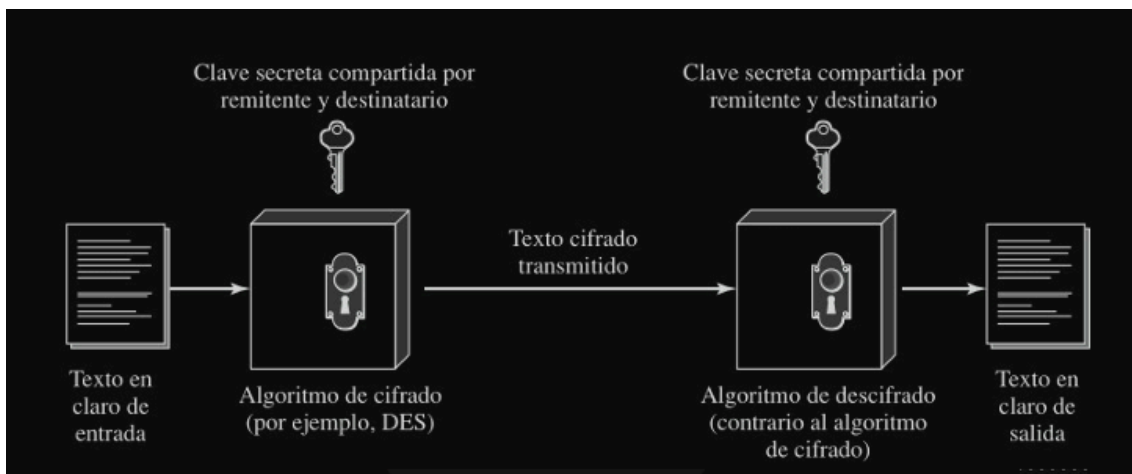
Componentes Clave:

- **Texto en claro:** El mensaje original.
- **Algoritmo de cifrado:** El proceso para transformar el texto en claro.
- **Clave secreta:** La entrada al algoritmo, compartida entre emisor y receptor.
- **Texto cifrado:** La salida codificada.
- **Algoritmo de descifrado:** El proceso inverso para recuperar el texto original.

Funcionamiento: El texto en claro se introduce en el algoritmo de cifrado junto con una clave secreta compartida, generando un texto cifrado que se transmite. El receptor utiliza el algoritmo de descifrado y la misma clave secreta para obtener el texto en claro de salida.

Requisitos para un Uso Seguro:

- Un algoritmo criptográfico robusto que impida descifrar el texto o deducir la clave, incluso con pares de texto en claro-texto cifrado.
- Emisor y receptor deben obtener y proteger la clave secreta de forma segura.



Tipos de Ataques:

- **Criptanálisis:** Explotación de las características del algoritmo para descifrar el texto o deducir la clave.
- **Ataque por fuerza bruta:** Prueba de cada posible clave sobre un fragmento de texto cifrado hasta obtener un texto en claro inteligible.

Los algoritmos de cifrado simétricos usados más comúnmente son los cifradores de bloque. Un cifrador de bloque procesa el texto en claro introducido en bloques de tamaño fijo y produce un bloque de texto cifrado de igual tamaño por cada bloque de texto en claro.

Algoritmos Destacados: Los cifradores de bloque más comunes son DES y AES.

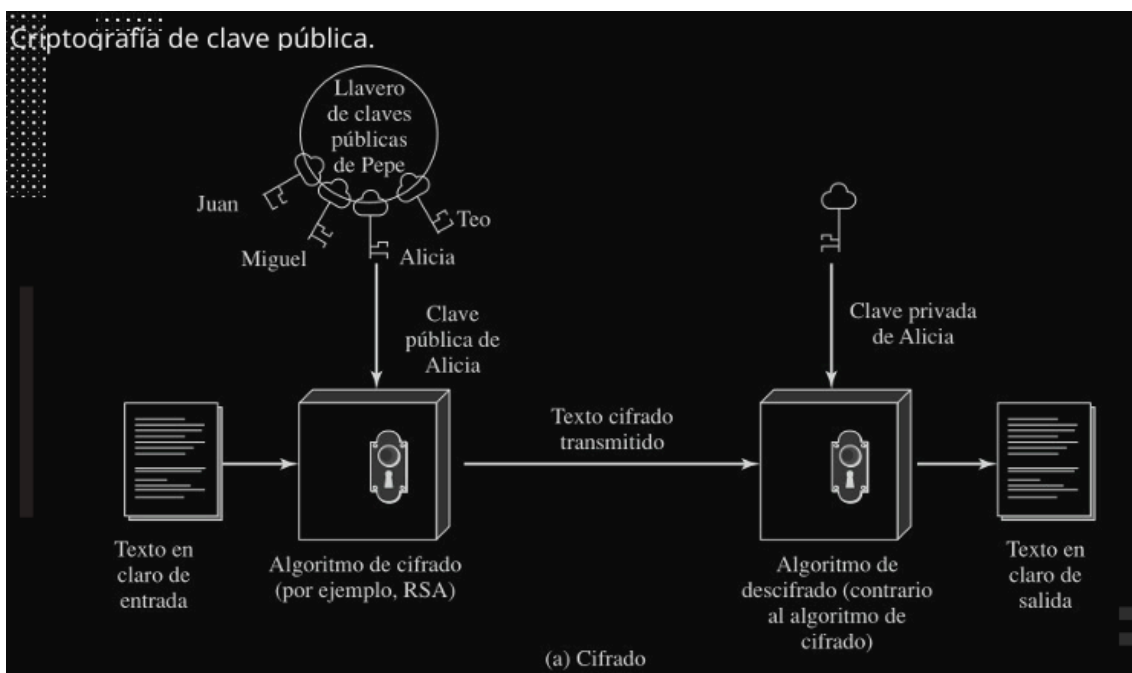
- **DES (Data Encryption Standard):** Utiliza claves de 56 bits y ya no se considera computacionalmente seguro debido a la velocidad de los ordenadores. Su vida útil se extendió con Triple DES (3DES).
- **AES (Advanced Encryption Standard):** Sustituto del DES, es un cifrador simétrico de bloque con bloques de 128 bits y soporta claves de 128, 192 y 256 bits.

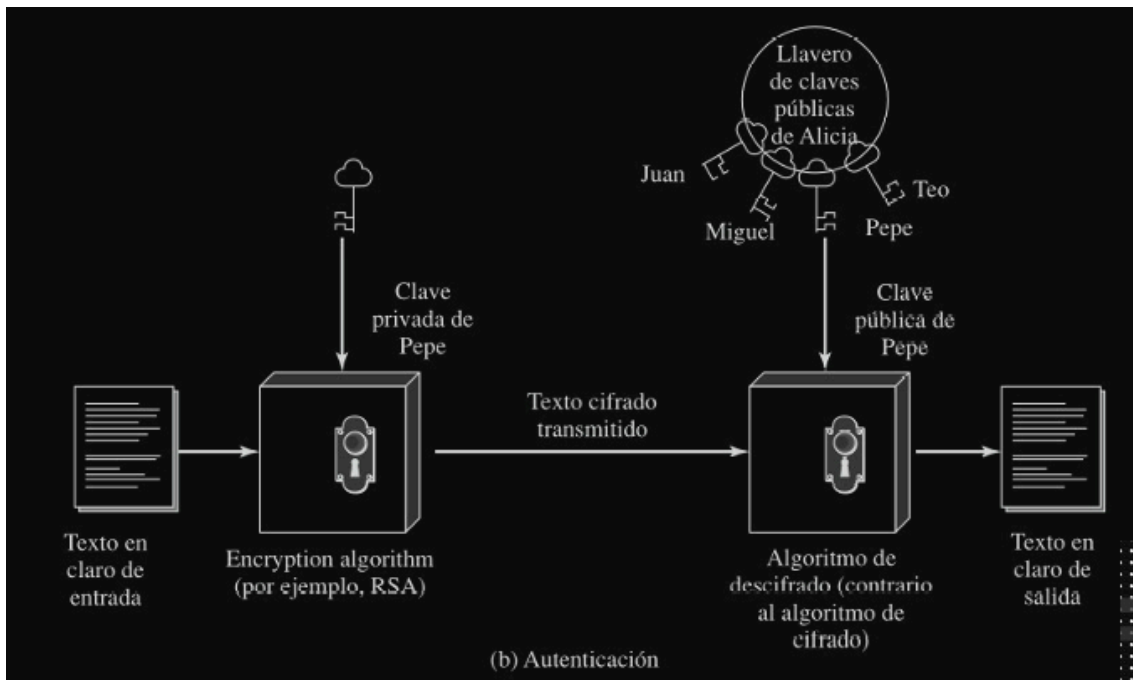
Criptografía de Clave Pública (Asimétrica)

Propuesta en 1976, es un método asimétrico que requiere el uso de dos claves distintas: una pública y una privada. No hay en principio nada en la criptografía simétrica o la de clave pública que haga a una superior a la otra desde el punto de vista de resistir al criptoanálisis.

- **Principio Fundamental:** La clave pública se comparte libremente para que otros la utilicen, mientras que la clave privada es exclusiva de su propietario. La seguridad de estos algoritmos se basa en funciones matemáticas complejas.
- **Componentes Clave:**
 1. **Texto en claro:** El mensaje original.
 2. **Algoritmo de cifrado:** El proceso de transformación.
 3. **Claves pública y privada:** Un par de claves donde si una cifra, la otra descifra.
 4. **Texto cifrado:** La salida codificada.
 5. **Algoritmo de descifrado:** El proceso inverso.

El proceso funciona (produce como salida el texto en claro correcto) sin importar el orden en que se utiliza la pareja de claves. Como el nombre sugiere, la clave pública del par se hace pública para que la usen otros, mientras que la clave privada es solamente para su propietario.





- **Propiedades de las claves:**
 1. **Es computacionalmente irrealizable determinar la clave de descifrado conociendo solamente el algoritmo criptográfico y la clave de cifrado.**
 2. **Cualquiera de las dos claves relacionadas puede usarse para el cifrado, y usarse la otra para el descifrado.**
- **Funciones de las Claves:** El proceso funciona independientemente del orden en que se utilicen las claves del par.
 1. **Confidencialidad:** Para enviar un mensaje confidencial, Pepe cifra con la clave pública de Alicia, y Alicia lo descifra con su clave privada.
 2. **Autenticación (Firma Digital):** Para probar la autoría, Pepe cifra con su clave privada, y Alicia descifra con la clave pública de Pepe, confirmando que el mensaje proviene de él.

Pasos para generar una criptografía (La explicación del gráfico):

1. Cada usuario genera un par de claves para ser usadas en el cifrado y descifrado de mensajes.
2. Cada usuario pone una de las dos claves en un registro público u otro fichero accesible.
3. Si Pepe desea enviar un mensaje privado a Alicia, Pepe cifra el mensaje usando la clave pública de Alicia
4. Cuando Alicia recibe un mensaje, lo descifra usando su clave privada.

Algoritmo Importante: El **Algoritmo Rivest-Shamir-Adleman (RSA)** es el enfoque más extendido y aceptado de criptografía de clave pública. Su fortaleza reside en la dificultad de factorizar números grandes en sus factores primos.

Unidad 8: Clase 20 Sistemas Distribuidos

Computación Cliente/Servidor

La **computación cliente/servidor** es una tendencia en el campo de la computación con su propia jerga. Un entorno cliente/servidor está formado por clientes y servidores.

- **Clientes:** Son típicamente PCs simples o estaciones de trabajo que proveen una **interfaz de fácil manejo al usuario final**. Suelen usar una **interfaz gráfica** (GUI) con ventanas y ratón (ejemplos son Microsoft Windows o Macintosh). Las aplicaciones en el cliente están diseñadas para ser fáciles de usar (como una hoja de cálculo).
- **Servidores:** Son computadores (estaciones de trabajo de gran potencia, minicomputadores o mainframes) que **almacenan la información para los clientes de la red**. Proveen un conjunto de **servicios compartidos** a los clientes. El más común es el **servidor de base de datos**, que controla una base de datos relacional y permite a los clientes compartir el acceso y usar sistemas de alto rendimiento para gestionarla.
- **Red:** Es el tercer elemento esencial. La computación cliente/servidor es una **computación distribuida** donde usuarios, aplicaciones y recursos se distribuyen y se unen a través de una **LAN, WAN o Internet**.

Terminología Clave

El documento define varios términos clave del entorno cliente/servidor:

- **Interfaz de programación de aplicaciones (API):** Un conjunto de funciones y programas que permiten a los clientes y servidores intercomunicarse.
- **Cliente:** Un elemento de la red que **solicita información**, normalmente un PC o estación de trabajo. Puede interrogar a una base de datos o solicitar información de un servidor.
- **Middleware:** Un conjunto de controladores, API, y software adicional que **mejoran la conectividad** entre una aplicación cliente y un servidor.
- **Base de datos relacional:** Una base de datos en la que el acceso a la información está restringido por la selección de filas que satisfacen todos los criterios de búsqueda.
- **Servidor:** Un computador (estación de trabajo de gran potencia, minicomputador o mainframe) que **almacena la información para los clientes de la red**.
- **Lenguaje Estructurado de Consultas (SQL):** Lenguaje desarrollado por IBM y estandarizado por ANSI que permite **acceder, crear, actualizar e interrogar bases de datos relacionales**.

Características Distintivas

La filosofía cliente/servidor se diferencia de otros tipos de procesamiento distribuido por:

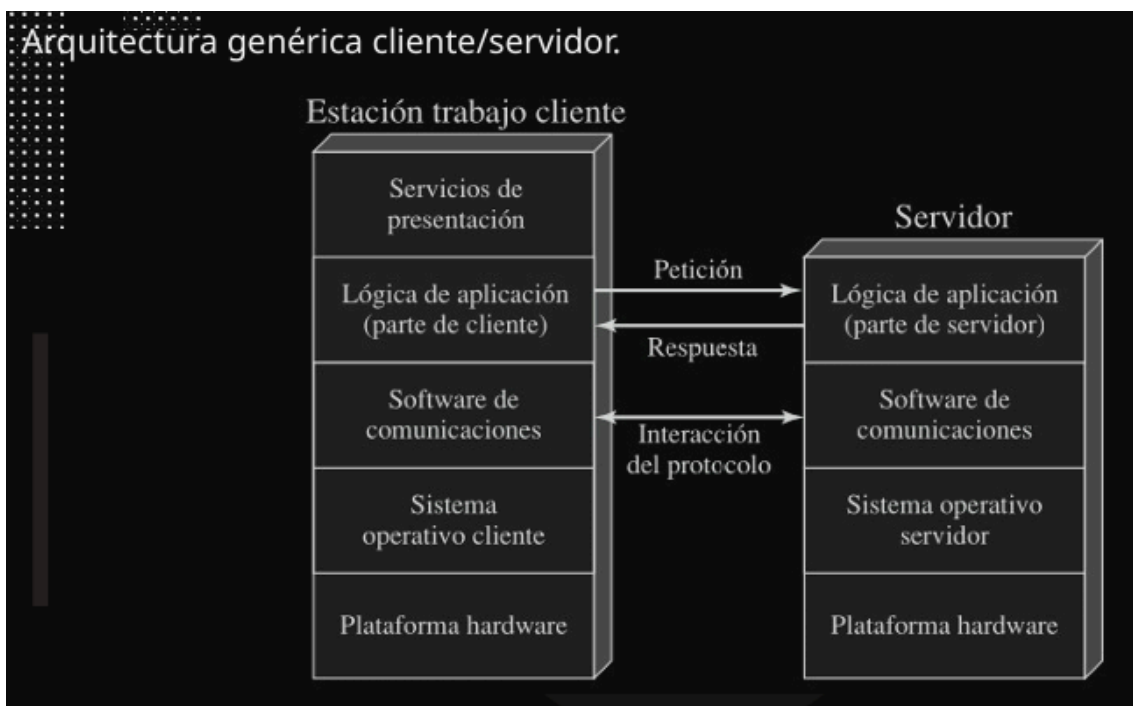
- La necesidad de que los usuarios tengan **aplicaciones de fácil manejo** para tener un gran control sobre su computadora.
- Un esfuerzo por **centralizar las bases de datos corporativas** y muchas funciones de utilidad y gestión de redes, lo que permite el control del capital y la interoperabilidad, mientras que descarga a los departamentos del mantenimiento de sistemas sofisticados.
- El compromiso de mantener un **sistema abierto y modular**, dando a los usuarios más libertad para seleccionar y combinar equipos de diferentes vendedores.
- La **red es fundamental**, por lo que su gestión y seguridad tienen una prioridad muy alta.

Aplicaciones Cliente/Servidor

La característica fundamental es la **distribución de las tareas de la aplicación entre el cliente y el servidor**. El software básico en ambas partes es el **sistema operativo** sobre la plataforma hardware, que puede ser diferente. El software de **comunicaciones** (ej. **TCP/IP**) permite la interacción.

El diseño de la **interfaz de usuario (GUI)** en la máquina cliente es decisivo para el éxito, enfatizando una GUI fácil de usar, aprender, potente y flexible. El módulo de **servicios de presentación** en el cliente es el responsable de esta interfaz.

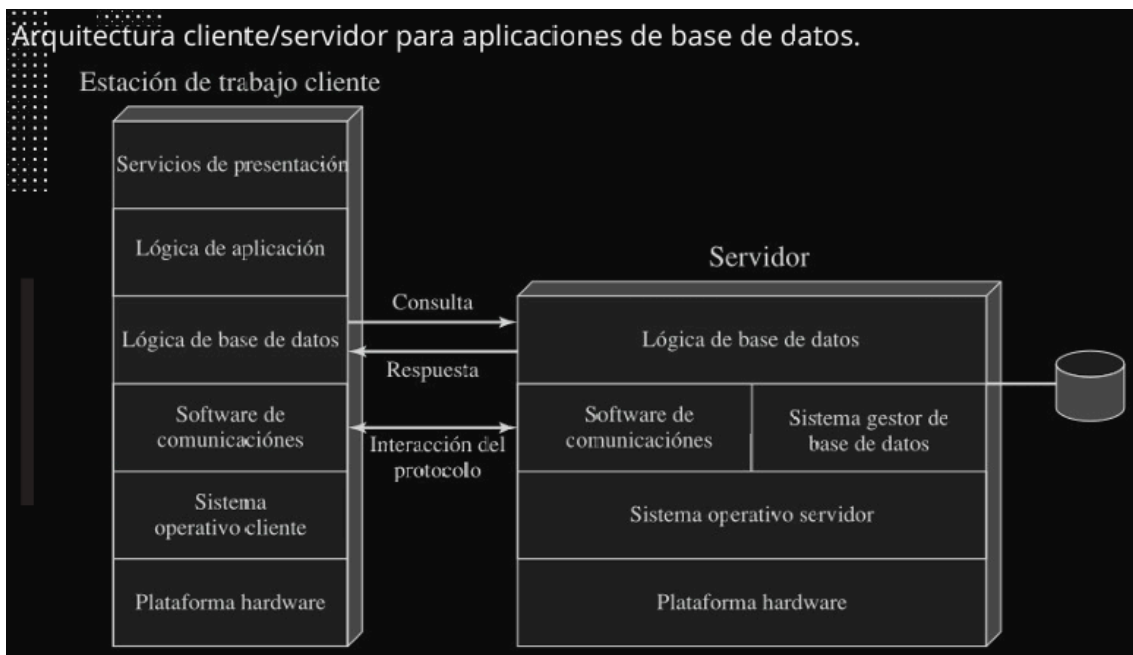
Arquitectura Genérica Cliente/Servidor



La arquitectura genérica incluye en la **Estación de trabajo cliente**: Servicios de presentación, Lógica de aplicación (parte de cliente), Software de comunicaciones y Sistema operativo cliente. En el **Servidor** incluye: Lógica de aplicación (parte de servidor), Software de comunicaciones y Sistema operativo servidor. La comunicación se da con **Petición, Respuesta e Interacción del protocolo**.

Aplicaciones de Base de Datos

En este entorno, el servidor es un **servidor de base de datos**. La interacción es por **transacciones**: el cliente hace una **petición (consulta)** y recibe una **respuesta**. El servidor mantiene la base de datos con un **sistema gestor de base de datos**. El "pegamento" que une al cliente y al servidor es el software que permite al cliente hacer peticiones, siendo **SQL** un ejemplo popular.



El **uso deseable cliente/servidor** se da cuando el **servidor** realiza el **trabajo pesado de búsqueda y ordenamiento** de una base de datos grande, ya que la potencia de E/S, CPU y disco es costosa para un PC cliente, y mover una gran cantidad de registros a través de la red sería demasiado costoso. El servidor debe tener la lógica para realizar búsquedas para el cliente.

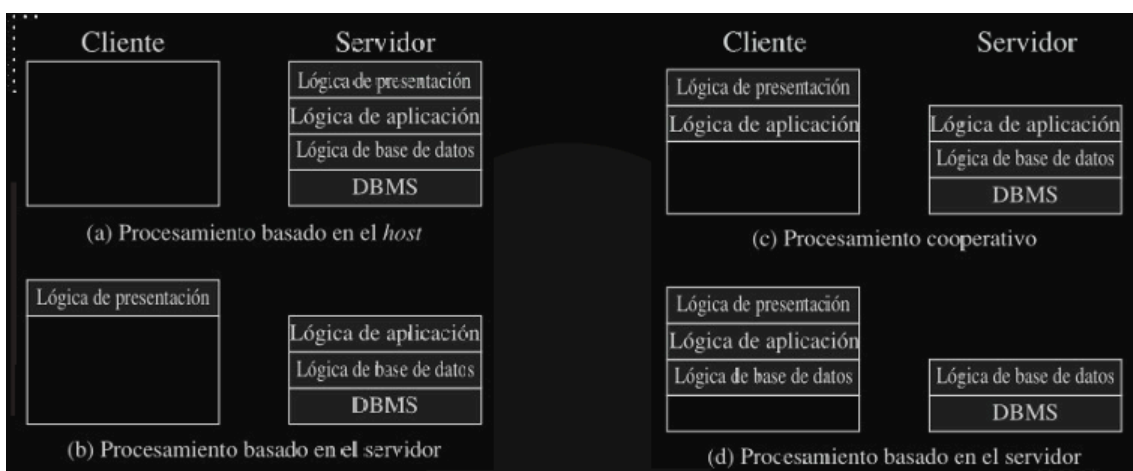


Un **uso incorrecto cliente/servidor (b)** ocurre si una consulta provoca la transmisión de una gran cantidad de registros (ej. 300.000 de 1 millón) por la red, lo que podría suceder si el cliente busca valores totales o medios de muchos registros. La solución es **trasladar parte de la lógica de aplicación al servidor** (análisis de datos, además de recepción y búsqueda).



Clases de Aplicaciones Cliente/Servidor

Existen diferentes implementaciones que dividen el trabajo entre cliente y servidor:



- Procesamiento basado en el host:** No es verdadera computación cliente/servidor. **Virtualmente todo el procesamiento se realiza en el host**

central (mainframe tradicional). La estación de usuario actúa como un **emulador de terminal (interfaz tonto)**.

- *Distribución de lógica:* Lógica de presentación, Lógica de aplicación, y Lógica de base de datos (DBMS) están en el **Servidor**.
- b. **Procesamiento basado en el servidor (Cliente ligero - *thin client***):**** El **cliente es el principal responsable de la interfaz gráfica de usuario**, mientras que **prácticamente todo el procesamiento se realiza en el servidor**. Es típica en sistemas a nivel de departamento al inicio. El cliente ligero imita el enfoque tradicional centralizado del host y es una ruta de migración de mainframes a entornos distribuidos.
 - *Distribución de lógica:* Lógica de presentación en el **Cliente**. Lógica de aplicación y Lógica de base de datos (DBMS) en el **Servidor**.
- c. **Procesamiento cooperativo:** El procesamiento de la aplicación se realiza de **forma óptima, beneficiándose de las máquinas cliente y servidora y de la distribución de los datos**. Es la más compleja de configurar y mantener, pero puede proporcionar mayor productividad y eficiencia de red.
 - *Distribución de lógica:* Lógica de presentación en el **Cliente**. Lógica de aplicación y Lógica de base de datos (DBMS) dividida entre **Cliente y Servidor**.
- d. **Procesamiento basado en el cliente (Cliente pesado - *fat client***):**** **Prácticamente todo el procesamiento se realiza en el cliente**, excepto rutinas de validación de datos y otras funciones de la lógica de la base de datos mejor realizadas en el servidor. Es el enfoque más común actualmente. Permite aplicaciones adaptadas a necesidades locales. La ventaja es que **descarga a los servidores**, haciéndolos más eficientes y menos propensos a ser un cuello de botella.
 - *Desventajas del cliente pesado:* Añadir funcionalidad sobrecarga PCs, requiere LANs de alta capacidad si el modelo escala, y es difícil mantener/actualizar aplicaciones distribuidas en muchos ordenadores.
 - *Distribución de lógica:* Lógica de presentación y Lógica de aplicación en el **Cliente**. Lógica de base de datos (DBMS) dividida entre **Cliente y Servidor**.

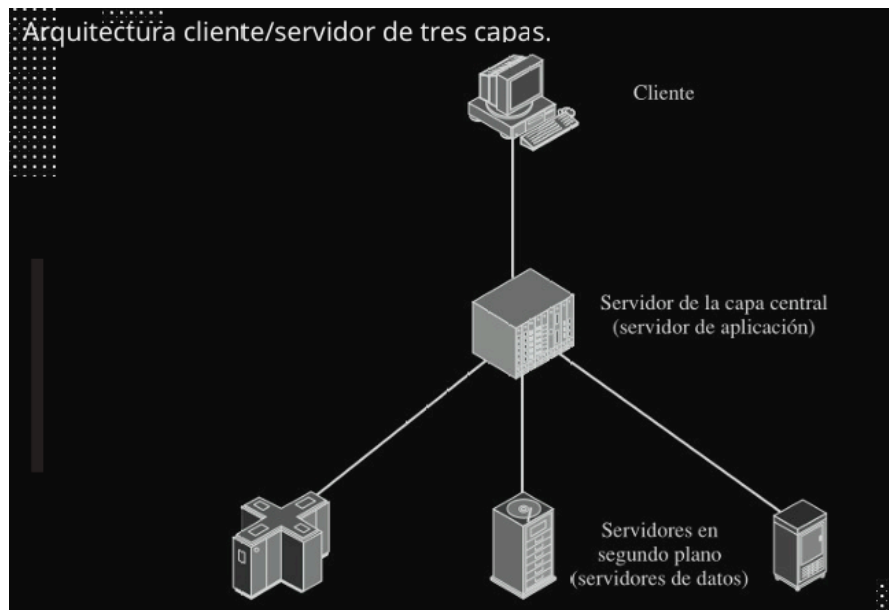
Arquitectura Cliente/Servidor de Tres Capas

Además de la tradicional de dos capas (cliente y servidor), es habitual la arquitectura de **tres capas**:

1. **Máquina de usuario (Cliente):** Generalmente un **cliente ligero**.
2. **Servidor en la capa central (Servidor de aplicación):** Actúa como **pasarela** entre los clientes ligeros y varios servidores de bases de datos en segundo plano. Puede **convertir protocolos**, cambiar tipos de consulta de base de datos, mezclar/integrar resultados de diferentes fuentes de datos y mediar entre

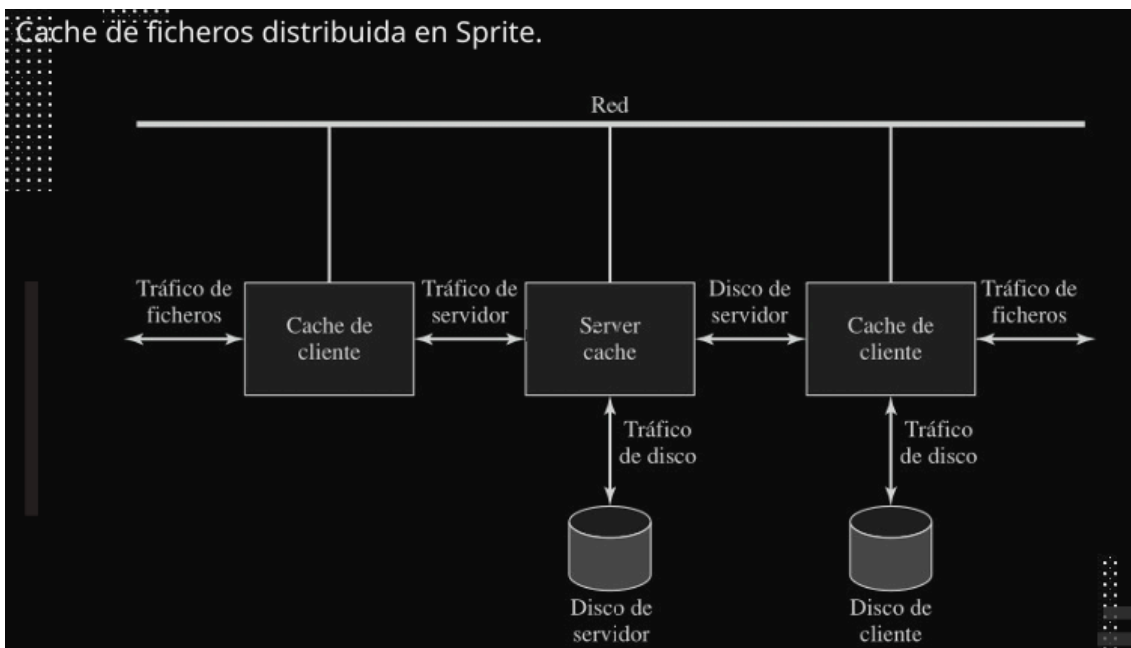
aplicaciones de escritorio y de servidor. El sistema de la capa intermedia actúa de **cliente y de servidor**.

3. **Servidor en segundo plano (Servidor de datos):** Almacena los datos.



Consistencia de la Cache de Ficheros

En un servidor de ficheros, el rendimiento de la E/S de ficheros puede degradarse por los retrasos de red. Para mitigarlo, se usa **cache de ficheros** en sistemas individuales para almacenar registros accedidos recientemente, aprovechando el principio de localidad.



- **Cache de cliente:** Busca reducir el **tráfico de comunicación**. Cuando un proceso accede a un fichero, la petición va primero a la cache local del cliente. Si no

está, va al disco local o al servidor de ficheros.

- **Cache de servidor:** Busca reducir la **E/S de disco**. En el servidor, se pregunta primero a la cache del servidor, y si no está, se accede al disco del servidor.

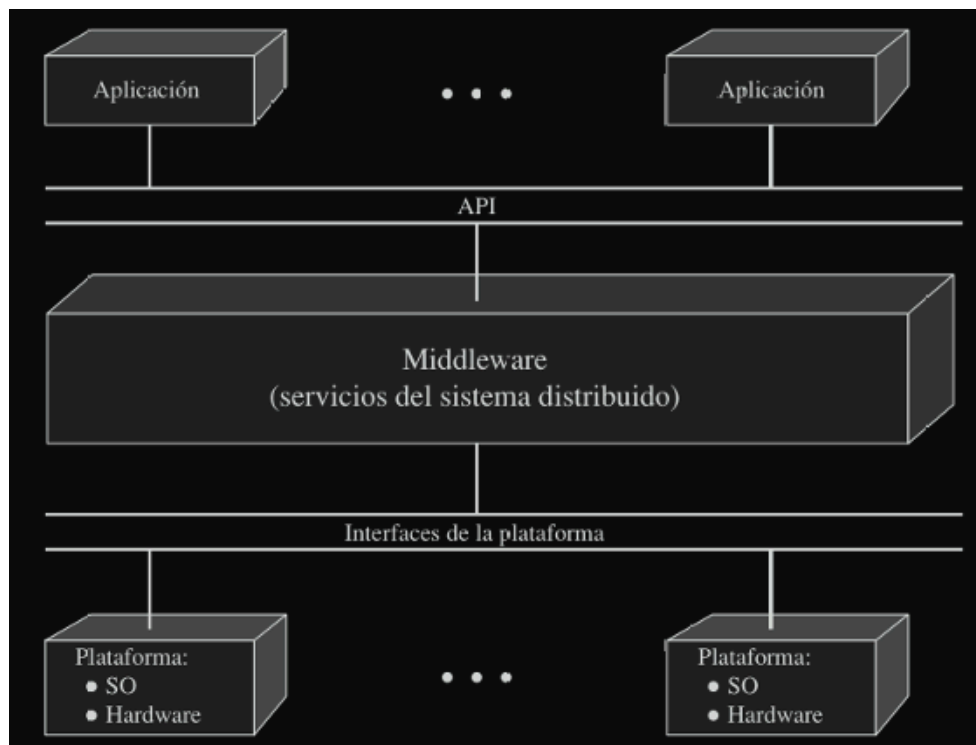
El **problema de la consistencia de cache** ocurre cuando las caches se vuelven inconsistentes porque los datos remotos son cambiados y las copias locales en la cache no se descartan. Esto pasa si un cliente modifica un fichero que está en la cache de otro cliente. La política de escritura inmediata o diferida en el servidor puede causar que otros clientes o incluso el servidor tengan datos obsoletos.

El mecanismo más sencillo para la consistencia es el **bloqueo de ficheros** para evitar accesos simultáneos a un fichero por más de un cliente, lo que garantiza consistencia a costa de rendimiento.

Arquitectura **Middleware**

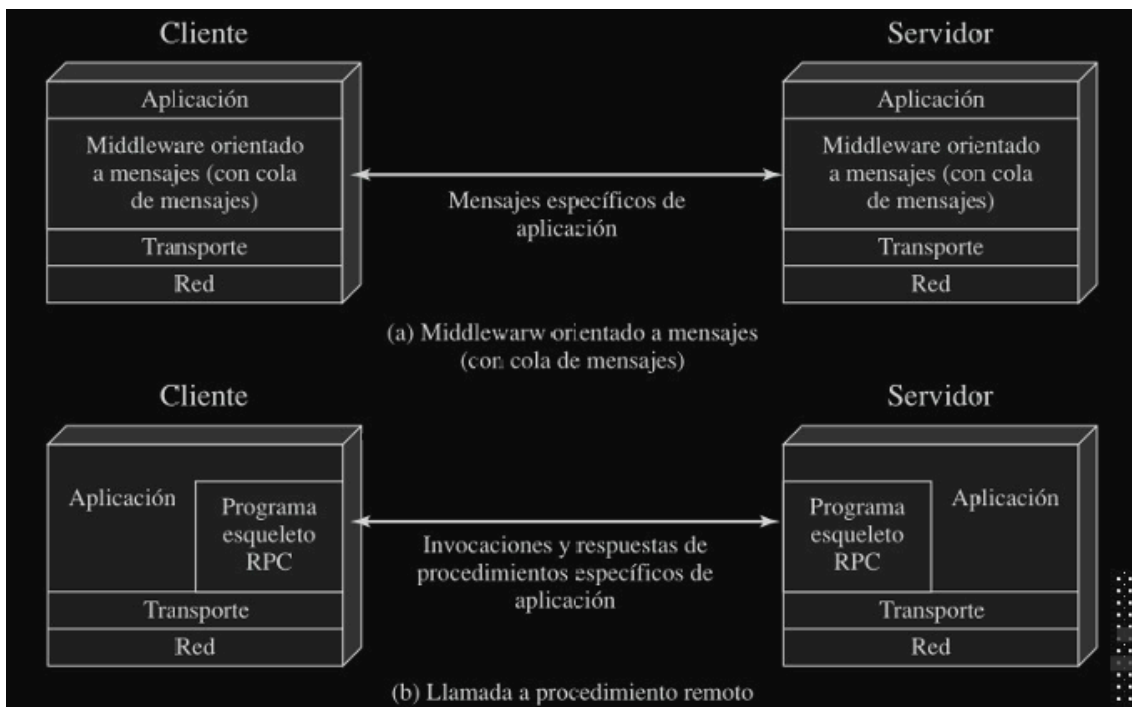
La falta de estándares en la computación distribuida dificulta la implementación de configuraciones cliente/servidor integradas y multivendedor. El **middleware** es la forma más común de solucionar esto, mediante el uso de **interfaces de programación (API) y protocolos estándares** entre la aplicación, el software de comunicaciones y el sistema operativo.

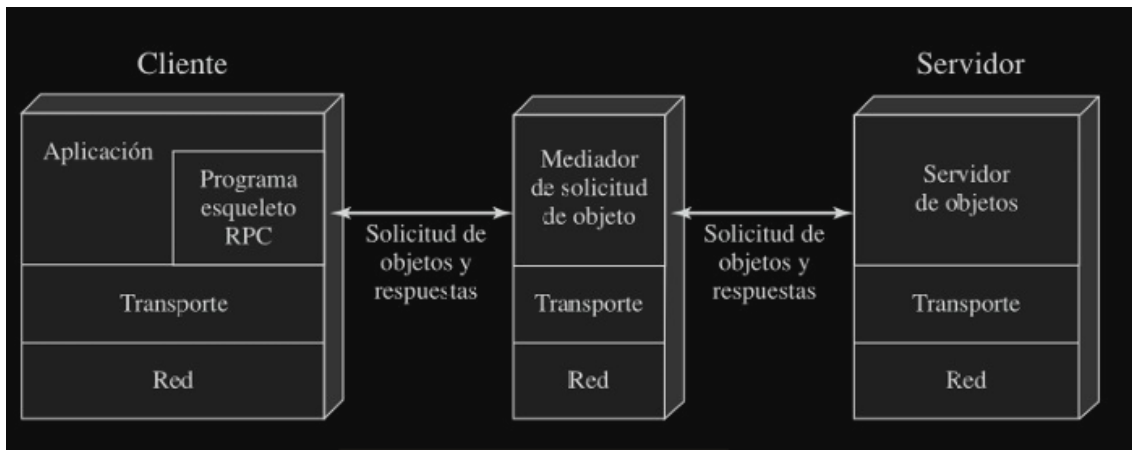
- **Función:** Proporciona una **manera y estilo de acceso uniforme a los recursos del sistema a través de todas las plataformas**. Permite construir aplicaciones que no solo se vean iguales, sino que usen el mismo método para acceder a los datos, sin importar su localización.
- **Beneficio:** Permite a una aplicación o usuario en el cliente acceder a una variedad de servicios en el servidor **sin preocuparse de las diferencias entre los servidores**. Esconde la **complejidad y disparidad de los diferentes protocolos de red y sistemas operativos**.
- **Rol Lógico:** El middleware permite que el sistema distribuido completo se vea como un conjunto de aplicaciones y recursos disponibles para los usuarios, sin que estos se preocupen por la localización de los datos o aplicaciones. Se sitúa entre todas las plataformas cliente y servidor y es responsable de **guiar las peticiones al servidor apropiado**.



Paso de Mensajes Distribuido

En sistemas de procesamiento distribuido donde las computadoras **no comparten la memoria principal**, se usan técnicas que se basan en el **paso de mensajes** para la comunicación entre procesos, en lugar de las que usan memoria compartida (como los semáforos).





- **Mecanismo Básico:** Un proceso **cliente** envía un **mensaje** con una **petición de servicio** a un proceso **servidor**, que realiza la petición y envía un **mensaje con la respuesta**.
- **Funciones Mínimas:** Solo se necesitan dos funciones: **Send (mandar)**, que especifica el destinatario y el contenido del mensaje; y **Receive (recibir)**, que indica de quién se desea recibir y provee un buffer para el mensaje entrante.

Fiable vs. No Fiable

- **Servicio fiable:** **Garantiza la entrega si es posible.** Hace uso de protocolos de transporte fiable, realizando comprobación de errores, acuse de recibo, retransmisión y reordenamiento de mensajes desordenados. Si falla el envío, se informa al proceso emisor.
- **Servicio no fiable:** Simplemente **envía el mensaje por la red, sin indicar el éxito ni el fracaso.** Reduce la complejidad y la sobrecarga de procesamiento y comunicación. Las aplicaciones deben gestionar su propia confirmación si la requieren.

Bloqueante vs. No Bloqueante

- **Primitivas no bloqueantes o asíncronas:** **No suspenden a un proceso** como resultado de realizar un *Send* o *Receive*. Permiten que el proceso continúe ejecutando.
 - *Send no bloqueante:* El control se devuelve tan pronto como el mensaje ha sido puesto en la cola o se ha realizado una copia. Se interrumpe al proceso emisor para informarle de que el buffer puede ser reutilizado.
 - *Receive no bloqueante:* El proceso continúa ejecutando; al llegar el mensaje se le informa con una interrupción o el proceso verifica el estado periódicamente.
 - *Ventaja/Desventaja:* Proporcionan uso eficiente y flexible, pero dificultan el chequeo y la depuración de programas por las secuencias

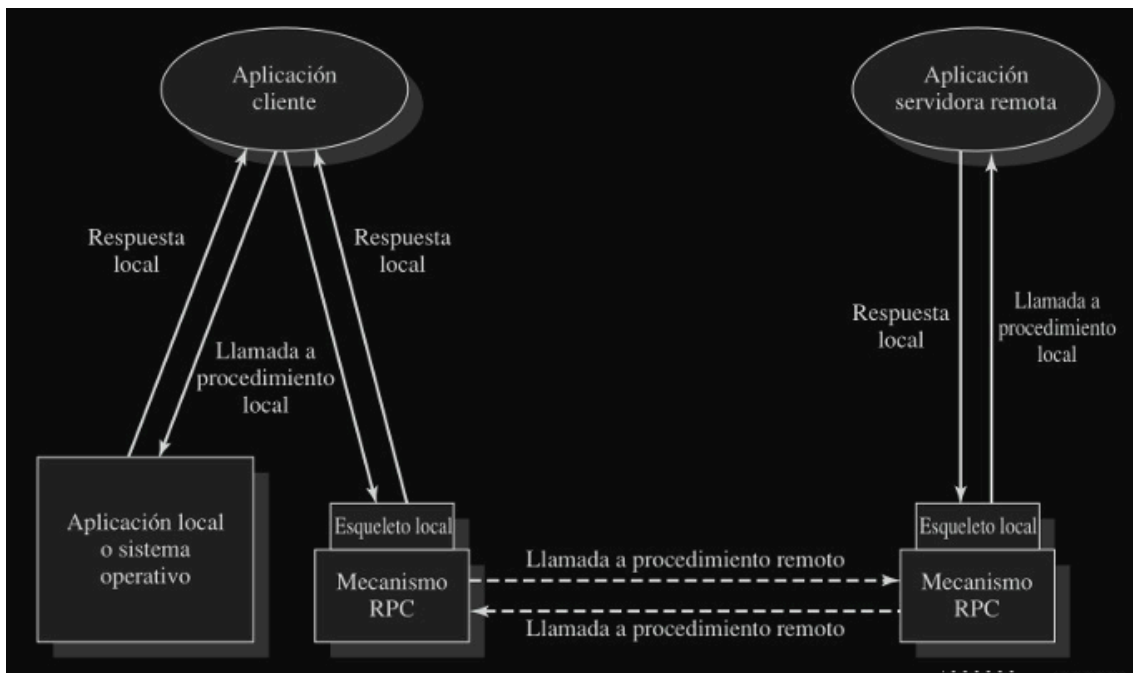
irreproducibles y dependientes del tiempo.

- **Primitivas bloqueantes o síncronas: No devuelven el control** hasta que la operación se ha completado.
 - *Send bloqueante*: No devuelve el control hasta que el mensaje ha sido transmitido (no fiable) o se ha recibido el acuse de recibo (fiable).
 - *Receive bloqueante*: No devuelve el control hasta que el mensaje ha sido situado en su correspondiente buffer.

Unidad 8: Clase 21 <Development>

Llamadas a Procedimiento Remoto (RPC)

Las **Llamadas a Procedimiento Remoto (RPC)** son una variante del modelo básico de **paso de mensajes**. Es un método ampliamente aceptado que permite a programas en diferentes máquinas interactuar usando la **sintaxis y semántica de llamadas a procedimiento**. El programa llamante y el llamado se comportan como si estuvieran en la misma máquina.



Ventajas del RPC

La popularidad del RPC se debe a que ofrece:

1. **Abstracción familiar**: Las llamadas a procedimiento son una abstracción ampliamente aceptada, útil y entendida.
2. **Interfaz definida**: Permite especificar las interfaces remotas como un conjunto de operaciones con nombre y tipos de datos definidos, facilitando la documentación y la **comprobación estática de errores** en los tipos de datos.

3. **Generación automática de código:** Se puede generar automáticamente el código de comunicación para una aplicación, ya que la interfaz es estandarizada y precisa.
4. **Portabilidad:** Los módulos cliente y servidor pueden moverse entre computadoras y sistemas operativos con mínimas modificaciones.

Enlace Cliente/Servidor

El **enlace** especifica cómo se establece la relación entre el procedimiento remoto y el programa llamante, formándose cuando las aplicaciones están listas para intercambiar datos.

- **Enlace no persistente (no permanente):** La conexión lógica se establece en el momento de la llamada y se **cierra inmediatamente** después de devolver los valores. Se utiliza para **conservar recursos** al no mantener información de estado. Sin embargo, la sobrecarga de establecer conexiones lo hace inapropiado para llamadas frecuentes.
- **Enlace persistente:** La conexión se **mantiene** después de la llamada y puede ser usada para futuras RPC. Si pasa un tiempo sin actividad, se finaliza. Reduce la sobrecarga de conexión para aplicaciones con llamadas repetidas.

RPC Síncronos vs. Asíncronos

Los conceptos son análogos a los mensajes bloqueantes y no bloqueantes.

- **RPC Síncrono:** Tradicional. Requiere que el proceso llamante **espere** hasta que el proceso llamado devuelva el valor. Se comporta como una llamada a subrutina. Es fácil de entender y programar, pero limita el paralelismo inherente en sistemas distribuidos, resultando en rendimientos más bajos.
- **RPC Asíncrono:** No bloquea al llamante, permitiendo al cliente **ejecutar en paralelo con el servidor**. Las respuestas se reciben cuando se requieren.

Mecanismos Orientados a Objetos

El diseño cliente/servidor ha adoptado la tecnología orientada a objetos.

- Clientes y servidores **mandan mensajes entre objetos**.
- El cliente manda una petición a un ****Mediador de Solicitud de Objeto (**Object Request Broker - ORB)**, que actúa como un **directorío de servicios remotos disponibles**.
- El mediador llama al objeto remoto apropiado, que atiende la petición y responde al mediador, devolviendo la información al cliente.
- Existen diseños que compiten: **Modelo de Componentes de Objeto (COM) de Microsoft** (base de OLE) y la **Arquitectura Común de Mediador de Solicitud de Objeto (CORBA)** (gran soporte de la industria: IBM, Apple, Sun, etc.).

Clusters

Un **Cluster** es un diseño de sistema de computación relativamente reciente. Un

cluster se define como un grupo de **computadoras completas e interconectadas**, que **trabajan juntas como un recurso de computación unificado** y que pueden **crear la ilusión de ser una única máquina**.

- **Computadora Completa:** Un sistema que puede ejecutarse por sí mismo, aparte del cluster.
- **Nodo:** Cada computadora de un Cluster se denomina nodo.

Beneficios y Objetivos de Diseño

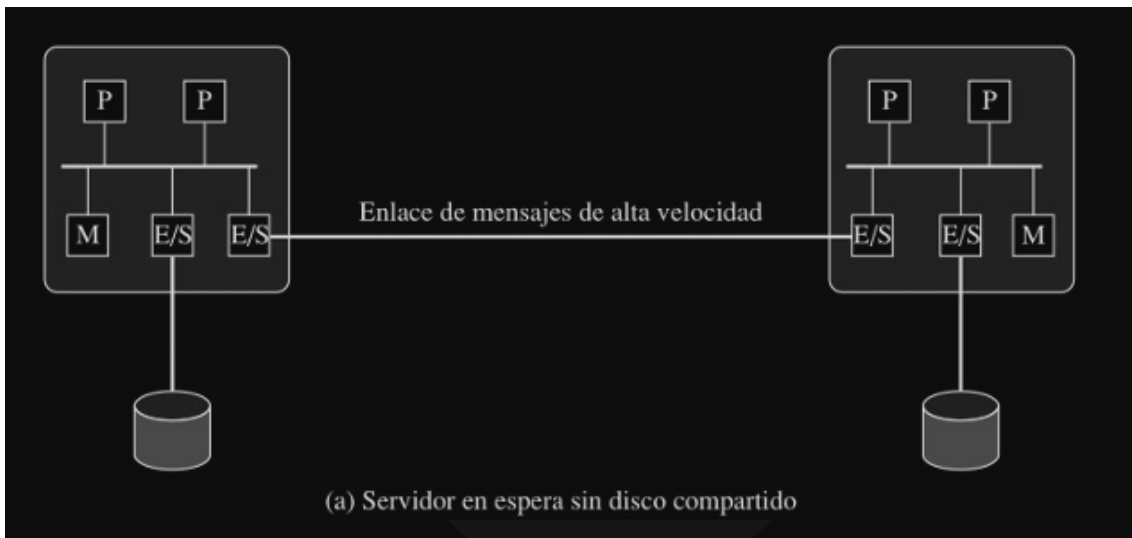
Los clusters son una alternativa al Multiprocesamiento Simétrico (SMP). Los beneficios son:

- **Escalabilidad absoluta:** Es posible crear un gran cluster que supere la potencia de las máquinas más grandes, pudiendo tener decenas o centenas de máquinas.
- **Escalabilidad incremental:** Es posible añadir nuevos sistemas al cluster en pequeños incrementos, permitiendo la expansión sin grandes actualizaciones.
- **Alta disponibilidad:** El fallo de un nodo no significa la pérdida del servicio, ya que cada nodo es una computadora en sí misma. El software a menudo gestiona automáticamente la **tolerancia a fallos**.
- **Relación precio/prestaciones:** Es posible lograr un poder computacional igual o mayor que una única máquina grande con **mucho menor coste**.

Configuración de los Clusters

La clasificación más sencilla se basa en si las computadoras comparten acceso a los discos.

1. **Servidor en espera sin disco compartido (Cluster con *Shared Nothing* para datos):**
 - Dos nodos se interconectan por un **enlace de mensajes de alta velocidad** (LAN o interconexión dedicada).
 - Cada nodo tiene sus propios discos.
 - **Figura (a): Configuraciones cluster** El diagrama muestra dos nodos, cada uno con múltiples Procesadores (\$P\$), Memoria (\$M\$), y E/S (\$E/S\$), conectado a un disco individual. Los nodos están unidos por un **Enlace de mensajes de alta velocidad**.



2. Cluster con disco compartido (*Shared Disk*):

- Existe un enlace de mensajes entre los nodos.
- Además, hay un **subsistema de discos** (a menudo **RAID**) que está **directamente enlazado con múltiples computadoras** del cluster. El RAID se usa para que la alta disponibilidad no se vea comprometida por el disco compartido.



Método de <i>Cluster</i>	Descripción	Beneficios	Limitaciones
Pasivo en Espera	En caso de fallo en el servidor primario, un servidor secundario toma control.	Fácil de implementar.	Alto coste debido a que el servidor secundario no está disponible para procesar otras tareas.
Secundario Activo	El servidor secundario también se utiliza para procesamiento de tareas	Coste reducido porque el servidor secundario puede ser utilizado para procesamiento.	Creciente complejidad.
Diferentes Servidores	Cada servidor tiene sus propios discos. Los datos se copian continuamente del servidor primario al secundario.	Alta disponibilidad.	Alta sobrecarga de red y de servidor debido a las operaciones de copia.

Método de <i>Cluster</i>	Descripción	Beneficios	Limitaciones
Servidores Conectados a Discos	Los servidores están unidos a los mismos discos, pero cada servidor posee sus propios discos. Si un servidor falla el otro servidor toma control de sus discos.	Sobrecarga de red y de servidores reducida debido a la eliminación de las operaciones de copia.	Normalmente requiere tecnologías de replicación de discos o RAID para compensar el riesgo de fallo de disco.
Servidores Comparten Discos	Varios servidores comparten acceso a disco de forma simultánea.	Baja sobrecarga de red y de servidores. Reducido riesgo de periodos de inactividad causados por fallos de disco.	Requiere software de gestión de cerrojos. Normalmente se utiliza con tecnologías de replicación de discos o RAID.

Aspectos de Diseño del SO en Clusters

Para aprovechar el hardware del cluster se necesitan mejoras en el SO:

Gestión de Fallos:

- **Alta Disponibilidad:** Ofrece alta posibilidad de que los recursos estén en servicio. Si falla un nodo, las peticiones en progreso se pierden y deben reintentarse, pero el SO no garantiza el estado de las transacciones parciales.
- **Tolerancia a Fallos:** Asegura que **todos los recursos están siempre disponibles**. Se logra con discos redundantes y mecanismos para deshacer transacciones incompletas.
- **Recuperación de Fallos (*Failover*):** Intercambiar una aplicación y sus datos de un sistema fallido a un sistema alternativo.
- **Restauración de Fallos (*Failback*):** Restaurar la aplicación y datos al sistema original una vez reparado.

Equilibrado de Carga (*Load Balancing*):

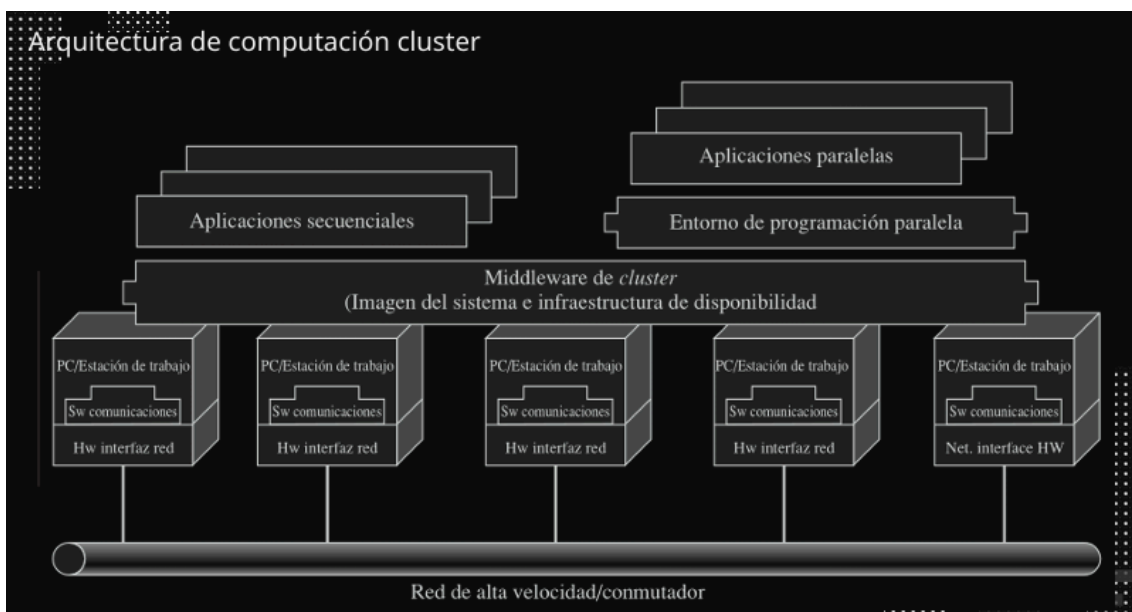
- El cluster debe tener la capacidad de **equilibrar la carga** entre todas las computadoras.
- Debe ser **escalable**, incluyendo automáticamente las nuevas computadoras en la planificación de aplicaciones.

Computación Paralela: Tres enfoques para ejecutar una única aplicación en paralelo:

- **Compilación Paralela:** Un compilador determina en tiempo de compilación qué partes se pueden ejecutar en paralelo.
- **Aplicaciones Paralelas:** El programador escribe la aplicación para el cluster y usa paso de mensajes para mover datos. Es mejor para explotar el cluster, pero pone mucha carga en el programador.
- **Computación Paramétrica:** El algoritmo se ejecuta muchas veces, cada una con un **conjunto diferente de condiciones o parámetros iniciales** (ej. modelos de simulación).

Arquitectura de un Cluster

- Las computadoras están conectadas a una **LAN de alta velocidad o conmutador hardware**.
- Se instala una **capa de software middleware** que permite la operación del cluster.
- El middleware proporciona una **Imagen Única del Sistema (Single-System Image**)** al usuario.



Servicios y Funciones Deseables para una Imagen Única del Sistema

Los elementos que se preocupan de proporcionar una imagen única del sistema son:

- **Punto de entrada único:** Un usuario se autentifica en el cluster, no en una computadora.
- **Jerarquía de ficheros única:** Los usuarios ven una sola jerarquía de directorios.
- **Punto de control único:** Un nodo por defecto se encarga de gestionar y controlar el cluster.
- **Red virtual única:** Cualquier nodo puede acceder a cualquier otro punto del cluster, operando sobre una red virtual única.

- **Espacio de memoria único:** La memoria compartida distribuida permite a los programas compartir variables.
- **Sistema de control de trabajos único:** Con un planificador, el usuario envía su trabajo sin especificar la computadora.
- **Interfaz de usuario único:** Todos los usuarios tienen un interfaz gráfico común.
- **Espacio de E/S único:** Cualquier nodo puede acceder remotamente a cualquier periférico de E/S o disco sin conocer su localización física.
- **Espacio de procesos único:** Se usa un esquema uniforme de identificación. Un proceso puede crear o comunicarse con cualquier otro proceso en un nodo remoto.

Los elementos que mejoran la disponibilidad son:

- **Puntos de control:** Salvan periódicamente el estado y los resultados intermedios para permitir la recuperación tras un fallo.
- **Migración de procesos:** Permite el balanceado de carga.

Cluster frente a SMP

Tanto los clusters como el multiprocesamiento simétrico proporcionan una configuración con múltiples procesadores para dar soporte a aplicaciones con alta demanda. Ambas soluciones están disponibles en el mercado, aunque SMP ha estado presente durante más tiempo.

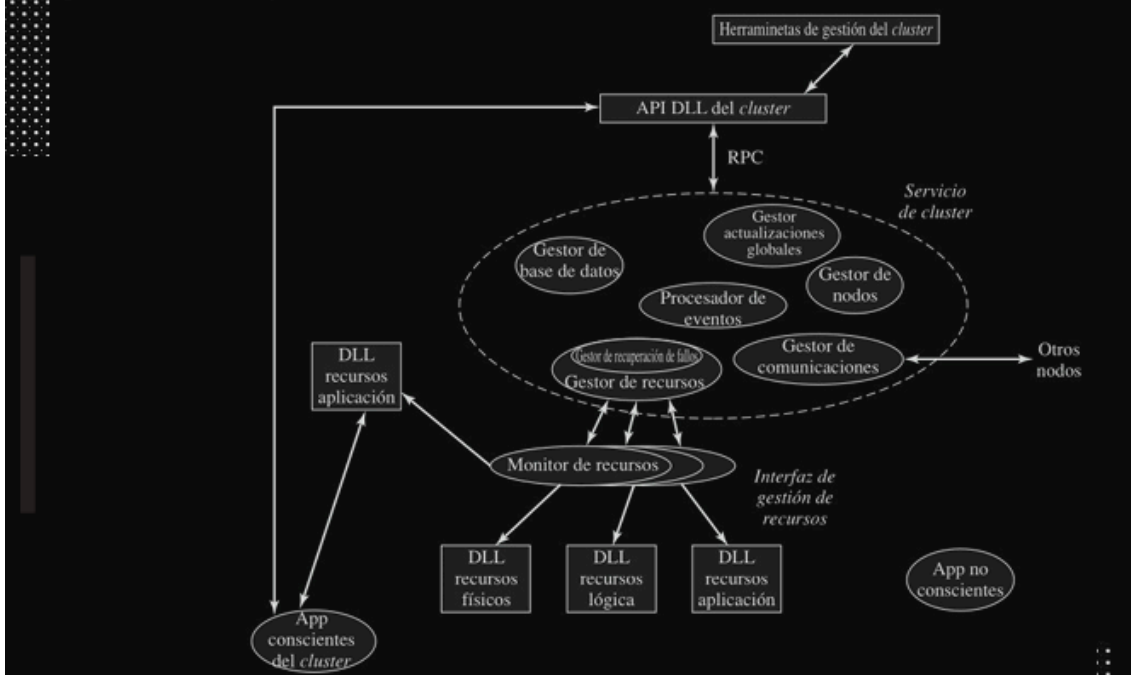
Servidor Clusters de Windows (Wolfpack)

Es un cluster de tipo **nada compartido (shared nothing)**. Cada volumen de disco y otros recursos son **propiedad de un único sistema** a la vez.

Conceptos Clave:

- **Servicio Cluster:** Colección de software en cada nodo que gestiona la actividad del cluster.
- **Recurso:** Elemento gestionado (ej. dispositivos hardware, volúmenes lógicos, direcciones TCP/IP, aplicaciones, bases de datos). Se implementan como **Bibliotecas Dinámicas (DLL)**.
- **En línea (online):** Un recurso está en línea en un nodo cuando está proporcionando servicio en ese nodo.
- **Grupo:** Colección de recursos gestionada como una unidad. Combina recursos en unidades mayores para **recuperación de fallos y balanceado de carga**.

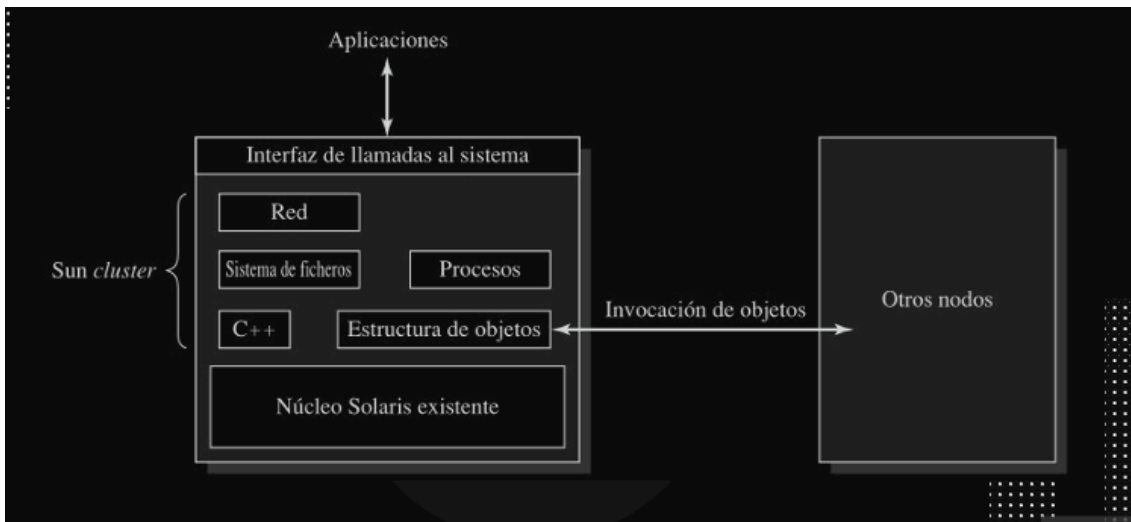
Diagrama de bloques del Windows Cluster Server



- **Gestor de nodo:** Mantiene la pertenencia al cluster. Envía mensajes a otros gestores de nodo. Si detecta la pérdida de mensajes de otro nodo, se le quita del cluster y **sus grupos activos se transfieren** a otros nodos activos.
- **Gestor de la base de datos de configuración:** Guarda la base de datos de configuración (recursos, grupos, pertenencia). Cooperan para mantener una **imagen consistente**.
- **Gestor de recursos/Gestor de recuperación de fallos:** Toma decisiones sobre grupos e inicia acciones como **recuperar fallos**. Negocia la distribución de grupos del sistema fallido al resto de sistemas activos. Se puede configurar un grupo con un **propietario preferente** para que regrese a ese nodo tras un reinicio.

Sun Clusters

Es un sistema operativo distribuido, construido como un conjunto de **extensiones del sistema UNIX Solaris**. Proporciona una **visión unificada**: los usuarios y aplicaciones ven el cluster como una única computadora ejecutando Solaris.

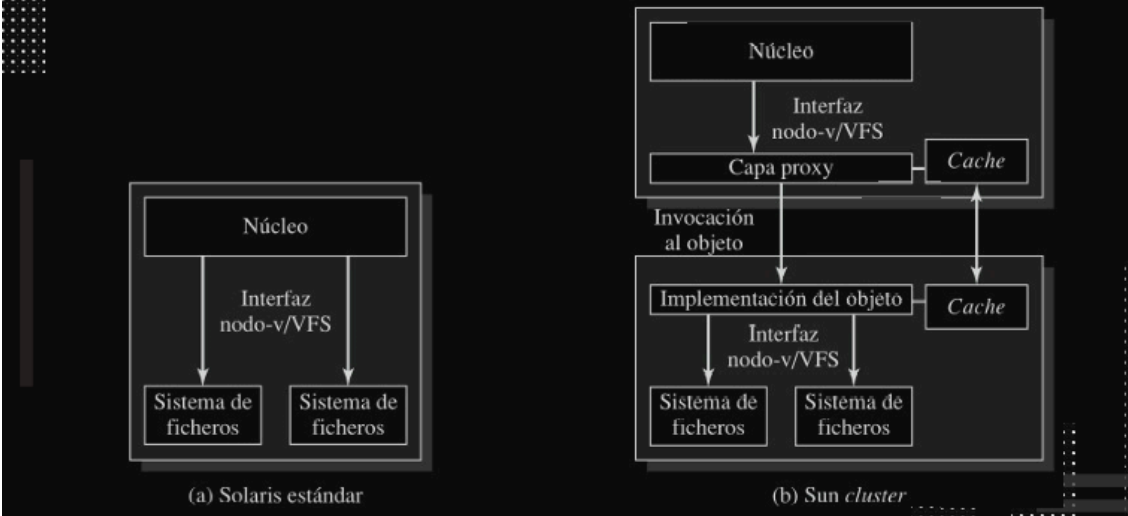


Componentes Principales:

1. **Soporte de objetos y comunicaciones:** Utiliza el modelo de objetos **CORBA** para definir objetos y RPC. El **Lenguaje de Definición de Interfaces (IDL)** de CORBA se usa para interfaces, y los objetos se implementan en **C++**.
2. **Gestión de procesos:** **Extiende las operaciones de los procesos** haciendo que su localización sea **transparente** al usuario. Mantiene una visión global con **identificador único por proceso**. Un proceso puede **moverse** de un nodo a otro para **balanceado de carga y recuperación de fallos**.
3. **Redes:** Adopta un filtro de paquetes para enviar los paquetes al nodo apropiado y realizar el procesamiento del protocolo allí. Externamente, parece un solo servidor con una **única dirección IP**. Las conexiones entrantes se **balancean** entre los nodos. Aplica 3 enfoques para el manejo de tráfico:
 - **Realizar todo el procesamiento del protocolo de red en un único nodo.**
 - **Asignar una dirección única a cada nodo y ejecutar los protocolos de red en cada nodo.**
 - **Utilizar un filtro de paquetes para enviar los paquetes al nodo apropiado y realizar el procesamiento del protocolo en dicho nodo.**
4. **Sistema de ficheros distribuido global:** Es el elemento más importante. Proporciona una **interfaz uniforme sobre los ficheros distribuidos**; un proceso puede abrir un fichero localizado en **cualquier sitio del cluster**.
 - Se basa en los conceptos **nodo-v** (nodo virtual) y **vfs** (sistema de ficheros virtual) de Solaris.
 - Se utiliza un **sistema de ficheros proxy** sobre el Solaris existente. Las operaciones **vfs/nodo-v** se convierten en **invocaciones a objetos** en la capa proxy, sin modificar el núcleo ni el sistema de ficheros existente.

Se utiliza una **cache** para reducir las invocaciones a objetos remotos.

Extensiones del sistema de ficheros de Sun Cluster.



Clusters de Beowulf y Linux

El proyecto **Beowulf** investigó el potencial de los **clusters de PC para computación de altas prestaciones con mínimo coste**. Hoy es una tecnología cluster ampliamente implementada, a menudo usando **Linux**.

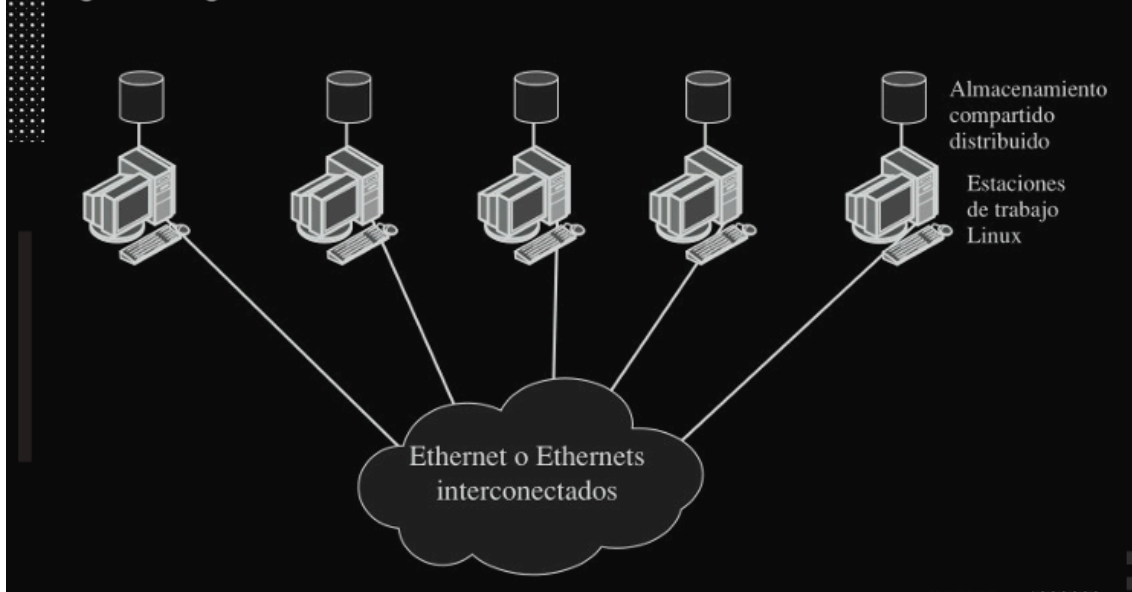
Características de Beowulf

- Componentes genéricos disponibles en el mercado.
- **Procesadores dedicados.**
- Una **red privada y dedicada** (LAN/WAN o combinación).
- **Ningún componente propio.**
- Basado en **software gratuito disponible.**
- Fácilmente replicable para múltiples vendedores
- E/S escalable.
- Retorno del diseño y de las mejoras de la comunidad.

Configuración Genérica:

- Una serie de **Estaciones de trabajo Linux** (posiblemente con diferente hardware).
- **Almacenamiento compartido distribuido** disponible para acceso distribuido.
- Los nodos se interconectan con **Ethernet o Ethernets interconectados** (conmutadores).

Configuración genérica de Beowulf.



Software Beowulf (Extensiones de Linux)

El software se implementa como una ampliación de distribuciones Linux gratuitas. Cada nodo ejecuta su propia copia del **núcleo de Linux**. Extensiones clave:

- **Espacio de procesos distribuido de Beowulf (BPROC):** Expande el espacio de identificadores de proceso en múltiples nodos, permitiendo **iniciar procesos en nodos remotos** sin iniciar sesión y haciendo que todos los procesos sean **visibles en la tabla de procesos del cluster** (Imagen única de sistema).
- **Unión de canales Ethernet Beowulf:** Mecanismo que **une múltiples redes de bajo coste en una sola red lógica de mayor ancho de banda**, permitiendo **balancear la carga** sobre múltiples Ethernet.
- **Pvmsync:** Entorno de programación que proporciona mecanismos de **sincronización y objetos de datos compartidos**.
- **EnFuzion:** Conjunto de herramientas para **computación paramétrica**. Simula usuarios robot que acceden a los clientes del cluster, configurando cada trabajo para ejecutarse en un escenario único.